

Java Programming

Dr. Pradeep K V

Associate Professor (Sr.)

School of Computer Science and Engineering

VIT - Chennai





- It is the quality of dealing with ideas rather than events.
- For **Example** : Consider **E-mail**, complex details such as what happens as soon as you send an e-mail, the protocol your e-mail server uses are hidden from the user.
- Therefore, to send an e-mail, just need to type the **content**, mention the **address** of the receiver, and click send.
- In OOP : **Abstraction** is a process of hiding the implementation details from the user, only the functionality will be provided to the user.
- In other words, User will have the information on what the object does instead of how it does it.
- In Java, **Abstraction** is achieved using **Abstract** classes and **Interfaces**.

A class which contains the **abstract** keyword in its declaration is known as **Abstract** class.

- Abstract classes may or may not contain **abstract** methods, i.e., methods without body (`public void get();`)
- But, if a class has at least one **abstract** method, then the class must be declared abstract.
- If a class is declared abstract, it cannot be instantiated.
- To use an abstract class, you have to inherit it from another class, provide implementations to the abstract methods in it.
- If you inherit an **abstract** class, you have to provide implementations to all the abstract methods in it.
- The **abstract** keyword is a *non-access* modifier, used for classes and methods

Example-1: Abstract Class I



```
/* File name : Employee.java */

public abstract class Employee {
    private String name, address;
    private int number;

    public Employee(String name, String address, int number) {
        System.out.println("Constructing an Employee");
        this.name = name;    this.address = address;
        this.number = number;
    }

    public double computePay() {
        System.out.println("Inside Employee computePay");
        return 0.0;
    }

    public void mailCheck() {
        System.out.println("Mailing a check to " +
                           this.name + " " + this.address);
    }

    public String toString() {
        return name + " " + address + " " + number;
    }
}
```



Example-1: Abstract Class II



```
public String getName() {    return name;    }

public String getAddress() {    return address;    }

public void setAddress(String newAddress) {    address = newAddress;    }

public int getNumber() {    return number;    }
}

/* File name : AbstractDemo.java */

public class AbstractDemo {
    public static void main(String [] args) {
        /* Following is not allowed and would raise error */
        Employee e = new Employee("Pradeep K V.", "Vandalur - Chennai", 50);
        System.out.println("\n Call mailCheck using Employee reference--");
        e.mailCheck();
    }
}

/* Exception in thread "main" java.lang.Error: Unresolved
 * compilation problem: Cannot instantiate the type Employee    */
```



Example-2: Abstract Class and Methods



```
// Abstract class
abstract class Animal {
    // Abstract method (does not have a body)
    public abstract void animalSound();
    // Regular method
    public void sleep() {
        System.out.println("Zzz");
    }
}

// Subclass (inherit from Animal)
class Pig extends Animal {
    public void animalSound() {
        // The body of animalSound() is provided here
        System.out.println("The pig says: wee wee");
    }
}

class Main {
    public static void main(String[] args) {
        Pig myPig = new Pig();          // Create a Pig object
        myPig.animalSound();
        myPig.sleep();
    }
}
```



Example-3: Abstract Class and Methods I



```
abstract class EMPLOYEE {  
    private String Name;  
    private int Number;  
  
    public EMPLOYEE(String name, int number) {  
        System.out.println("Constructing an Employee");  
        this.Name = name;  
        this.Number = number;  
    }  
    public String getName() {    return Name;    }  
  
    public abstract double computePay();    // Remainder of class definition  
}  
  
class SAL extends EMPLOYEE {  
    private double salary;    // Annual salary  
  
    public SAL(String name, int number, double newsalary) {  
        super(name, number);  
        salary = newsalary;  
    }  
  
    public double computePay() {
```



```
        System.out.print("The Salary for " + getName() + " is : " );  
        return salary+2000;  
    }  
}  
  
class Emp{  
    public static void main(String[] args){  
        SAL e = new SAL("Pradeep K V", 3, 2000.0);  
        System.out.println(e.computePay());  
    }  
}  
  
/* Constructing an Employee  
 * The Salary for Pradeep K V is : 4000.0    */
```

- Another way to achieve abstraction in Java, is with **interfaces**.
- An **interface** is a completely "abstract class" that is used to group related methods with empty bodies
- An **interface** is a reference type. It is similar to class.
- It is a collection of **abstract** methods.
- A class implements an **interface**, thereby inheriting the abstract methods of the interface.
- It is similar to writing a class. But a class describes the **attributes** and **behaviors** of an object. And an interface contains **behaviors** that a class implements.

```
// interface
interface Animal {

    public void animalSound(); // interface method (does not have a body)

    public void run();          // interface method (does not have a body)
}
```



An interface is **similar** to a class in the following ways :

- An interface can contain any number of methods.
- An interface is written in a file with a **.java** extension, with the name of the interface matching the name of the file.
- The byte code of an interface appears in a **.class** file.
- Interfaces appear in **packages**, and their corresponding bytecode file must be in a directory structure that matches the package name.

However, an interface is **different** from a class in several ways, including :

- You cannot instantiate an interface.
- It does not contain any constructors and the methods in an interface are **abstract**.
- It cannot contain instance fields. The only fields that can appear in an interface must be declared both **static** and **final**.
- It is not extended by a class; it is implemented by a class.
- It can extend multiple interfaces.



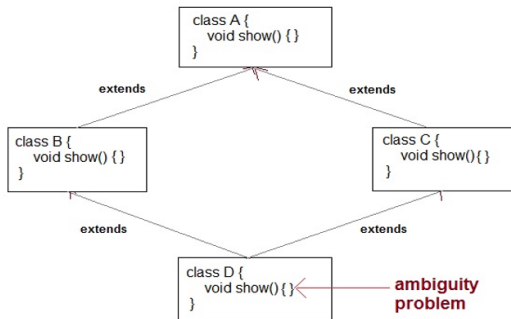
Keyword Interface and its properties.

So, interface is a keyword to achieve multiple inheritance in java. let's have a small look over keyword interface.

CLASS	INTERFACE
Supports only multilevel and hierarchical inheritances but not multiple inheritance	Supports all types of inheritance: multilevel, hierarchical and multiple
"extends" keyword should be used to inherit	"implements" keyword should be used to inherit
Should contain only concrete methods (methods with body)	Should contain only abstract methods (methods without body)
The methods can be of any access specifier (all the four types)	The access specifier must be public only
Methods can be final and static	Methods should not be final and static
Variables can be private	Variables should be public only
Can have constructors	Cannot have constructors
Can have main() method	Cannot have main() method as main() is a concrete method

Multiple inheritance

- To remove ambiguity.
- To provide more maintainable and clear design.



Multiple inheritance

- *Multiple inheritance* allows a class to be derived from two or more classes, inheriting the members of all parents
- Collisions, such as the same variable name in two parents, have to be resolved
- Java does not support this directly, rather it uses *Interfaces*

```
interface FirstInterface {  
    public void myMethod(); // interface method  
}  
  
interface SecondInterface {  
    public void myOtherMethod(); // interface method  
}  
  
class DemoClass implements FirstInterface, SecondInterface {  
    public void myMethod() {  
        System.out.println("myMethod() from First Interface");  
    }  
  
    public void myOtherMethod() {  
        System.out.println("myOtherMethod from Second Interface");  
    }  
}  
  
public class Interface_Demo {  
    public static void main(String[] args) {  
        DemoClass myObj = new DemoClass();  
        myObj.myMethod();  
        myObj.myOtherMethod();  
    }  
}  
  
// myMethod() from First Interface  
// myOtherMethod from Second Interface
```

Exampe-2 : Java Interface



```
// Interface
interface Animals {
    public void animalSound(); // interface method (does not have a body)
    public void sleep();       // interface method (does not have a body)
}

class Cat implements Animals {
    public void animalSound() { System.out.println("The Cat makes meow meow"); }
    public void sleep() { System.out.println("Zzz"); }
}

class Dog implements Animals {
    public void animalSound() { System.out.println("The Dog makes bow bow"); }
    public void sleep() { System.out.println("Zzz"); }
}

class Interface_Animal{
    public static void main(String[] args) {
        Cat c = new Cat();    c.animalSound();
        Dog d = new Dog();    d.animalSound();
    }
}

// The Cat makes meow meow
// The Dog makes bow bow
```



```
interface Language {  
    void getName(String name);  
}  
  
class ProgrammingLanguage implements Language {  
    public void getName(String name) {  
        System.out.println("Programming Language: " + name);  
    }  
}  
  
public class Interface_Language {  
    public static void main(String[] args) {  
        ProgrammingLanguage language = new ProgrammingLanguage();  
        language.getName("Object Oriented Programming using Java");  
    }  
}  
  
// Programming Language: Object Oriented Programming using Java
```



```
interface Polygon { void getArea(int length, int breadth); }

class Rectangle implements Polygon {
    public void getArea(int length, int breadth) {
        System.out.println("The area of the rectangle is " + (length * breadth));
    }
}

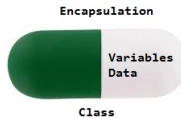
class Square implements Polygon {
    public void getArea(int s1, int s2 ) {
        System.out.println("The area of the square is " + (s1 * s2));
    }
}

public class Interface_Figures {
    public static void main(String[] args) {
        // create an object
        Rectangle r1 = new Rectangle();
        r1.getArea(5, 6);

        Square s1 = new Square();
        s1.getArea(4,4);
    }
}

// The area of the rectangle is 30
// The area of the square is 16
```

- It is a process of wrapping code and data together into a single unit.
- For Example, a capsule which is mixed of several medicines.



- The meaning of **Encapsulation**, is to make sure that "sensitive" data is hidden from users.
- **Advantages** :
 - It provides us to control over the data.
 - It is a way to achieve data hiding.
 - Easy to Test (Unit Testing).

```
class Area {  
    // fields to calculate area  
    int length, breadth;  
  
    // constructor to initialize values  
    Area(int length, int breadth) {  
        this.length = length;  this.breadth = breadth;  
    }  
  
    // method to calculate area  
    public void getArea() {  
        int area = length * breadth;  
        System.out.println("Area: " + area);  
    }  
}  
  
class AreaEncapsulation {  
    public static void main(String[] args) {  
        // create object of Area  
        // pass value of length and breadth  
        Area rectangle = new Area(5, 6);  
        rectangle.getArea();  
    }  
}
```


Example-2: Encapsulation



```
// Data hiding using the private specifier

class Person {

    private int age;    // private field

    public int getAge() { return age; }    // getter method

    public void setAge(int age) { this.age = age; }    // setter method
}

class Person_Encapsulate{
    public static void main(String[] args) {

        Person p1 = new Person();    // create an object of Person

        p1.setAge(24);    // change age using setter

        // access age using getter
        System.out.println("My age is " + p1.getAge());
    }
}

// My age is 24
```



Example-3: Encapsulation



// STUD.java in which Java class is fully encapsulated class.

```
public class STUD{  
    private String Name;    //private data member  
    public String getName(){ //getter method for name  
        return Name;  
    }  
    public void setName(String name){ //setter method for name  
        this.Name=name;  
    }  
}
```

// Student.java in which Instantiated STUD.

```
class Student{  
    public static void main(String[] args){  
        //creating instance of the encapsulated class  
        STUD s=new STUD();  
        //setting value in the name member  
        s.setName("Pradeep K V");  
        //getting value of the name member  
        System.out.println(s.getName());  
    }  
}
```

// Pradeep .K .V



Example-4: Encapsulation I



// Account.java : A Account class which is a fully encapsulated class.

```
class Account {  
    private long acc_no;           //private data members  
    private String name,email;  
    private float amount;  
  
    //public getter and setter methods  
    public long getAcc_no() { return acc_no; }  
  
    public void setAcc_no(long acc_no) { this.acc_no = acc_no; }  
  
    public String getName() { return name; }  
  
    public void setName(String name) { this.name = name; }  
  
    public String getEmail() { return email; }  
  
    public void setEmail(String email) { this.email = email; }  
  
    public float getAmount() { return amount; }  
  
    public void setAmount(float amount) { this.amount = amount; }  
}
```



```
// Acc_Encapsulation.java : Testing the encapsulated class Account.
public class Acc_Encapsulation {
    //A Java class to test the encapsulated class Account.
    public static void main(String[] args) {
        //creating instance of Account class
        Account acc=new Account();
        //setting values through setter methods
        acc.setAcc_no(9445825675L);

        acc.setName("Pradeep K V");

        acc.setEmail("pradeep.kv@gmail.com");

        acc.setAmount(5000000f);
        //getting values through getter methods
        System.out.println(acc.getAcc_no()+" "+acc.getName()+" "+acc.getEmail()+
    }
}

// 9445825675 Pradeep K V pradeep.kv@gmail.com 500000.0
```



- Inheritance can be defined as the process where one class acquires the properties (methods and fields) of another.
- It is an important part of OOPs (Object Oriented programming system).
- The class which inherits the properties of other is known as **subclass** (derived class, child class).
- The class whose properties are inherited is known as **superclass** (base class, parent class).
- Inheritance represents the **IS-A** relationship which is also known as a **Parent-Child** relationship.
- The idea behind inheritance is to create new(Child/Sub) classes that are built upon existing(Parent/Super) classes. By Inheriting, can reuse **methods** and **fields** of the Parent class. Moreover, you can add new methods and fields in your current class also.

extends is the keyword used to inherit the properties of a class.

Syntax :

```
class Super { // Parent/Base Class
    ....
    ....
}

class Sub extends Super { // Child/Derived Class
    ....
    ....
}
```

Note

A subclass inherits all the members (fields, methods, and nested classes) from its superclass. Constructors are not members, so they are not inherited by subclasses, but the constructor of the superclass can be invoked from the subclass.



```
class Employee{
    float salary=40000;
}

class Programmer extends Employee{
    int bonus=10000;

    public static void main(String args[]){
        Programmer p=new Programmer();
        System.out.println("Programmer salary is:"+p.salary);
        System.out.println("Bonus of Programmer is:"+p.bonus);
    }
}

/*  Programmer salary is:40000.0
   *  Bonus of programmer is:10000    */
```


Example-2 : Inheritance



```
class Vehicle {
    protected String brand = "Ford";           // Vehicle attribute
    public void honk() {                         // Vehicle method
        System.out.println("Tuut, tuut!");
    }
}

class Car extends Vehicle {
    private String modelName = "Mustang";      // Car attribute
    public static void main(String[] args) {

        Car myCar = new Car();

        myCar.honk();

        System.out.println(myCar.brand + " " + myCar.modelName);
    }
}

/* Tuut, tuut!
   * Ford Mustang */
```



```
class Calculation {
    int z;
    public void addition(int x, int y) {
        z = x + y;
        System.out.println("The sum :"+z);
    }
    public void Subtraction(int x, int y) {
        z = x - y;
        System.out.println("The difference :"+z);
    }
}

public class My_Calculation extends Calculation {
    public void multiplication(int x, int y) {
        z = x * y;
        System.out.println("The product o:"+z);
    }
    public static void main(String args[]) {
        int a = 20, b = 10;
        My_Calculation C = new My_Calculation();
        C.addition(a, b);      C.Subtraction(a, b);      C.multiplication(a, b);
    }
}

/* The sum :30
 * The difference :10
 * The product :200 */
```

```
//Java program to illustrate the concept of inheritance

class Bicycle {           // base class
    public int gear, speed;    // the Bicycle class has two fields

    public Bicycle(int gear, int speed) { // Constructor
        this.gear = gear;    this.speed = speed;
    }

    public void applyBrake(int decrement) { speed -= decrement; }

    public void speedUp(int increment) { speed += increment; }

    // toString() method to print info of Bicycle
    public String toString() {
        return("No of gears are "+gear +"\n"
            + "speed of bicycle is "+speed);
    }
}
```

```
class MountainBike extends Bicycle {           // derived class

    public int seatHeight;    // MountainBike class has one field

    public MountainBike(int gear,int speed, int startHeight) {
        super(gear, speed); seatHeight = startHeight;
    }

    public void setHeight(int newValue) {
        seatHeight = newValue;
    }

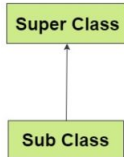
    // overriding toString() method of Bicycle to print more info
    @Override
    public String toString() {
        return (super.toString()+ "\nseat height is "+seatHeight);
    }

}
```

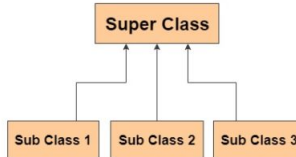
```
public class Test {                                // driver class
    public static void main(String args[]) {
        MountainBike mb = new MountainBike(3, 100, 25);
        System.out.println(mb.toString());
    }
}
```

```
/*      No of gears are 3
*       speed of bicycle is 100
*       seat height is 25          */
```

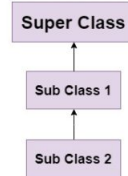
Single Inheritance



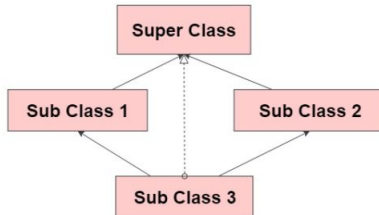
Hierarchical Inheritance



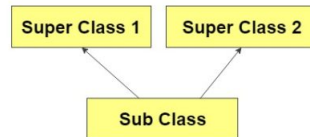
MultiLevel Inheritance

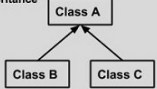
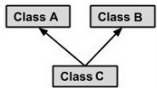


Hybrid Inheritance



Multiple Inheritance



Single Inheritance  <pre> graph BT B[Class B] --> A[Class A] </pre>	<pre> public class A { } public class B extends A { } </pre>
Multi Level Inheritance  <pre> graph BT C[Class C] --> B[Class B] B --> A[Class A] </pre>	<pre> public class A { } public class B extends A { } public class C extends B { } </pre>
Hierarchical Inheritance  <pre> graph BT B[Class B] --> A[Class A] C[Class C] --> A </pre>	<pre> public class A { } public class B extends A { } public class C extends A { } </pre>
Multiple Inheritance  <pre> graph BT A[Class A] --> C[Class C] B[Class B] --> C </pre>	<pre> public class A { } public class B { } public class C extends A,B { } // Java does not support multiple Inheritance </pre>

Example-1 : Single Inheritance



```
class Animal{
    void eat() {    System.out.println("eating...");    }
}

class Dog extends Animal{
    void bark() {    System.out.println("barking...");    }
}

class SingletInheritance
{
    public static void main(String args[])
    {
        Dog d=new Dog();
        d.bark();
        d.eat();
    }
}

    /* barking...
       * eating...    */
```



Example-1 : Single Inheritance



```
class Animal{
    void eat() {    System.out.println("eating...");    }
}

class Dog extends Animal{
    void bark() {    System.out.println("barking...");    }
}

class SingletInheritance
{
    public static void main(String args[])
    {
        Dog d=new Dog();
        d.bark();
        d.eat();
    }
}

    /* barking...
       * eating...    */
```



Example-2 : Multilevel Inheritance



```
class Animal {  
    void eat() {    System.out.println("eating...");    }  
}  
  
class Dog extends Animal {  
    void bark() {    System.out.println("barking...");    }  
}  
  
class BabyDog extends Dog {  
    void weep() {    System.out.println("weeping...");    }  
}  
class MultilevelInheritance {  
    public static void main(String args[]) {  
        BabyDog d = new BabyDog();  
        d.weep();    d.bark();    d.eat();  
    }  
}  
  
    /* weeping...  
    *  barking...  
    *  eating...    */
```



```
class Animal{
    void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
    void bark(){System.out.println("barking...");}
}
class Cat extends Animal{
    void meow(){System.out.println("meowing...");}
}
class HierarchicalInheritance{
    public static void main(String args[]){
        Cat c=new Cat();
        c.meow();    c.eat();
        //c.bark();           //C.T.Error (Not permitted to access)
    }

    /*    meowing...
     *    eating...        */
}
```

Why multiple inheritance is not supported in java?

- To reduce the complexity and simplify the language, **Multiple inheritance is not supported in java.**
- **Example** : Consider a scenario where A, B, and C are three classes. The C class inherits A and B classes. If A and B classes have the same method and you call it from child class object, there will be ambiguity to call the method of A or B class.
- Since Compile-time errors are better than Runtime errors, java renders compile-time error if you inherit 2 classes. So whether you have same method or different, there will be compile time error.

Example-4 : Multiple Inheritance



```
class A{  
    void msg(){System.out.println("Hello");}  
}  
  
class B{  
    void msg(){System.out.println("Welcome");}  
}
```

```
class C extends A,B{           //suppose if it were // ✗ Not allowed  
  
    public static void main(String args[]){  
        C obj=new C();  
        obj.msg();             //Now which msg() method would be invoked?  
    }  
}
```

// Compile Time Error

Which `msg()` should Java call?

A.msg() → Hello
B.msg() → Welcome

Java cannot unambiguously decide.



2. How does an interface solve this?

Java allows a class to **implement multiple interfaces**.

```
interface A {  
    void msg();  
}  
  
interface B {  
    void msg();  
}  
  
class C implements A, B {  
  
    public void msg() {  
        System.out.println("Hello and Welcome");  
    }  
}
```

Output

Hello and Welcome

```
public class Test {  
    public static void main(String[] args) {  
  
        C obj = new C();  
        obj.msg();  
    }  
}
```

2. How does an interface solve this?

Java allows a class to **implement multiple interfaces**.

Why is there no ambiguity?

Because **A** and **B** only specify the requirement:

```
void msg();
```

They don't provide two competing implementations.

C provides **one implementation**:

```
public void msg() {  
    System.out.println("Hello and Welcome");  
}
```

So Java knows exactly which **msg()** to execute.

If a class have an entity reference, it is known as **Aggregation**.
It represents **HAS-A** relationship.

For **Example** : Employee object contains many informations such as **id**, **name**, **email-Id** etc. It contains one more object named **address**, which contains its own informations such as **city**, **state**, **country**, **zipcode** etc. as given below.

```
class Employee{  
    int id;  
    String name;  
    Address address;  
    ...  
}
```

Here, Employee has an Address object/reference.

//Address is a class

Employee has an entity reference address, Therefore, Employee **HAS-A** address.
Why Aggregation? (Code Reusability)

When one class contains a reference/object of another class, we say it has a HAS-A relationship.



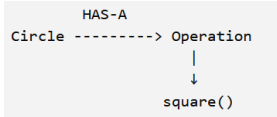
Example-1 : Aggregation



```
class Operation{  
    int square(int n){ return n*n; }  
}
```

```
class Circle{  
    Operation op;  
    double pi=3.14;  
  
    double area(int radius){  
        op=new Operation();  
        //code reusability (i.e. delegates the method call).  
        int rsquare=op.square(radius);  
        return pi*rsquare;  
    }  
  
    public static void main(String args[]){  
        Circle c=new Circle();  
        double result=c.area(5);  
        System.out.println(result);  
    }  
}
```

//aggregation
// Here, Circle contains a reference to an
Operation
object. i.e. Circle HAS-A Operation



// Output:78.5



```
class Address {
    String city,state,country;

    public Address(String city, String state, String country) {
        this.city = city;  this.state = state;
        this.country = country;
    }
}

class Emp_Aggregation {
    int id;
    String name;
    Address address;

    public Emp_Aggregation(int id, String name,Address address) {
        this.id = id;  this.name = name;
        this.address=address;
    }

    void display(){
        System.out.print(id+" "+name);
        System.out.println(address.city+" "+address.state+" "+address.country);
    }
}
```

```
public static void main(String[] args) {  
    Address address1=new Address("Kurnool","AP","India");  
    Address address2=new Address("Chennai","TN","India");  
  
    Emp_Aggregation e=new Emp_Aggregation(50396,"Anna",address1);  
    Emp_Aggregation e2=new Emp_Aggregation(50394,"Biju",address2);  
  
    e.display();  
    e2.display();  
}  
  
    // 50396 Pradeep K V Kurnool AP India  
    // 50394 Nachiyappan Chennai TN India
```

- The **super** keyword is similar to **this** keyword.
- It is used to differentiate the members of superclass from the members of subclass, if they have same names.
- It is used to invoke the superclass constructor from subclass.

```
class Super_class {  
    int num = 20;  
    // display method of superclass  
    public void display() {  
        System.out.println("This is the display method of superclass");  
    }  
}  
  
public class Sub_class extends Super_class {  
    int num = 10;  
    // display method of sub class  
    public void display() {  
        System.out.println("This is the display method of subclass");  
    }  
    public void my_method() {  
        // Instantiating subclass  
        Sub_class sub = new Sub_class();  
    }  
}
```



```
// Invoking the display() method of sub class
sub.display();
// Invoking the display() method of superclass
super.display();
// printing the value of variable num of subclass
System.out.println("value of the variable named num in sub class:"
    + sub.num);
// printing the value of variable num of superclass
System.out.println("value of the variable named num in super class:"
    + super.num);
}

public static void main(String args[]) {
    Sub_class obj = new Sub_class();
    obj.my_method();
}
}
```

Output:

```
This is the display method of subclass
This is the display method of superclass
value of the variable named num in sub class:10
value of the variable named num in super class:20
```



Example : Invoking **super** [Method Overriding]



```
class Animal {
    // method in the superclass
    public void eat() {    System.out.println("I can eat");    }
}

class Dog extends Animal {           // Dog inherits Animal
    @Override                       // overriding the eat() method
    public void eat() {
        super.eat();                // call method of superclass
        System.out.println("I eat dog food");
    }
    public void bark() {              // new method in subclass
        System.out.println("I can bark");
    }
}

class UseSuperClass {
    public static void main(String[] args) {
        Dog labrador = new Dog();    // create an object of the subclass
        labrador.eat();               // call the eat() method
        labrador.bark();
    }
}

/*    I can eat
 *    I eat dog food
 *    I can bark        */
```



Example : Invoking **super** constructor



```
class Superclass {
    int age;

    Superclass(int age) {
        this.age = age;
    }

    public void getAge() {
        System.out.println("The value of the variable named age"
                           + "in super class is: " +age);
    }
}

public class Subclass extends Superclass {
    Subclass(int age) {    super(age);    }

    public static void main(String args[]) {
        Subclass s = new Subclass(24);
        s.getAge();
    }
}
```

// The value of the variable named age in super class is: 24



- It is an important concept of object-oriented programming.
- It simply means more than one form.
- It as the ability of a message to be displayed in more than one form.
- It is the ability of an object to take on many forms.
- Polymorphism in Java is a concept by which we can perform a single action in different ways.
- It is derived from 2 **Greek** words: **poly** and **morphs**.
- The word "**poly**" means many and "**morphs**" means forms. So polymorphism means many forms.
- There are two types of polymorphism in Java:
 - **Compile-Time** polymorphism - **Static**
(Example : Method/Operator Overloading)
But, Java doesn't support Operator Overloading.
 - **Runtime** polymorphism - **Dynamic**
(Example : Method Overriding)

- When there are multiple functions with same name but different parameters then these functions are said to be overloaded.
- Functions can be overloaded by **change in number of arguments** or/**and change in type of arguments**.
- Why Method Overloading ?
 - Suppose, a task to perform the addition of given numbers but there can be any number of arguments (let's say either 2 or 3 arguments for simplicity).
 - In order to accomplish the task, can create two methods **sum2num(int, int)** and **sum3num(int, int, int)** for two and three parameters respectively. However, other programmers, as well as you in the future may get confused as the behavior of both methods are the same but they differ by name.
 - The better way to accomplish this task is by **overloading** methods. And, depending upon the argument passed, one of the overloaded methods is called. This helps to increase the readability of the program.

```
// Java program for Method overloading

class MultiplyFun {

    // Method with 2 parameter
    static int Multiply(int a, int b) {    return a * b;    }

    // Method with the same name but 2 double parameter
    static double Multiply(double a, double b) {    return a * b;    }
}

class MehtodOverloading {
    public static void main(String[] args) {

        System.out.println(MultiplyFun.Multiply(2, 4));

        System.out.println(MultiplyFun.Multiply(5.5, 6.3));
    }
}

/* 8
 * 34.65 */
```

```
// Java program for Method overloading

class MultiplyFun {

    // Method with 2 parameter
    static int Multiply(int a, int b) {    return a * b;  }

    // Method with the same name but 3 parameter
    static int Multiply(int a, int b, int c) {    return a * b * c;  }

}

class Main {
    public static void main(String[] args) {
        System.out.println(MultiplyFun.Multiply(2, 4));

        System.out.println(MultiplyFun.Multiply(2, 7, 3));
    }
}

/*      8
 * 42 */
```

Example : Method Overloading (Only with 1 Argument)



```
class HelperService {  
  
    private String formatNumber(int value) {  
        return String.format("%d", value);  
    }  
  
    private String formatNumber(double value) {  
        return String.format("%.3f", value);  
    }  
  
    private String formatNumber(String value) {  
        return String.format("%.2f", Double.parseDouble(value));  
    }  
  
    public static void main(String[] args) {  
        HelperService hs = new HelperService();  
        System.out.println(hs.formatNumber(500));  
        System.out.println(hs.formatNumber(89.9934));  
        System.out.println(hs.formatNumber("550"));  
    }  
}  
  
/* 500  
 * 89.993  
 * 550.00 */
```



Important Points to remember :

- Two or more methods can have same name inside the same class if they accept different arguments. This feature is known as **method overloading**.
- Method overloading is achieved by either:
 - Changing the **number** of arguments.
 - Changing the **datatype** of arguments.
- Method overloading is not possible by changing the return type of methods.

- It is also known as **Dynamic** Method Dispatch.
- It is a process in which a function call to the overridden method is resolved at Runtime and can be achieved by **Method Overriding**.
- It occurs when a derived class has a definition for one of the member functions of the base class. That base function is said to be overridden.
- **Rules :**
 - Both the **superclass** and the **subclass** must have the same method name, the same return type and the same parameter list.
 - Can't override the method declared as **final** and **static**.
 - Always override abstract methods of the superclass
- Can we access the method of the superclass after overriding?
(Yes, using **super** keyword)

```
class Animal {
    public void displayInfo() {
        System.out.println("I am an animal.");
    }
}

class Dog extends Animal {
    @Override    // Not Compulsory (Tells explicitly to Compiler)
    public void displayInfo() {
        System.out.println("I am a dog.");
    }
}

class MethodOverriding {
    public static void main(String[] args) {
        Dog d1 = new Dog();
        d1.displayInfo();
    }
}

//    I am a dog.
```

Example-2 : Method Overriding(using super)



```
class Animal {
    public void displayInfo() {
        System.out.println("I am an animal.");
    }
}

class Dog extends Animal {
    public void displayInfo() {
        super.displayInfo();    // Invoking Base Class Method
        System.out.println("I am a dog.");
    }
}

class MethodOverriding {
    public static void main(String[] args) {
        Dog d1 = new Dog();
        d1.displayInfo();
    }
}

// I am an animal.
// I am a dog.
```




```
class Vehicle{
    void run(){
        System.out.println("Vehicle is running");
    }
}

//Creating a child class
class Bike extends Vehicle{
    public static void main(String args[]){
        //creating an instance of child class
        Bike obj = new Bike();
        //calling the method with child class instance
        obj.run();
    }
}

// Vehicle is running
```

```
//To illustrate the use of Java Method Overriding
//creating a parent class.

class Vehicle{
    //defining a method
    void run(){ System.out.println("Vehicle is running"); }
}

class Bike extends Vehicle{    //Creating a child class
    //defining the same method as in the parent class
    void run(){ System.out.println("Bike is running safely"); }

    public static void main(String args[]){
        Bike obj = new Bike();    //creating object
        obj.run();                //calling method
    }
}

// Bike is running safely
```

//Program to demonstrate the real scenario of Java Method Overriding

```
class Bank{  int getRateOfInterest(){return 0;} }
```

//Creating child classes.

```
class SBI extends Bank {  int getRateOfInterest(){return 8;} }
```

```
class ICICI extends Bank{  int getRateOfInterest(){return 7;} }
```

```
class AXIS extends Bank {  int getRateOfInterest(){return 9;} }
```

//MethodOveridding class to create objects and call the methods

```
class MethodOveridding{  
    public static void main(String args[]){  
        SBI s=new SBI();  
        ICICI i=new ICICI();  
        AXIS a=new AXIS();  
        System.out.println("SBI Rate of Interest: "+s.getRateOfInterest());  
        System.out.println("ICICI Rate of Interest: "+i.getRateOfInterest());  
        System.out.println("AXIS Rate of Interest: "+a.getRateOfInterest());  
    }  
}
```

Output:

```
SBI Rate of Interest: 8  
ICICI Rate of Interest: 7  
AXIS Rate of Interest: 9
```

- Can we override static method?

No, a static method cannot be overridden. It can be proved by runtime polymorphism

- Why can we not override static method?

It is because the static method is bound with class whereas instance method is bound with an object. Static belongs to the class area, and an instance belongs to the heap area.

- Can we override java main method?

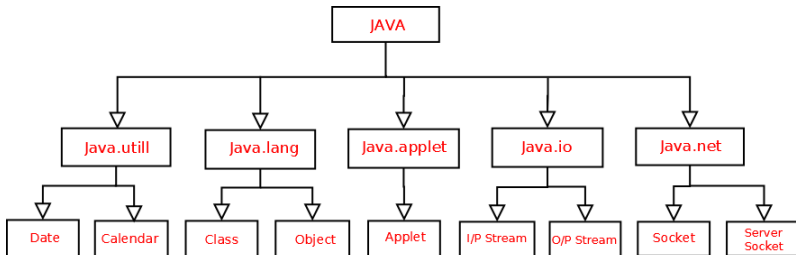
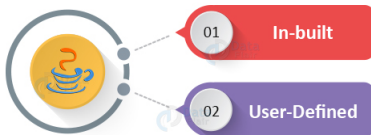
No, because the main is a static method.

Differences between Method Overloading and Method Overriding

No.	Method Overloading	Method Overriding
1)	It is used to increase the readability of the program.	It is used to provide the specific implementation of the method i.e., already provided by its super class.
2)	It is performed within class.	It occurs in two classes that have IS-A (inheritance) relationship.
3)	In case of method overloading, parameter must be different.	In case of method overriding, parameter must be same.
4)	Compile time polymorphism.	Run time polymorphism.
5)	It can't be performed by changing return type of the method only. Return type can be same or different in method overloading. But must have to change the parameter.	Return type must be same or covariant in method overriding.

- **PACKAGE** in Java is a collection of **classes**, **sub-packages**, and **interfaces**.
- It helps organize the classes into a folder structure and make it easy to locate and use them.
- More importantly, it helps improve code reusability.
- Each package in Java has its **unique** name and organizes its classes and interfaces into a separate namespace, or name group.
- Package in java can be categorized in to two forms :
 - **Built-in** Packages
 - **User-Defined** Packages
- **Advantages** :
 - It categorizes the classes and interfaces. so, they can be easily maintained.
 - It provides access protection.
 - It removes naming collision.
 - Re-usability and Data Encapsulation

Types of Packages in Java



- The Java API is a library of prewritten classes, that are free to use, included in the Java Development Environment.
- The library contains components for managing input, database programming, and much much more.
- The complete list can be found at Oracles website:
<https://docs.oracle.com/en/java/javase/15/docs/api/index.html>.
- The library is divided into **packages** and **classes**.
- To use a **class** or a **package** from the library, you need to use the **import** keyword:
- **Example :**

```
import package.name.Class;    // Import a single class
import package.name.*;        // Import the whole package

import java.util.Scanner;
// java.util is a package,
// while Scanner is a class of the java.util package.
```




```
javac KVP_Packages.java
```

The compilation is completed.

A **class file** MyClass is created.

However, no **package** is created?

- Every class is part of some package.
- All classes in a file are part of the same package.
- You can specify the package using a package declaration: package name ; as the first (non-comment) line in the file. It can be created using a keyword **package**:

Syntax:

```
package nameOfPackage;
```

Examples :

```
package MyPackages;           // Meaningfull Names
package My_Packages;
```

- Multiple files can specify the same package name.
- If no package is specified, the classes in the file go into a special unnamed package (the same unnamed package for all files).
- If package name is specified, the file must be in a sub-directory called name (i.e., the directory name must match the package name).

- To create User-Defined package, Java uses a **file system directory** to store them (Like Computers...).
- **File Structure** :

Example

```
└─ root
   └─ mypack
      └─ MyPackageClass.java
```

- To create a package, use the **package** keyword:

```
javac -d . MyClass.java
javac -d .. MyClass.java
```

1. The `-d` is a **switch** that tells the compiler where to put the **class file**
2. The `"."` operator represents the current working directory.
3. The `".."` indicates the parent directory.

- First **Compile the java** file :

```
javac MyPackageClass.java
```

- Now **Compile the package** :

```
javac -d . MyPackageClass.java  
javac -d .. MyPackageClass.java
```

1. The `-d` is a **switch** that tells the compiler where to put/save the **class file**
2. The `"."` operator represents the current working directory.
3. The `".."` indicates the parent directory.

Note : When we compiled the **package** as above,
a **new** folder was created, called **"mypack"**.

- Finally, **Run the package**:

```
java mypack.MyPackageClass  
  
// This is my first package!
```

There are three ways to access the package from outside the package.

1. `import package.*;`
2. `import package.classname;`
3. fully qualified name.

- **Example-1 :**

```
//save by A.java
package pack;
public class A{
    public void msg(){
        System.out.println("Hello");
    }
}

//save by B.java
package mypack;
import pack.*;
class B{
    public static void main(String args[]){
        A obj = new A();
        obj.msg();
    }
}
```

There are three ways to access the package from outside the package.

1. `import package.*;`
2. `import package.classname;`
3. fully qualified name.

- **Example-2 :**

```
//save by A.java
package pack;
public class A{
    public void msg(){
        System.out.println("Hello");
    }
}

//save by B.java
package mypack;
import pack.A;
class B{
    public static void main(String args[]){
        A obj = new A();
        obj.msg();
    }
}
```

There are three ways to access the package from outside the package.

1. `import package.*;`
2. `import package.classname;`
3. fully qualified name.

- **Example-3 :**

```
//save by A.java
package pack;
public class A{
    public void msg(){
        System.out.println("Hello");
    }
}
```

```
//save by B.java
package mypack;
class B{
    public static void main(String args[]){
        //using fully qualified name
        pack.A obj = new pack.A();
        obj.msg();
    }
}
```



- Package inside the package is called the subpackage.
- It should be created to categorize the package further.
- **Example** : Sun Microsystem has defined a package called **java** that contains several classes including **System**, **String**, **Reader**, **Writer**, **Socket** etc.
- These groups represent a specific category
 - The classes **Reader** and **Writer** are for the operation of Input/Output.
 - The classes **Socket** and **ServerSocket** are for networking etc and so on.
- Therefore, Sun sub-categorized the java package into sub-packages, such as **lang**, **net**, **io**, etc.
- The standard of defining package is :

Syntax :

`domain.company.package.`

Eg :

`in.vit.cse1007`


```
package in.vit.cse1007;           // Save it as Sub_Package.java

class Sub_Package{
    public static void main(String args[]){
        System.out.println("Vellore Institute of Technology");
    }
}
```

```
// 1. Compile Java File : javac Sub_Package.java
// 2. Compile Package :   javac -d . Sub_Package.java
// 3. Run Package :       java in.vit.cse1007.Sub_Package
```

Thanks



*Thank
you!*

Java Programming

Exception Handling

Dr. Pradeep K V

Associate Professor (Sr.)

School of Computer Science and Engineering

VIT - Chennai



- **Exception Handling**

- Exceptions & Errors
- Types of Exception
- Control Flow in Exceptions
- Use of **try**, **catch**, **finally**, **throw**, **throws** in Exception Handling
- User Defined Exceptions

- **Multithreading**

- Thread creation
- Thread Life cycle
- Thread priorities
- Methods to prevent Thread execution
- Sharing Workload among Threads
- Synchronization – IPC and Deadlock

- Exception handling is one of the most important feature of java programming that allows us to handle the **runtime errors** caused by **exceptions**.
- It is a mechanism to handle runtime errors such as [ClassNotFoundException](#), [IOException](#), [SQLException](#), [RemoteException](#), etc.
- **What is an Exception?**
 - An Exception is an unwanted event that interrupts the normal flow of the program and terminates program abruptly.
- **Why an exception occurs?**
 - There can be several reasons that can cause a program to throw exception. For **Example**: Opening a non-existing file in your program, Network connection problem, bad input data provided by user etc.
- **Advantages :**
 - It ensures, the flow of the program doesn't break when an exception occurs.
 - By handling, make sure that all the statements execute and the flow of program doesn't break.

Error Vs Exception

Error: indicates serious problem that a reasonable application should not try to catch.

Exception: indicates conditions that a reasonable application might try to catch.

Types of Exceptions

There are mainly 2 types of Exceptions.
But, as per Oracle, they are :

- Checked Exception
- Unchecked Exception
- Error (Unchecked Exception)



① Checked Exceptions :

- All exceptions other than Runtime Exceptions are known as Checked exceptions as the compiler checks them during compilation to see whether the programmer has handled them or not.
- If these exceptions are not handled/declared in the program, you will get compilation error.
- **Example** : `SQLException`, `IOException`, `ClassNotFoundException` etc.

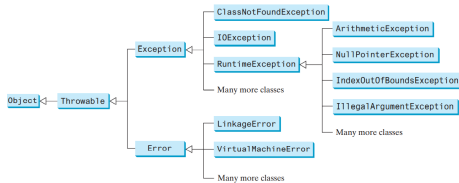
② Un-Checked Exceptions :

- Runtime Exceptions are also known as Unchecked Exceptions.
- These exceptions are not checked at compile-time so compiler does not check whether the programmer has handled them or not but it's the responsibility of the programmer to handle these exceptions and provide a safe exit.
- **Example** : `ArithmeticException`, `NullPointerException`, `ArrayIndexOutOfBoundsException` etc.

③ Error : It is irrecoverable

Example : `OutOfMemoryError`, `VirtualMachineError`, `AssertionError` etc.





The **java.lang.Throwable** class is the root class of Java Exception hierarchy which is inherited by two subclasses:
Exception and **Error**.

Common scenarios where exceptions may occur :

- ❶ `int a=50/0;` `//ArithmeticException`
- ❷ `String s=null;`
`System.out.println(s.length());` `//NullPointerException`
- ❸ `String s="abc";`
`int i=Integer.parseInt(s);` `//NumberFormatException`
- ❹ `int a[]=new int[5];`
`a[10]=50;` `//ArrayIndexOutOfBoundsException`

- Imagine that you are driving a car on a straight road in a national highway with high speed. When suddenly a dog comes from the bushes on the central divider and crosses the road. After spotting the dog, you started to apply the brakes. But, to bring down your car from such a high speed takes time and it hit the dog. What is the inference on the above situation?
- Assume that you want to reach a particular destination. The actual travelling time of the destination is 30 minutes but you started 1 hour before. Why?
- Imagine that you have developed a program to read only the numbers as inputs, and by mistake if you read an alphabet or a string. What is the impact of the output and how to handle this situation?
- Assume that you are searching a file in the hard disk, if it is not available, then how to handle this situation?

Java defines several exception classes inside the standard package **java.lang**.

Sr.No.	Exception & Description
1	ClassNotFoundException Class not found.
2	CloneNotSupportedException Attempt to clone an object that does not implement the Cloneable interface.
3	IllegalAccessException Access to a class is denied.
4	InstantiationException Attempt to create an object of an abstract class or interface.
5	InterruptedException One thread has been interrupted by another thread.
6	NoSuchFieldException A requested field does not exist.
7	NoSuchMethodException A requested method does not exist.

Sr.No.	Exception & Description
1	ArithmeticException Arithmetic error, such as divide-by-zero.
2	ArrayIndexOutOfBoundsException Array index is out-of-bounds.
3	ArrayStoreException Assignment to an array element of an incompatible type.
4	ClassCastException Invalid cast.
5	IllegalArgumentException Illegal argument used to invoke a method.
6	IllegalMonitorStateException Illegal monitor operation, such as waiting on an unlocked thread.
7	IllegalStateException Environment or application is in incorrect state.
8	IllegalThreadStateException Requested operation not compatible with the current thread state.

Sr.No.	Exception & Description
9	IndexOutOfBoundsException Some type of index is out-of-bounds.
10	NegativeArraySizeException Array created with a negative size.
11	NullPointerException Invalid use of a null reference.
12	NumberFormatException Invalid conversion of a string to a numeric format.
13	SecurityException Attempt to violate security.
14	StringIndexOutOfBoundsException Attempt to index outside the bounds of a string.
15	UnsupportedOperationException An unsupported operation was encountered.

Sr.No.	Method & Description
1	public String getMessage() : Returns a detailed message about the exception that has occurred. This message is initialized in the Throwable constructor.
2	public Throwable getCause() : Returns the cause of the exception as represented by a Throwable object.
3	public String toString() : Returns the name of the class concatenated with the result of getMessage().
4	public void printStackTrace() : Prints the result of toString() along with the stack trace to System.err, the error output stream.
5	public StackTraceElement [] getStackTrace() : Returns an array containing each element on the stack trace. The element at index 0 represents the top of the call stack, and the last element in the array represents the method at the bottom of the call stack.
6	public Throwable fillInStackTrace() : Fills the stack trace of this Throwable object with the current stack trace, adding to any previous information in the stack trace.

There are 5 keywords which are used in handling exceptions in Java.

Keyword	Description
try	It is used to specify a block where we should place exception code. It must be followed by either catch or finally. It means, we can't use try block alone.
catch	It is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.
finally	It is used to execute the important code of the program. It is executed whether an exception is handled or not.
throw	is used to throw an exception.
throws	It is used to declare exceptions. It doesn't throw an exception. It specifies that there may occur an exception in the method. It is always used with method signature.

Syntax :

// The try and catch keywords come in pairs:

```
try {  
    .....  
    .....  
    // Block of code to try  
    .....  
    .....  
}  
catch(Exception e) {  
    .....  
    .....  
    // Block of code to handle errors  
    .....  
    .....  
}
```


Without Exception Handling

```
public class Exception-1 {  
    public static void main(String[ ] args) {  
        int[] myNum= {1, 2, 3};  
        System.out.println(myNum[10]); // error!  
    }  
}
```

```
// Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 10  
// at Main.main(Main.java:4)
```

With Exception Handling

```
public class Exception-1 {  
    public static void main(String[ ] args) {  
        try {  
            int[] myNum= {1, 2, 3};  
            System.out.println(myNum[10]);  
        }  
        catch (Exception e) {  
            System.out.println("Something went wrong.");  
        }  
    }  
}  
  
// Something went wrong.
```

Without Exception Handling

```
public class Testtrycatch1 {  
    public static void main(String args[]){  
        int data=50/0;  
        System.out.println("rest of the code...");  
    }  
}
```

Output: Exception in thread main java.lang.ArithmeticException:/ by zero
(rest of the code is not executed)

With Exception Handling

```
public class Testtrycatch1{  
    public static void main(String args[]){  
        try{  
            int data=50/0;  
        }catch(ArithmeticException e){System.out.println(e);}  
        System.out.println("rest of the code...");  
    }  
}
```

Output: Exception in thread main java.lang.ArithmeticException:/ by zero
rest of the code...



Example-3 : Multiple **catch** block



```
public class MultipleCatchBlock{
    public static void main(String args[]){
        try{
            int a[]=new int[5];
            a[5]=30/0;
        }
        catch(ArithmeticException e){
            System.out.println("Arithmetic Exception Caught");
        }
        catch(ArrayIndexOutOfBoundsException e) {
            System.out.println("ArrayIndexOutOfBounds Exception");
        }
        catch(Exception e){
            System.out.println("Default Exception");
        }
        System.out.println("Rest of the code...");
    }
}
```

Output: Arithmetic Exception Caught
 Rest of the code...



Example-4 : Multiple **catch** (Ordered) block



```
public class MultipleCatchBlock{
    public static void main(String args[]){
        try{
            int a[]=new int[5];
            a[5]=30/0;
        }
        catch(Exception e){
            System.out.println("Default Exception");
        }
        catch(ArithmeticException e){
            System.out.println("Arithmetic Exception Caught");
        }
        catch(ArrayIndexOutOfBoundsException e) {
            System.out.println("ArrayIndexOutOfBoundsException");
        }
        System.out.println("Rest of the code...");
    }
}
```

Output: Compilation Error



```
package Rajkumar;
class Multiple{
    public static void main(String args[]) {
        try {
            int a=args.length;
            //Do not give any argument during execution and
            //check the result
            System.out.println("a="+a);
            int b=5/a;      int c[]={3};      c[10]=87;
        }
        catch(ArrayIndexOutOfBoundsException e) {
            System.out.println("Array Index Out of Bound"+e);
        }
        catch(ArithmeticException e) { System.out.println("Divide by zero");
            System.out.println("After try/catch block");
        }
    }
}

// Divide by zero java.lang.ArithmeticException: / by zero
// After try/catch block
```

Control flow occurs in each of the given case:

- Control flow in try-catch clause OR try-catch-finally clause
 - **Case 1:** Exception occurs in try block and handled in catch block
 - **Case 2:** Exception occurs in try-block is not handled in catch block
 - **Case 3:** Exception doesn't occur in try-block
- try-finally clause
 - **Case 1:** Exception occurs in try block
 - **Case 2:** Exception doesn't occur in try-block

Definition

Block within another Block is called as Nested Block.

The **try** block within a **try** block is known as **Nested try** block.

Why use nested try block...?

Sometimes a situation may arise where a part of a block may cause one error and the entire block itself may cause another error.

In such cases, exception handlers have to be **nested**.

```
....  
try  
{  
    statement 1;  
    statement 2;  
    try {  
        statement 1;  
        statement 2;  
    }  
    catch(Exception e) {  
    }  
}  
catch(Exception e) {  
}  
....
```

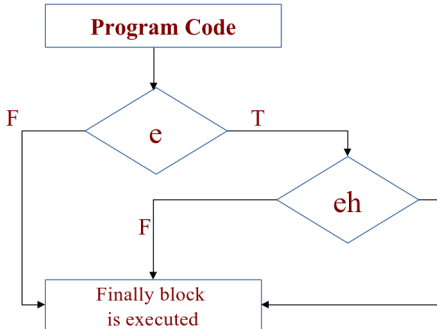
Example-6 : Nested **try** block



```
class NestedException{
    public static void main(String args[]){
        try{
            try{
                System.out.println("going to divide");
                int b =39/0;
            }
            catch(ArithmeticException e){System.out.println(e);}
            try{
                int a[]=new int[5];
                a[5]=4;
            }
            catch(ArrayIndexOutOfBoundsException e){
                System.out.println(e);
            }
            System.out.println("other statement");
        }
        catch(Exception e){System.out.println("handeled");}
        System.out.println("normal flow..");
    }
}
```



- It is used to execute vital code such as **closing connection**, **stream** etc.
- It is always executed whether exception is **handled** or **not**.
- It must be followed by **try** or **catch** block.



Why Use Java finally?

It is used to put "clean-up" code such as closing a file, closing connection etc.

Example -7 : Without & With **finally** block



Without **finally**

```
package Rajkumar;
class withoutfinally1{
    public static void main(String args[]){
        try{ int a=args.length; int x=5/a; }
        System.out.println("Statement is placed without finally block");
    }
}
// Exception in thread "main" java.lang.Error: Unresolved compilation problem:
// Syntax error, insert "Finally" to complete BlockStatements
// at withoutfinally1.main(withoutfinally1.java:6)
```

With **finally**

```
package Rajkumar;
class finally1 {
    public static void main(String args[]) {
        try { int a=args.length; int x=5/a; }
        catch(ArithmeticException e){ System.out.println(e); }
        finally{ System.out.println("Finally statement"); }
    }
}
// java.lang.ArithmeticException: / by zero
// Finally statement
```



```
class FinallyBlock-1{  
    public static void main(String args[]){  
        try{  
            int data=25/5;  
            System.out.println(data);  
        }  
        catch(NullPointerException e){  
            System.out.println(e);  
        }  
        finally{  
            System.out.println("finally block is always executed");  
        }  
        System.out.println("rest of the code...");  
    }  
}
```

Output: 5
finally block is always executed
rest of the code...

```
package Rajkumar;
class example2 {
    public static void main(String args[]){
        int a=5,b=0;
        try{ System.out.println(a/b); }
        catch(ArithmeticException e) { System.out.println(e); }
        finally { System.out.println("This is the finally statement");
        System.out.println("This is the statement after finally");
        }
    }
}
```

```
// java.lang.ArithmeticException: / by zero
// This is the finally statement
// This is the statement after finally
```

- It is used to explicitly throw an exception either checked or unchecked.
- It is mainly used to throw custom exception.

Syntax : **throw** exception;

Example : **throw new** IOException("sorry device error");

- Consider the below program :

```
public class TestThrow-1{
    static void validate(int age){
        if(age<18)
            throw new ArithmeticException("not valid");
        else
            System.out.println("welcome to vote");
    }
    public static void main(String args[]){
        validate(13);
        System.out.println("rest of the code...");
    }
}

// Exception in thread main java.lang.ArithmeticException:not valid
```

- It is used to declare an exception.
- It gives an information to the programmer that there may occur an exception so it is better for the programmer to provide the exception handling code so that normal flow can be maintained.
- Exception Handling is mainly used to handle the checked exceptions.
- If there occurs any unchecked exception such as **NullPointerException**, it is programmers fault that he is not performing check up before the code being used.
- **Syntax :**

```
return_type method_name() throws exception_class_name {  
    .....  
    //method code  
    .....  
}
```

- **Which exception should be declared?**

Ans : checked exception only,
unchecked Exception: under your control so correct your code.
error: beyond our control

- Advantage of Java **throws** keyword

- Now Checked Exception can be propagated (forwarded in call stack).
- It provides information to the caller of the method about the exception.

Example-16 : Java **throws** block



```
import java.io.IOException;
class Testthrows1{
    void m() throws IOException{
        throw new IOException("device error");
    }
    void n() throws IOException{ m(); }
    void p(){
        try{ n(); }
        catch(Exception e){
            System.out.println("exception handled");
        }
    }
    public static void main(String args[]){
        Testthrows1 obj=new Testthrows1();
        obj.p();
        System.out.println("normal flow...");
    }
}

//      exception handled
//      normal flow...
```

//check



Case-1: You handle the exception

```
import java.io.*;
class M{
    void method()throws IOException{
        throw new IOException("device error");
    }
}
public class Testthrows2{
    public static void main(String args[]){
        try{
            M m=new M();
            m.method();
        }
        catch(Exception e) {
            System.out.println("exception handled");    }
            System.out.println("normal flow...");
        }
    }
}
// exception handled
// normal flow...
```

Case-2: You declare the exception

A) Program if exception does not occur

```
import java.io.*;
class M{
    void method()throws IOException{
        System.out.println("device operation performed");
    }
}

class Testthrows3{
    public static void main(String args[])throws IOException{
        M m=new M();
        m.method();
        System.out.println("normal flow...");
    }
}

// device operation performed
// normal flow...
```

Case-2: You declare the exception

B) Program if exception occurs

```
import java.io.*;
```

```
class M{  
    void method()throws IOException{  
        throw new IOException("device error");  
    }  
}
```

```
class Testthrows4{  
    public static void main(String args[])throws IOException{//declare  
        M m=new M();  
        m.method();  
        System.out.println("normal flow...");  
    }  
}
```

// Runtime Exception

A list of differences between **throw** and **throws** are given below:

No.	throw	throws
1)	It is used to explicitly throw an exception.	It is used to declare an exception.
2)	Checked exception cannot be propagated using throw only.	Checked exception can be propagated with throws.
3)	It is followed by an instance.	It is followed by class.
4)	It used within the method.	It is used with the method signature.
5)	cannot throw multiple exceptions.	You can declare multiple exceptions e.g. public void method()throws IOException,SQLException.

A list of differences between **final**, **finally** and **finalize** are given below:

No.	final	finally	finalize
1)	It is used to apply restrictions on class, method and variable. final class can't be inherited, final method can't be overridden and final variable value can't be changed.	It is used to place important code, it will be executed whether exception is handled or not.	It is used to perform clean up processing just before object is garbage collected.
2)	final is a keyword.	finally is a block.	finalize is a method.

- We can define our own set of conditions or rules and throw an exception explicitly using throw keyword.
- For example, we can throw **ArithmeticException** when we divide number by 0, or any other numbers, what we need to do is just set the condition and throw any exception using throw keyword.

Syntax:

```
throw new exception_class("error message");
```

Example :

```
throw new ArithmeticException("Dividing by Zero");
```

Points to Remember :

- All exceptions must be a child of Throwable.
- If you want to write a checked exception that is automatically enforced by the Handle or Declare Rule, you need to extend the Exception class.
- If you want to write a runtime exception, you need to extend the RuntimeException class.



Example-1 : User-Defined Exception



```
class InvalidAgeException extends Exception{
    InvalidAgeException(String s){ super(s); }
}
class TestCustomException1{
    static void validate(int age)throws InvalidAgeException{
        if(age<18)
            throw new InvalidAgeException("not valid");
        else System.out.println("welcome to vote");
    }
    public static void main(String args[]){
        try{
            validate(13);
        }
        catch(Exception m){
            System.out.println("Exception occurred: "+m);
        }
        System.out.println("rest of the code...");
    }
}
// Exception occurred: InvalidAgeException:not valid
// rest of the code...
```



```
public class ThrowExampleDemo{
    static void checkEligibility(int grescore){
        if(grescore<315) {
            throw new ArithmeticException("Not eligible ");
        }
        else System.out.println("Student Entry is Valid!!");
    }
    public static void main(String args[]){
        System.out.println("Welcome to the Admission process!!");
        checkEligibility(300);
        System.out.println("Have a nice day..");
    }
}

class MyException extends Exception{
    public MyException(String s){
        super(s);
    }
}

// Not eligible
```

Example-3 : User-Defined Exception I



```
// File Name InsufficientFundsException.java
import java.io.*;
public class InsufficientFundsException extends Exception {
    private double amount;
    public InsufficientFundsException(double amount) {
        this.amount = amount;
    }
    public double getAmount() { return amount; }
}

// File Name CheckingAccount.java
import java.io.*;
public class CheckingAccount {
    private double balance;
    private int number;
    public CheckingAccount(int number) { this.number = number; }
    public void deposit(double amount) { balance += amount; }
    public void withdraw(double amount) throws InsufficientFundsException {
        if(amount <= balance) balance -= amount;
        else {
            double needs = amount - balance;
            throw new InsufficientFundsException(needs);
        }
    }
}
```



Example-3 : User-Defined Exception II



```
public double getBalance() { return balance; }
public int getNumber() { return number; }
}

// File Name BankDemo.java
public class BankDemo {
    public static void main(String [] args) {
        CheckingAccount c = new CheckingAccount(101);
        System.out.println("Depositing $500...");
        c.deposit(500.00);
        try {
            System.out.println("\nWithdrawing $100...");
            c.withdraw(100.00);
            System.out.println("\nWithdrawing $600...");
            c.withdraw(600.00);
        }
        catch (InsufficientFundsException e) {
            System.out.println("Sorry, but you are short " + e.getAmount());
            e.printStackTrace();
        }
    }
}
```

// Depositing \$500...



```
// Withdrawing $100...  
// Withdrawing $600...  
// Sorry, but you are short $200.0  
// InsufficientFundsException  
//      at CheckingAccount.withdraw(CheckingAccount.java:25)  
//      at BankDemo.main(BankDemo.java:13)
```

Example-4 : User-Defined Exception



```
package Rajkumar;
class userdefined extends Exception{
    userdefined(String s) {
        System.out.println("Untamed exception is thrown out:"+s);
    }
    public static void main(String args[]) {
        userdefined rajkumar=new userdefined("Rajkumar");
        try { throw rajkumar; }
        catch(Exception e) {
            System.out.println("I caught this exception and tamed it:"+e);
        }
    }
}

// Untamed exception is thrown out:Rajkumar
// I caught this exception and tamed it:userdefined
```



- Multithreading in Java is a process of executing multiple threads simultaneously.
- A thread is a lightweight sub-process, the smallest unit of processing.
- Multiprocessing and multithreading, both are used to achieve multitasking.
- **Why Multithreading**, not Multiprocessing?
 - Threads use a shared memory area, They don't allocate separate memory area so saves memory.
 - Context-switching between the threads takes less time than process.
- Java Multithreading is mostly used in games, animation, etc.
- **Advantages:**
 - It doesn't block the user, because threads are independent and you can perform multiple operations at the same time.
 - You can perform many operations together, so it saves time.
 - Threads are independent, so it doesn't affect other threads if an exception occurs in a single thread.
 - Cost of thread intercommunication is relatively low that that of process intercommunication.
 - Threads allow different tasks to be performed concurrently.



Definition...!

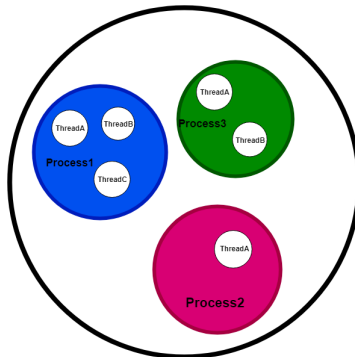
Multitasking is a process of executing multiple tasks simultaneously. It is used to utilize the CPU efficiently.

Multitasking can be achieved in two ways:

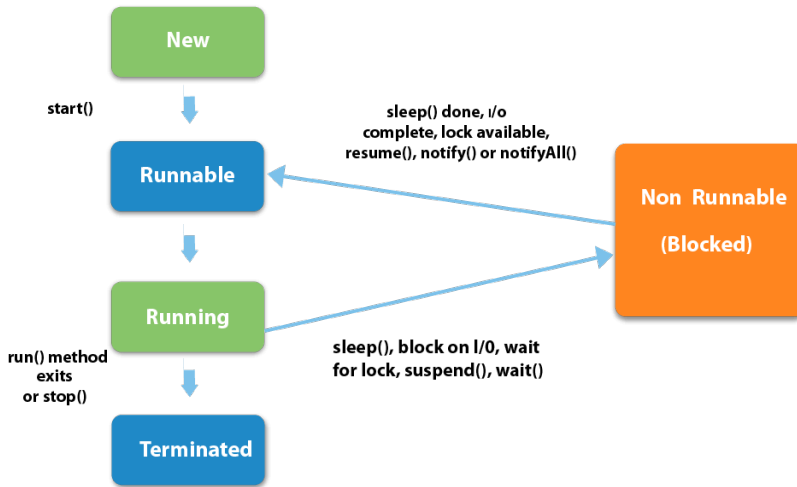
- ❶ **Process-based Multitasking (Multiprocessing) :**
 - Each process has an address in memory. In other words, each process allocates a separate memory area.
 - A process is heavyweight.
 - Cost of communication between the process is high.
 - Switching from one process to another requires some time for saving and loading registers, memory maps, updating lists, etc.
- ❷ **Thread-based Multitasking (Multithreading) :**
 - Threads share the same address space.
 - A thread is lightweight.
 - Cost of communication between the thread is low.

Definition....!

- It is a lightweight subprocess and the smallest unit of processing, has a separate path of execution.
- Threads are independent. If there occurs exception in one thread, it doesn't affect other threads.
- It uses a shared memory area.



Thread Life Cycle (Thread States)



A thread can be in one of the five states

- **New** : The thread is in new state if you create an instance of Thread class but before the invocation of start() method.
- **Runnable** : The thread is in runnable state after invocation of start() method, but the thread scheduler has not selected it to be the running thread.
- **Running** : The thread is in running state if the thread scheduler has selected it.
- **Non-Runnable** (Blocked) : This is the state when the thread is still alive, but is currently not eligible to run.
- **Terminated** : A thread is in terminated or dead state when its run() method exits.

How to Create Thread?

There are two ways to create a thread:

- By extending **Thread** class
- By implementing **Runnable** interface.

Thread class

- It provide **constructors** and **methods** to create and perform operations on a thread.
- It extends Object class and implements **Runnable** interface.

Commonly used **constructors**

1. Thread()
2. Thread(String name)
3. Thread(Runnable r)
4. Thread(Runnable r, String name)
5. Thread(ThreadGroup group, Runnable r)
6. Thread(ThreadGroup group, Runnable r, String name)
7. Thread(ThreadGroup group, Runnable r, String name, long stackSize)
8. Thread(ThreadGroup group, String name)



1. `public void run()`: is used to perform action `for` a thread.
2. `public void start()`: starts the execution of the thread.
JVM calls the `run()` method on the thread.
3. `public void sleep(long milliseconds)`: Causes the currently executing thread to `sleep` (temporarily cease execution) `for` the specified number of milliseconds.
4. `public void join()`: waits `for` a thread to die.
5. `public void join(long milliseconds)`: waits `for` a thread to die `for` the specified milliseconds.
6. `public int getPriority()`: returns the priority of the thread.
7. `public int setPriority(int priority)`: changes the thread priority.
8. `public String getName()`: returns the name of the thread.

9. `public void setName(String name)`: changes the name of the thread.
10. `public Thread currentThread()`: returns the reference of currently executing thread.
11. `public int getId()`: returns the id of the thread.
12. `public Thread.State getState()`: returns the state of the thread.
13. `public boolean isAlive()`: tests if the thread is alive.
14. `public void yield()`: causes the currently executing thread object to temporarily pause and allow other threads to execute.
15. `public void suspend()`: is used to suspend the thread(deprecated).
16. `public void resume()`: is used to resume the suspended thread(deprecated).
17. `public void stop()`: is used to stop the thread(deprecated).



- 18. `public boolean isDaemon()`: tests `if` the thread is a daemon thread.
- 19. `public void setDaemon(boolean b)`: marks the thread as daemon or user thread.
- 20. `public void interrupt()`: interrupts the thread.
- 21. `public boolean isInterrupted()`: tests `if` the thread has been interrupted.
- 22. `public static boolean interrupted()`: tests `if` the current thread has been interrupted.

Example-1 : Create Thread



```
class MyThread extends Thread {    // Extend Thread
    public void run() {
        System.out.println("MyThread is Created using Thread Class.");
    }
    public static void main(String[] args){
        MyThread Thd = new MyThread();
        Thd.run();
    }
}    // MyThread is Created using Thread Class
```

```
public class MyThread implements Runnable {    // Implements
    public void run() {
        System.out.println("MyThread is Created using Runnable");
    }
    public static void main(String[] args){
        MyThread Thd = new MyThread();
        Thd.run();
    }
}    // MyThread is Created using Runnable
```



start() method of **Thread** class is used to start a newly created thread.

It performs following tasks:

- A new thread starts(with new callstack).
- The thread moves from New state to the Runnable state.
- When the thread gets a chance to execute, its target run() method will run.

Example-2 : Create Thread (with start())



```
class MyThread extends Thread {
    public void run() {
        System.out.println("MyThread is Created using Thread Class.");
    }
    public static void main(String[] args){
        MyThread Thd = new MyThread();
        Thd.start();
    }
}    // MyThread is Created using Thread Class

public class MyThread implements Runnable {
    public void run() {
        System.out.println("MyThread is Created using Runnable");
    }
    public static void main(String[] args){
        MyThread Thd = new MyThread();
        Thread myThd = new Thread(Thd);
        myThd.start();
    }
}    // MyThread is Created using Runnable
```



Example-3 : Multithreading (Extends Thread)



```
// Java code for thread creation by extending the Thread class
class Multithreading extends Thread {
    public void run() {
        try{           // Displaying the thread that is running
            System.out.println ("Thread " +
                Thread.currentThread().getId() + " is running");
        }
        catch (Exception e) { System.out.println ("Exception is caught"); }
    }
}

// Main Class
public class Multithread {
    public static void main(String[] args) {
        int n = 5;           // Number of threads
        for (int i=0; i<n; i++){
            Multithreading object = new Multithreading();
            object.start();
        }
    }

    // Thread 13 is running
    // Thread 16 is running
    // Thread 17 is running
    // Thread 15 is running
    // Thread 14 is running
}
```



Example-4 : MultiThreading (Extends Threads) I



```
class MyThd extends Thread {
    MyThd() { }
    MyThd(String threadName) {
        super(threadName); // Initialize thread.
        System.out.println(this);    start();
    }
    public void run() { //Display info about this particular thread
        System.out.println(Thread.currentThread().getName());
    }
}

public class Thread1 {
    public static void main(String[] args) {
        Thread thread1 = new Thread(new MyThd(), "thread1");
        Thread thread2 = new Thread(new MyThd(), "thread2");

        //The below 2 threads are assigned default names
        Thread thread3 = new MyThd();
        Thread thread4 = new MyThd();

        Thread thread5 = new MyThd("thread5");

        //Start the threads
        thread1.start();    thread2.start();
    }
}
```



Example-4 : MultiThreading (Extends Threads) II



```
thread3.start();          thread4.start();

try { Thread.currentThread().sleep(1000); }
catch (InterruptedException e) { }

//Display info about the main thread
Thread.currentThread().setPriority(6);
System.out.println(Thread.currentThread());
}
}

/*      Thread[thread5,5,main]
      thread5
      thread1
      thread2
      Thread-2
      Thread-3
      Thread[main,6,main]      */
```

Once we start any thread, its execution depends on the OS implementation of time slicing and we can't control their execution. However we can set threads priority but even then it doesn't guarantee that higher priority thread will be executed first.



Example-5 : MultiThreading (Implements Runnable)



```
// Java code for thread creation by extending the Thread class
class Multithreading implements Runnable {
    public void run() {
        try { // Displaying the thread that is running
            System.out.println ("Thread " + Thread.currentThread().getId() +
                                " is running");
        }
        catch (Exception e) { // Throwing an exception
            System.out.println ("Exception is caught");
        }
    }
}

// Main Class
class Multithread {
    public static void main(String[] args) {
        int n = 5; // Number of threads
        for (int i=0; i<n; i++) {
            Thread object = new Thread(new Multithreading());
            object.start();
        }
    }

    // Thread 14 is running
    // Thread 20 is running
    // Thread 18 is running
    // Thread 16 is running
    // Thread 22 is running
}
```



Example-6 : MultiThreading (Implements Runnable) I



```
class RunnableThread implements Runnable {
    Thread runner;
    public RunnableThread() { }
    public RunnableThread(String threadName) {
        runner = new Thread(this, threadName); // (1) Create a new thread.
        System.out.println(runner.getName());
        runner.start(); // (2) Start the thread.
    }
    public void run() { //Display info about this particular thread
        System.out.println(Thread.currentThread().getName());
    }
}
```

```
public class Runnable_Thread {
    public static void main(String[] args) {
        Thread thread1 = new Thread(new RunnableThread(), "Thread1");
        Thread thread2 = new Thread(new RunnableThread(), "Thread2");
        RunnableThread thread3 = new RunnableThread("Thread3");

        //Start the threads
        thread1.start();          thread2.start();

        try { Thread.currentThread().sleep(1000); }
        catch (InterruptedException e) { }
```



Example-6 : MultiThreading (Implements Runnable) II



```
        //Display info about the main thread
        Thread.currentThread().setPriority(3);
        System.out.println(Thread.currentThread());
    }

    }

    /* Thread3
       Thread3
       Thread1
       Thread2
       Thread[main,3,main]    */
```

Once we start any thread, its execution depends on the OS implementation of time slicing and we can't control their execution. However we can set threads priority but even then it doesn't guarantee that higher priority thread will be executed first.



```
class GuessANumber extends Thread {
    private int number;
    public GuessANumber(int number) {    this.number = number;    }
    public void run() {
        int counter = 0, guess = 0;
        do {
            guess = (int) (Math.random() * 100 + 1);
            System.out.println(this.getName() + " guesses " + guess);
            counter++;
        } while(guess != number);
        System.out.println(this.getName() + " in "
                            + counter + " guesses.**");
    }
}

class FindFact extends Thread{
    private int number;
    public FindFact(int n) { number = n; }
    public void run() {
        int Fact = 1;
        for(int i=1; i<=number; i++) Fact = Fact * i;
        System.out.println(this.getName()+" returns the Factorial of "
                            + number +" : "+Fact);
    }
}
```

```
class FindReverse implements Runnable{
    private int number;
    public FindReverse(int n) { number = n; }
    public void run() {
        int Rev = 0;
        while(number > 0){
            int Rem = number % 10;
            Rev = Rev * 10 + Rem;
            number = number / 10;
        }
        System.out.println("Thread-3 returns the reverse of "
                           + n + " : "+Rev);
    }
}

public class GuessThread {
    public static void main(String [] args) {
        System.out.println("Starting T1...");
        Thread T1 = new GuessANumber(27);
        T1.start();
        try { T1.join(); }
        catch (InterruptedException e) {
            System.out.println("Thread interrupted.");
        }
        System.out.println("Starting T2...!");
        Thread T2 = new FindFact(4);
    }
}
```

```
T2.start();
try { T2.join();    }
catch (InterruptedException e) {
    System.out.println("Thread interrupted.");
}
System.out.println("Starting T3...!");
Runnable Obj = new FindReverse(1234);
Thread T3 = new Thread(Obj);
T3.start();
try { T3.join();    }
catch (InterruptedException e) {
    System.out.println("Thread interrupted.");
}
System.out.println("main() is ending...");
}
```

Starting T1...

Thread-0 guesses 40	Thread-0 guesses 61	Thread-0 guesses 12
Thread-0 guesses 38	Thread-0 guesses 27	Thread-0 in 5 guesses.**
Starting T2...!	Thread-1 returns the Factorial of 4 : 24	
Starting T3...!	Thread-3 returns the reverse of 1234 : 4321	
main() is ending...		

- It is the part of the JVM that decides which thread should run.
- There is no guarantee that which runnable thread will be chosen to run by the thread scheduler.
- Only one thread at a time can run in a single process.
- It uses **Preemptive** or **Time slicing** scheduling to schedule the threads.
- Under **Preemptive** scheduling, the highest priority task executes until it enters the waiting or dead states or a higher priority task comes into existence.
- Under **Time slicing**, a task executes for a predefined slice of time and then reenters the pool of ready tasks. The scheduler then determines which task should execute next, based on priority and other factors.

- In Multithreading, there is the asynchronous behavior of the programs. If one thread is **writing** some data and another thread which is **reading** data at the same time, might create inconsistency in the application.
- When there is a need to access the shared resources by two or more threads, then synchronization approach is utilized.
- Java has provided synchronized methods to implement synchronized behavior.
- In this approach, once the thread reaches inside the synchronized block, then no other thread can call that method on the same object. All threads have to wait till that thread finishes the synchronized block and comes out of that.
- In this way, the synchronization helps in a multithreaded application. One thread has to wait till other thread finishes its execution only then the other threads are allowed for execution.

```
synchronized(objectidentifier) {  
    // Access shared variables and other shared resources  
}
```



There are two types of thread synchronization mutual exclusive and inter-thread communication.

- **Mutual Exclusive** : it keep threads from interfering with one another while sharing data.
 - Synchronized method.
 - Synchronized block.
 - static synchronization.
- **Cooperation** (IPC - InterProcess/InterThreaded Communication)

Concept of Lock in Java :

- Synchronization is built around an internal entity known as the lock or monitor.
- Every object has a lock associated with it.
- By convention, a thread that needs consistent access to an object's fields has to acquire the object's lock before accessing them, and then release the lock when it's done with them.

Example-1 : Threads Without Synchronization I



```
class KVP_Print {  
    public void printCount() {  
        try {  
            for(int i = 5; i > 0; i--) {  
                System.out.println("Counter    ---    " + i );  
            }  
        }  
        catch (Exception e) { System.out.println("Thread interrupted."); }  
    }  
}
```

```
class KVP_Thread extends Thread {  
    private Thread t;  
    private String threadName;  
    KVP_Print PD;  
  
    KVP_Thread( String name, KVP_Print pd) {  
        threadName = name;    PD = pd;  
    }  
  
    public void run() {  
        PD.printCount();  
        System.out.println("Thread " + threadName + " exiting.");  
    }  
}
```



Example-1 : Threads Without Synchronization II



```
public void start () {
    System.out.println("Starting " + threadName );
    if (t == null) {
        t = new Thread (this, threadName);
        t.start ();
    }
}

}

public class ThreadSync1 {
    public static void main(String args[]) {

        KVP_Print PD = new KVP_Print();

        KVP_Thread T1 = new KVP_Thread( "Thread - 1 ", PD );
        KVP_Thread T2 = new KVP_Thread( "Thread - 2 ", PD );

        T1.start();    T2.start();

        try { T1.join(); T2.join(); }    // wait for threads to end
        catch ( Exception e) { System.out.println("Interrupted"); }

    }

}

/*      Starting Thread - 1
```



Example-1 : Threads Without Synchronization III



```
Starting Thread - 2
Counter    ---    5
Counter    ---    4
Counter    ---    3
Counter    ---    5
Counter    ---    2
Counter    ---    4
Counter    ---    1
Counter    ---    3
Thread Thread - 1 exiting.
Counter    ---    2
Counter    ---    1
Thread Thread - 2 exiting.    */
```

May or may not print counter value in sequence and every time we run it, it produces a different result based on CPU availability to a thread.



Example-2 : Threads With Synchronization method I



```
class KVP_Print {
    public void printCount() {
        try {
            for(int i = 5; i > 0; i--) {
                System.out.println("Counter   ---   " + i );
            }
        }
        catch (Exception e) { System.out.println("Thread interrupted."); }
    }
}
```

```
class KVP_Thread extends Thread {
    private Thread t;
    private String threadName;
    KVP_Print PD;

    KVP_Thread( String name, KVP_Print pd) {
        threadName = name;    PD = pd;
    }

    public void run() {
        synchronized(PD) { PD.printCount(); }    // Synchronization
        System.out.println("Thread " + threadName + " exiting.");
    }
}
```



Example-2 : Threads With Synchronization method II



```
public void start () {
    System.out.println("Starting " + threadName );
    if (t == null) {
        t = new Thread (this, threadName);
        t.start ();
    }
}

}

public class ThreadSync1 {
    public static void main(String args[]) {

        KVP_Print PD = new KVP_Print();

        KVP_Thread T1 = new KVP_Thread( "Thread - 1 ", PD );
        KVP_Thread T2 = new KVP_Thread( "Thread - 2 ", PD );

        T1.start();    T2.start();

        try { T1.join(); T2.join(); }    // wait for threads to end
        catch ( Exception e) { System.out.println("Interrupted"); }

    }

}

/*      Starting Thread - 1
```



Example-2 : Threads With Synchronization method III



```
Starting Thread - 2
Counter    ---    5
Counter    ---    4
Counter    ---    3
Counter    ---    2
Counter    ---    1
Thread Thread - 1 exiting.
Counter    ---    5
Counter    ---    4
Counter    ---    3
Counter    ---    2
Counter    ---    1
Thread Thread - 2 exiting.    */
```

Example-3 : Threads Without Synchronization I



```
package kvp;
class KVP_Table{
    void printTable(int n){                                //without synchronized method
        for(int i=1;i<=5;i++){
            System.out.println(n+" * "+i+" = "+n*i);
            try{ Thread.sleep(400); }
            catch(Exception e){System.out.println(e);}
        }
    }
}

class KVP_Thread1 extends Thread{
    KVP_Table t;
    KVP_Thread1(KVP_Table t){ this.t=t; }
    public void run(){ t.printTable(5); }
}

class KVP_Thread2 extends Thread{
    KVP_Table t;
    KVP_Thread2(KVP_Table t){ this.t=t; }
    public void run(){ t.printTable(10); }
}
```



Example-3 : Threads Without Synchronization II



```
public class TestSync{
    public static void main(String args[]){
        KVP_Table Obj = new KVP_Table();
        KVP_Thread1 T1=new KVP_Thread1(Obj);    //First Thread
        KVP_Thread2 T2 = new KVP_Thread2(Obj);    //Second Thread

        T1.start();
        T2.start();
    }
}

/*      10 * 1 = 10
        5 * 1 = 5
        5 * 2 = 10
        10 * 2 = 20
        5 * 3 = 15
        10 * 3 = 30
        5 * 4 = 20
        10 * 4 = 40
        5 * 5 = 25    */
```

May or may not print counter value in sequence and every time we run it, it produces a different result based on CPU availability to a thread.



Example-4 : Threads With Synchronization method I



```
package kvp;
class KVP_Table{
    synchronized void printTable(int n){           //synchronized method
        for(int i=1;i<=5;i++){
            System.out.println(n+" * "+i+" = "+n*i);
            try{ Thread.sleep(400); }
            catch(Exception e){System.out.println(e);}
        }
    }
}

class KVP_Thread1 extends Thread{
    KVP_Table t;
    KVP_Thread1(KVP_Table t){ this.t=t; }
    public void run(){ t.printTable(5); }
}

class KVP_Thread2 extends Thread{
    KVP_Table t;
    KVP_Thread2(KVP_Table t){ this.t=t; }
    public void run(){ t.printTable(10); }
}
```



Example-4 : Threads With Synchronization method II



```
public class TestSync{
    public static void main(String args[]){
        KVP_Table Obj = new KVP_Table();
        KVP_Thread1 T1=new KVP_Thread1(Obj);    //First Thread
        KVP_Thread2 T2 = new KVP_Thread2(Obj);    //Second Thread

        T1.start();
        T2.start();
    }
}

/*
5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25
10 * 1 = 10
10 * 2 = 20
10 * 3 = 30
10 * 4 = 40
10 * 5 = 50    */
```

Example-5 : Threads Without Synchronization I



```
package kvp;
import java.util.Scanner;

class KVP_Table{
    void printTable(int n){           //With out synchronized method
        for(int i=1;i<=5;i++){
            System.out.println(n+" * "+i+" = "+n*i);
            try{ Thread.sleep(400); }
            catch(Exception e){System.out.println(e);}
        }
    }
}

class KVP_Thread extends Thread{
    KVP_Table t;
    KVP_Thread1(KVP_Table t){ this.t=t; }
    public void run(){
        Scanner in = new Scanner(System.in);
        int n = in.nextInt();
        t.printTable(n); }
}
```



Example-5 : Threads Without Synchronization II



```
public class TestSync{
    public static void main(String args[]){
        KVP_Table Obj = new KVP_Table();
        KVP_Thread T1 = new KVP_Thread(Obj);           //First Thread
        KVP_Thread T2 = new KVP_Thread(Obj);           //Second Thread
        T1.start();
        T2.start();
    }
}

/*          7      5
5 * 1 = 5
7 * 1 = 7
5 * 2 = 10
7 * 2 = 14
5 * 3 = 15
7 * 3 = 21
5 * 4 = 20
7 * 4 = 28
5 * 5 = 25
7 * 5 = 35      */
```

May or may not print counter value in sequence and every time we run it, it produces a different result based on CPU availability to a thread



Example-6 : Threads With Synchronization method I



```
package kvp;
import java.util.Scanner;

class KVP_Table{
    synchronized void printTable(int n){           //With synchronized method
        for(int i=1;i<=5;i++){
            System.out.println(n+" * "+i+" = "+n*i);
            try{ Thread.sleep(400); }
            catch(Exception e){System.out.println(e);}
        }
    }
}

class KVP_Thread extends Thread{
    KVP_Table t;
    KVP_Thread1(KVP_Table t){ this.t=t; }
    public void run(){
        Scanner in = new Scanner(System.in);
        int n = in.nextInt();
        t.printTable(n); }
}
```



Example-6 : Threads With Synchronization method II



```
public class TestSync{  
    public static void main(String args[]){  
        KVP_Table Obj = new KVP_Table();  
        KVP_Thread T1 = new KVP_Thread(Obj);  
        KVP_Thread T2 = new KVP_Thread(Obj);  
        T1.start();  
        T2.start();  
    }  
}
```

//First Thread
//Second Thread

```
/*      8  
9  
9 * 1 = 9  
9 * 2 = 18  
9 * 3 = 27  
9 * 4 = 36  
9 * 5 = 45  
8 * 1 = 8  
8 * 2 = 16  
8 * 3 = 24  
8 * 4 = 32  
8 * 5 = 40  
5  
5 * 1 = 5  
5 * 2 = 10  
5 * 3 = 15  
5 * 4 = 20  
5 * 5 = 25  
7  
7 * 1 = 7  
7 * 2 = 14  
7 * 3 = 21  
7 * 4 = 28  
7 * 5 = 35  
*/
```

- Synchronized block can be used to perform synchronization on any specific resource of the method.
- Suppose you have 50 lines of code in your method, but you want to synchronize only 5 lines, you can use synchronized block.
- If you put all the codes of the method in the synchronized block, it will work same as the synchronized method.

Points to Remember

- Synchronized block is used to lock an object for any shared resource.
- Scope of synchronized block is smaller than the method.

```
synchronized(objectidentifier) {  
    // Access shared variables and other shared resources  
}
```

Example-7 : Threads With Synchronization block I



```
package kvp;
import java.util.Scanner;
class KVP_Table{
    void printTable(int n){
        synchronized(this) {
            for(int i=1;i<=5;i++){
                System.out.println(n+" * "+i+" = "+n*i);
                try{ Thread.sleep(400); }
                catch(Exception e){System.out.println(e);}
            }
        }
    }
}

class KVP_Thread1 extends Thread{
    KVP_Table t;
    KVP_Thread1(KVP_Table t){ this.t=t; }
    public void run(){
        Scanner in = new Scanner(System.in);
        int n = in.nextInt();
        t.printTable(n); }
}
```



Example-7 : Threads With Synchronization block II



```
public class TestSync{  
    public static void main(String args[]){  
        KVP_Table Obj = new KVP_Table();  
        KVP_Thread1 T1=new KVP_Thread1(Obj);    //First Thread  
        KVP_Thread1 T2 = new KVP_Thread1(Obj);    //Second Thread  
  
        T1.start();  
        T2.start();  
    }  
}
```

/* 8

5

9
9 * 1 = 9
9 * 2 = 18
9 * 3 = 27
9 * 4 = 36
9 * 5 = 45
8 * 1 = 8
8 * 2 = 16
8 * 3 = 24
8 * 4 = 32
8 * 5 = 40

5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25
7
7 * 1 = 7
7 * 2 = 14
7 * 3 = 21
7 * 4 = 28
7 * 5 = 35

*/



Example-8 : Threads Without Synchronization block I



// Without Synchronized block using Anonymous class

```
package kvp;
```

```
class KVP_Table{
```

```
    void printTable(int n){
```

```
        for(int i=1;i<=5;i++){
```

```
            System.out.println(n+" * "+i+" = "+n*i);
```

```
            try{ Thread.sleep(400); } 
```

```
            catch(Exception e){System.out.println(e);} 
```

```
        }
```

```
    }
```

```
}
```

```
public class AnonymousClass{
```

```
    static KVP_Table Obj = new KVP_Table();
```

```
    public static void main(String args[]){
```

```
        Thread T1 = new Thread() { //First Thread
```

```
            public void run(){ Obj.printTable(5); } 
```

```
    };
```



Example-8 : Threads Without Synchronization block II



```
Thread T2 = new Thread() { //Second Thread
    public void run(){ Obj.printTable(6); }
};

T1.start();
T2.start();
}

/*
5 * 1 = 5
6 * 1 = 6
5 * 2 = 10
6 * 2 = 12
5 * 3 = 15
6 * 3 = 18
5 * 4 = 20
6 * 4 = 24
5 * 5 = 25
6 * 5 = 30 */
```

Example-9 : Threads With Synchronization block I



```
// With Synchronized block using Anonymous class

package kvp;

class KVP_Table{
    void printTable(int n){
        synchronized(this) {
            for(int i=1;i<=5;i++){
                System.out.println(n+" * "+i+" = "+n*i);
                try{ Thread.sleep(400); }
                catch(Exception e){System.out.println(e);}
            }
        }
    }
}

public class AnonymousClass{
    static KVP_Table Obj = new KVP_Table();

    public static void main(String args[]){

        Thread T1 = new Thread() { //First Thread
            public void run(){ Obj.printTable(5); }
        };
    }
}
```



Example-9 : Threads With Synchronization block II



```
Thread T2 = new Thread() { //Second Thread
    public void run(){ Obj.printTable(6); }
};

T1.start();
T2.start();
}

/* 5 * 1 = 5
   5 * 2 = 10
   5 * 3 = 15
   5 * 4 = 20
   5 * 5 = 25
   6 * 1 = 6
   6 * 2 = 12
   6 * 3 = 18
   6 * 4 = 24
   6 * 5 = 30 */
```


- **static** synchronization is achieved by static synchronized methods.
- **static** synchronized method locked on class.
- **non-static** synchronized method locked on current object i.e. Static and Non-Static synchronized methods can run at same time. It can produce **inconsistency problem**.
- If **static** synchronized method is called a class level lock is acquired and then if an object is tries to access non-static synchronized method at the same time it will not be accessible because class level lock is already acquired.

Example-10 : Threads With **static** Synchronization I



```
import java.util.Scanner;

class KVP_Table{           // static synchronized
    public synchronized static void printTable(int n){
        System.out.println("Table of " + n);
        for(int i=1;i<=5;i++){
            System.out.println(n+" * "+i+" = "+n*i);
            try{ Thread.sleep(500); }
            catch(Exception e){ System.out.println(e); }
        }
    }
}

class KVP_Thd extends Thread{
    public void run(){
        Scanner Input = new Scanner(System.in);
        int a = Input.nextInt();
        KVP_Table.printTable(a);
    }
}
```



Example-10 : Threads With **static** Synchronization II



```
public class MultiThreadingStatic{
    public static void main(String args[]){

        KVP_Thd t1=new KVP_Thd();    //creating threads.
        KVP_Thd t2=new KVP_Thd();

        t1.start();    //start threads.
        t2.start();

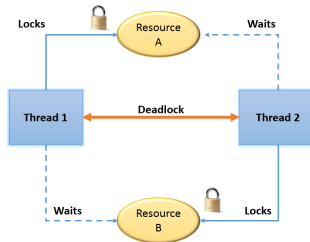
    }

    /*      4
           8
      Table of 4
      4 * 1 = 4
      4 * 2 = 8
      4 * 3 = 12
      4 * 4 = 16
      4 * 5 = 20
      Table of 8
      8 * 1 = 8
      8 * 2 = 16
      8 * 3 = 24
      8 * 4 = 32
      8 * 5 = 40    */
}
```



- **non-static** synchronized method :
 - Lock is on object.
 - Commonly used.
 - **static** keyword is not used.
- **static** synchronized method :
 - Lock is on class.
 - Not commonly used.
 - Used with **static** keyword.

- Java supports Multithreading, and it involves multiple threads running simultaneously for multitasking.
- But in certain cases, the threads find themselves in the **waiting state** forever.
- **Deadlock** : is a condition where two or more threads are blocked forever, waiting for each other. This usually happens when multiple threads need the same locks but obtain them in different orders.
- Multithreaded Programming in Java suffers from the deadlock situation because of the **synchronized** keyword.
- It causes the executing thread to block while waiting for the lock, or monitor, associated with the specified object.



Example-1 : DeadLock I



```
public class ThreadDeadlock {  
    public static void main(String[] args) throws InterruptedException {  
  
        // Objects are created from Object Class  
        Object obj2 = new Object();  
        Object obj1 = new Object();  
        Object obj3 = new Object();  
  
        // Threads are Created  
        Thread t1 = new Thread(new SyncThread(obj1, obj2), "Thread-1");  
        Thread t2 = new Thread(new SyncThread(obj2, obj3), "Thread-2");  
        Thread t3 = new Thread(new SyncThread(obj3, obj1), "Thread-3");  
  
        // Threads are started.  
        t1.start();        Thread.sleep(5000);  
        t2.start();        Thread.sleep(5000);  
        t3.start();  
    }  
}  
  
class SyncThread implements Runnable{  
  
    private Object obj1;  
    private Object obj2;
```



```
public SyncThread(Object o1, Object o2){ obj1=o1; obj2=o2; }  
    @Override  
    public void run() {  
        String name = Thread.currentThread().getName();  
        System.out.println(name + " : acquiring lock on "+obj1);  
  
        synchronized (obj1) {  
            System.out.println(name + " : acquired lock on "+obj1);  
            work();  
            System.out.println(name + " acquiring lock on "+obj2);  
            synchronized (obj2) {  
                System.out.println(name + " : acquired lock on "+obj2);  
                work();  
            }  
            System.out.println(name + " : released lock on "+obj2);  
        }  
  
        System.out.println(name + " : released lock on "+obj1);  
  
        System.out.println(name + " : finished execution.");  
    }  
  
    private void work() {  
        try { Thread.sleep(30000); }  
    }
```

```
        catch (InterruptedException e) { e.printStackTrace(); }
    }
}
/*
Thread-1 acquiring lock on java.lang.Object@3f42cb5f
Thread-1 acquired lock on java.lang.Object@3f42cb5f
Thread-2 acquiring lock on java.lang.Object@28d01fb4
Thread-2 acquired lock on java.lang.Object@28d01fb4
Thread-3 acquiring lock on java.lang.Object@3980f490
Thread-3 acquired lock on java.lang.Object@3980f490
....
.... */
```

In `main()`, 3 threads are running `for` `SyncThread` and a shared resource between each of the threads. The threads are run in such a way that it will be able to acquire lock on the first object but when it is trying to acquire lock on second object, it goes on wait state because it is already locked by another thread and forms a cyclic dependency `for` resources between Threads causing deadlock.

By executing, above is the output generated but program never terminates because of deadlock in java threads.

Guidelines to avoid most of the deadlock situations.

- ④ **Avoid Nested Locks:** Avoid locking another resource if a thread is already hold one.

```
public void run() {  
    String name = Thread.currentThread().getName();  
    System.out.println(name + " acquiring lock on " + obj1);  
  
    synchronized (obj1) {  
        System.out.println(name + " acquired lock on " + obj1);  
        work();  
    }  
  
    System.out.println(name + " released lock on " + obj1);  
    System.out.println(name + " acquiring lock on " + obj2);  
  
    synchronized (obj2) {  
        System.out.println(name + " acquired lock on " + obj2);  
        work();  
    }  
    System.out.println(name + " released lock on " + obj2);  
    System.out.println(name + " finished execution.");  
}
```



- ② **Lock Only What is Required** : Acquire lock only on the resources that the thread is working on.

For **Example** : In the above program, thread is locking the complete Object resource but if it is interested in one of it's fields, then it should lock only that specific field not complete object.

- ③ **Avoid waiting indefinitely**: it can occur deadlock if two threads are waiting for each other to finish indefinitely using thread join. If thread has to wait for another thread to finish, it's always best to use join with maximum time you want to wait for thread to finish.

```
public class DeadlockExample {  
    public static void main(String[] args) {  
        final String resource1 = "Java Programming";  
        final String resource2 = "C++ Programming";  
  
        // T1 tries to lock resource1 then resource2  
        Thread T1 = new Thread() {  
            public void run() {  
                synchronized (resource1) { // Holding  
                    System.out.println("Resource-1 : Locked by Thread 1");  
                    try { Thread.sleep(100);} catch (Exception e) {}  
  
                    synchronized (resource2) { // Seeking  
                        System.out.println("and Trying to Seek Resource-2 ");  
                    }  
                }  
            }  
        };  
  
        // T2 tries to lock resource2 then resource1
```

Example-2 : DeadLock II



```
Thread T2 = new Thread() {
    public void run() {
        synchronized (resource2) { // Holding
            System.out.println("Resource-2 : Locked by Thread 2");
            try { Thread.sleep(100);} catch (Exception e) {}

            synchronized (resource1) { // Seeking
                System.out.println("and Trying to Seek Resource-1 ");
            }
        }
    }
};
T1.start();    T2.start();
}

//      Resource-1 : Java Programming Locked by Thread 1
//      Resource-2 : C++ Programming Locked by Thread 2
```



Example-2 : DeadLock (Avoid Nested Synchronized) |



```
public class DeadlockExample {  
    public static void main(String[] args) {  
        final String resource1 = "Java Programming";  
        final String resource2 = "C++ Programming";  
  
        // T1 tries to lock resource1 then resource2  
        Thread T1 = new Thread() {  
            public void run() {  
                synchronized (resource1) { // Holding  
                    System.out.println("Resource-1 : Locked by Thread 1");  
                    try { Thread.sleep(100);} catch (Exception e) {}  
                }  
                synchronized (resource2) { // Seeking  
                    System.out.print("and Trying to Seek Resource-2 ");  
                }  
            }  
        };  
    }  
};
```

// T2 tries to lock resource2 then resource1



Example-2 : DeadLock (Avoid Nested Synchronized) II



```
Thread T2 = new Thread() {
    public void run() {
        synchronized (resource2) { // Holding
            System.out.println("Resource-2 : Locked by Thread 2");
            try { Thread.sleep(100);} catch (Exception e) {}
        }
        synchronized (resource1) { // Seeking
            System.out.print("and Trying to Seek Resource-1 ");
        }
    }
};
T1.start();
T2.start();
}
//      Resource-1 : Java Programming Locked by Thread 1 and
//      Trying to Seek Resource-2
//      Resource-2 : C++ Programming Locked by Thread 2 and
//      Trying to Seek Resource-1
```



- **ITC** or **Co-operation** is all about allowing synchronized threads to communicate with each other.
- It is a mechanism in which a thread is paused running in its critical section and another thread is allowed to enter (or lock) in the same critical section to be executed.
- It is implemented by following methods of Object class:
 - **wait()** : Causes current thread to release the lock and wait until either another thread invokes the **notify()** method or the **notifyAll()** method for this object, or a specified amount of time has elapsed.
 - **notify()** : Wakes up a single thread that is waiting on this object's monitor. If any threads are waiting on this object, one of them is chosen to be awakened. The choice is arbitrary and occurs at the discretion of the implementation.
 - **notifyAll()** : Wakes up all threads that are waiting on this object's monitor.

// Syntax : wait()

1. `public final void wait()throws` InterruptedException :
waits until object is notified.
2. `public final void wait(long timeout)throws` InterruptedException :
waits for the specified amount of time.

// Syntax : notify()

3. `public final void notify()` :
Wakes up a single thread that is waiting on object.

// Syntax : notifyAll()

4. `public final void notifyAll()` :
Wakes up all threads that are waiting on object.

Why all these methods are defined in Object `class` not Thread `class`?

Answer : It is because they are related to lock and object has a lock.

Table: Differences between **sleep()** and **wait()**

wait()	sleep()
wait() method releases the lock	sleep() method doesn't release the lock.
It is the method of Object class	It is the method of Thread class
It is the non-static method	It is the static method
It is the non-static method	It is the static method
should be notified by notify() or notifyAll() methods	after the specified amount of time, sleep is completed.

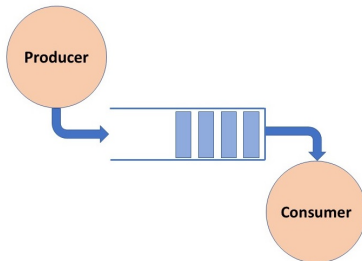


Figure: Producer-Consumer Problem

```
class Produce_Consume{
    int i;
    synchronized void Produce(int i) {
        this.i = i; System.out.println("Data Delivered: " +i);
    }
    synchronized int Consume() {
        System.out.println("Data Received: " + i);
        return i;
    }
}
```

```
class ProduceThread extends Thread
{
    Produce_Consume obj;
    ProduceThread(Produce_Consume obj){ this.obj = obj; }
    public void run() {
        for(int k = 0; k <= 5; k++) obj.Produce(k);
    }
}
```

```
class ConsumeThread extends Thread
{
    Produce_Consume obj;
    ConsumeThread(Produce_Consume obj){ this.obj = obj; }
    public void run() {
        for(int k = 0; k <= 5; k++) obj.Consume();
    }
}
```

```
class NoCommunication {
    public static void main(String[] args) {
        Produce_Consume obj = new Produce_Consume();
        ProduceThread P = new ProduceThread(obj);
        P.start();
        ConsumeThread C = new ConsumeThread(obj);
        C.start();
    }
}
```

```
/*          Data Delivered: 0
```

```
Data Delivered: 1
Data Delivered: 2
Data Delivered: 3
Data Delivered: 4
Data Delivered: 5
Data Received: 5
Data Received: 5
Data Received: 5
Data Received: 5
Data Received: 5    */
```

Explanation

Here, data delivered by **Thread1** is used by **Thread2**.

Thread1 produces data 1 to 5 and delivered to **Thread2** to receive it.

But due to no communication between these two threads, Thread2 uses data 5 five times in a row.

Example-2 : ITC using **wait()** and **notify()** |



```
class Produce_Consume{
    int i;
    boolean flag = false;
    synchronized void Produce(int i) {
        if(flag)
            try { // Wait till a notification is received from Thread2.
                wait();
            }
        catch(InterruptedException ie) { System.out.println(ie);
        this.i = i;
        System.out.println("Data Delivered: " +i);
        flag = true; // When data production is over.
        notify(); // Notification to Thread2, when data Produced.
    }
    synchronized int Consume() {
        if(!flag)
            try { // Wait till a notification is received from Thread1.
                wait();
            }
        catch(InterruptedException ie){ System.out.println(ie); }
        System.out.println("Data Received: " + i);
    }
}
```



```
        flag = false; // When data is received.
        notify(); //Notification to Thread1, when data Consumed.
        return i;
    }
}

class ProduceThread extends Thread {
    Produce_Consume obj;
    ProduceThread(Produce_Consume obj){ this.obj = obj; }
    public void run() { for(int k = 0; k <= 5; k++) obj.Produce(k); }
}

class ConsumeThread extends Thread {
    Produce_Consume obj;
    ConsumeThread(Produce_Consume obj){ this.obj = obj; }
    public void run() { for(int k = 0; k <= 5; k++) obj.Consume(); }
}

class Communication {
```

Example-2 : ITC using **wait()** and **notify()** III



```
public static void main(String[] args) {
    Produce_Consume obj = new Produce_Consume();
    ProduceThread P = new ProduceThread(obj);
    P.start();
    ConsumeThread C = new ConsumeThread(obj);
    C.start();
}
}

/*      Data Produced: 0      Data Consumed: 0
      Data Produced: 1      Data Consumed: 1
      Data Produced: 2      Data Consumed: 2
      Data Produced: 3      Data Consumed: 3
      Data Produced: 4      Data Consumed: 4
      Data Produced: 5      Data Consumed: 5      */
```



Thanks



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

*Thank
you!*



Java Programming

Java IO, Files and Java Collection Frameworks

Dr. Pradeep K V

Associate Professor (Sr.)

School of Computer Science and Engineering

VIT - Chennai



- Java IO Streams
 - Java InputStream & Java OutputStream
 - Java FileInputStream & Java FileOutputStream
 - Java ByteArrayInputStream & Java ByteArrayOutputStream
 - Java ObjectInputStream & Java ObjectOutputStream
 - Java BufferedInputStream & Java BufferedOutputStream
 - Java PrintStream
 - Java Reader & Java Writer
 - Java InputStreamReader & Java OutputStreamWriter
 - Java FileReader & Java FileWriter
 - Java BufferedReader & Java BufferedWriter
 - Java StringReader & Java StringWriter
 - Java PrintWriter
- Java Files
- Java Collection Frameworks
 - Java Collection Interface, Map Interface, Set Interface
 - Examples : Java List, ArrayList, Vector, Stack, Map, HashMap, Set, and HashSet

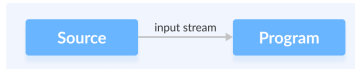
- In Java, streams are the sequence of data that are read from the source and written to the destination.

```
class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

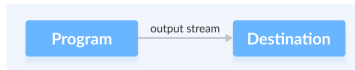
In the above Program, `System.out` is used to print a string. Here, the `System.out` is a type of output stream.

- Similarly, there are input streams to take input.

Reading data from source



Writing data to destination



- A stream is a sequence of data.
- In Java, a stream is composed of bytes.
- It's called a stream because it is like a stream of water that continues to flow.
- In Java, 3 streams are created automatically. All these streams are attached with the console.
 - **System.out**: standard output stream
 - **System.in**: standard input stream
 - **System.err**: standard error stream

- Example :

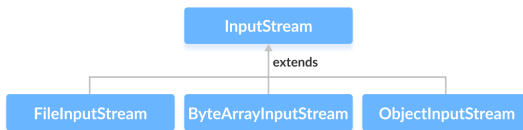
```
System.out.println("simple message");  
System.err.println("error message");
```

- Types of Streams :
 - **Byte** Stream
 - **Character** Stream

- **Byte stream** : is used to read and write a single byte (8 bits) of data.
- All byte stream classes are derived from base **abstract** classes called **InputStream** and **OutputStream**.

- 1 **Java InputStream** Class: It is of the **java.io** package is an abstract superclass that represents an input stream of bytes.

Subclass of InputStream :



Create an InputStream :

```
java.io.InputStream;    // Import Package First

// Creates an InputStream
InputStream object1 = new FileInputStream();
```

Here, we have created an input stream using FileInputStream. It is because InputStream is an **abstract class**.
Hence we cannot create an object of InputStream.



Methods of InputStream : different methods that are implemented by its subclasses.

1. `read()` - reads one **byte** of data from the input stream
2. `read(byte[] array)` - reads bytes from the stream and stores in the specified array
3. `available()` - returns the number of bytes available in the input stream
4. `mark()` - marks the position in the input stream up to which data has been read
5. `reset()` - returns the control to the point in the stream where the mark was set
6. `markSupported()` - checks **if** the `mark()` and `reset()` method is supported in the stream
7. `skip()` - skips and discards the specified number of bytes from the input stream
8. `close()` - closes the input stream



```
import java.io.FileInputStream;
import java.io.InputStream;

public class FileIOStream {
    public static void main(String args[]) {
        byte[] array = new byte[400];
        try {
            InputStream input = new FileInputStream("Input.txt");
            System.out.println("Available bytes in the file: "
                               + input.available());

            // Read byte from the input stream
            input.read(array);
            System.out.println("Data read from the file: ");

            // Convert byte array into string
            String data = new String(array);
            System.out.println(data);

            // Close the input stream
            input.close();
        }
        catch (Exception e) { e.printStackTrace(); }
    }
}
```

```
/*Available bytes in the file: 296
```

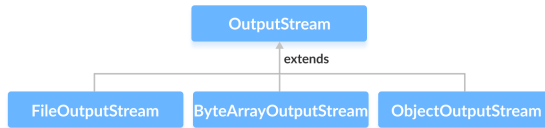
```
Data read from the file:
```

```
Java is a class-based, object-oriented programming language  
that is designed to have as few implementation dependencies  
as possible. ...
```

```
Java applications are typically compiled to bytecode that can  
run on any Java virtual machine (JVM) regardless of the  
underlying computer architecture.*/
```

- ② **Java OutputStream** Class: It is of the **java.io** package is an **abstract** superclass that represents an output stream of bytes.

Subclasses of OutputStream



Create an OutputStream

```
java.io.OutputStream    // Import this Package  
  
// Creates an OutputStream  
OutputStream object = new FileOutputStream();
```

Methods of OutputStream : different methods that are implemented by its subclasses.

1. `write()` -
writes the specified **byte** to the output stream
2. `write(byte[] array)` -
writes the bytes from the specified array to the output stream
3. `flush()` - forces to write all data present in output stream to the destination
4. `close()` - closes the output stream



```
import java.io.FileOutputStream;
import java.io.OutputStream;

public class FileOutputStream {
    public static void main(String args[]) {
        String data = "Java is a class-based, OOP language \n"
            + "that is designed to have as few implementation \n"
            + "dependencies as possible. ... \n";

        try {
            OutputStream out = new FileOutputStream("Output.txt");

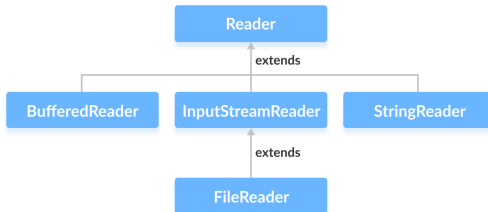
            byte[] dataBytes = data.getBytes(); // Converts Str to Bytes.
            out.write(dataBytes); // Writes data to the output stream
            out.close(); // Closes the output stream
        }
        catch (Exception e) { e.printStackTrace(); }
    }
}

/*Output.txt
Java is a class-based, object-oriented programming language
that is designed to have as few implementation dependencies
as possible. ... */
```



- Character stream is used to **read** and **write** a single character of data.
- All the character stream classes are derived from base abstract classes **Reader** and **Writer**.
- ① **Java Reader Class** : It is of the **java.io** package is an abstract superclass that represents a stream of characters.

Subclasses of Reader Class :



Create a Reader :

```
java.io.Reader // import this package

// Creates a Reader
Reader input = new FileReader();
```



Methods of Reader Class

1. `ready()` - checks `if` the reader is ready to be read
2. `read(char[] array)` - reads the characters from the stream and stores in the specified array
3. `read(char[] array, int start, int length)` - reads the no of characters equal to length from the stream and stores in the specified array starting from the start
4. `mark()` - marks the position in the stream up to which data has been read
5. `reset()` - returns the control to the point in the stream where the mark is set
6. `skip()` - discards the specified number of characters from the stream

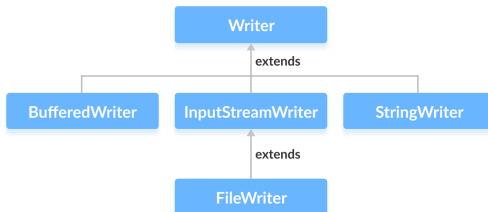
```
import java.io.Reader;
import java.io.FileReader;

class ReaderClass {
    public static void main(String[] args) {
        // Creates an array of character
        char[] array = new char[400];
        try {
            // Creates a reader using the FileReader
            Reader input = new FileReader("Input.txt");
            // Checks if reader is ready
            System.out.println("Is there data in the stream? "
                               + input.ready());
            // Reads characters
            input.read(array);
            System.out.println("Data in the stream:");
            System.out.println(array);
            // Closes the reader
            input.close();
        }
        catch(Exception e) {    e.printStackTrace(); }
    }
}
```



- ② **Java Writer Class** : It is of the **java.io** package is an **abstract** superclass that represents a stream of characters.

Subclasses of Writer :



Create a Writer :

```
java.io.Writer // Import this package first;

// Creates a Writer
Writer output = new FileWriter();
```

Methods a Writer :

1. `write(char[] array)` - writes the characters from the specified array to the output stream
2. `write(String data)` - writes the specified string to the writer
3. `append(char c)` - inserts the specified character to the current writer
4. `flush()` - forces to write all the data present in the writer to the corresponding destination
5. `close()` - closes the writer

```
import java.io.FileWriter;
import java.io.Writer;

public class WriterClass {

    public static void main(String args[]) {

        String data = "Java is a class-based, OOP language \n"
            + "that is designed to have as few implementation \n"
            + "dependencies as possible. ... \n";
```

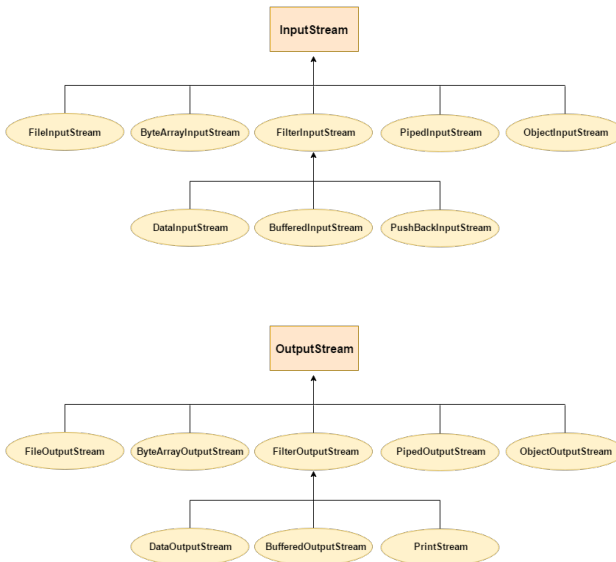


```
try {  
    // Creates a Writer using FileWriter  
    Writer output = new FileWriter("output.txt");  
  
    // Writes string to the file  
    output.write(data);  
  
    // Closes the writer  
    output.close();  
}  
catch (Exception e) { e.printStackTrace(); }  
}
```

*/*Output.txt*

*Java is a class-based, object-oriented programming language
that is designed to have as few implementation dependencies
as possible. ...*

**/*



- It is of the **java.io** package can be used to read an array of input data (in bytes). And extends the **InputStream** abstract class.
- **Create a ByteArrayInputStream :**

```
import java.io.ByteArrayInputStream // import this package first
// Creates a ByteArrayInputStream that reads entire array
ByteArrayInputStream input = new ByteArrayInputStream(byte[] arr);
// Creates a ByteArrayInputStream that reads a portion of array
ByteArrayInputStream input = new ByteArrayInputStream(byte[] arr,
                                                         int start, int length);
```

- **Methods of ByteArrayInputStream :**

1. `read()` - reads the single **byte** from the array present in the input stream
2. `read(byte[] array)` - reads bytes from the input stream and stores in the specified array
3. `read(byte[] array, int start, int length)` - reads the number of bytes equal to length from the stream and stores in the specified array starting from the position start.
4. `available()` - To get the number of available bytes



in the input stream.

5. `skip()` - To discard and skip the specified number of bytes
6. `close()` - To close the input stream.
7. `finalize()` - ensures that the `close()` method is called.
8. `mark()` - marks the position in input stream up to which data has been read.
9. `reset()` - returns the control to the point in the input stream where the mark was set.
10. `markSupported()` - checks **if** the input stream supports `mark()` and `reset()`

```
import java.io.ByteArrayInputStream;

public class ByteInputSTREAM {
    public static void main(String[] args) {
        byte[] array = {10, 20, 30, 40};    // Creates an Byte Array
        try {
            ByteArrayInputStream input = new ByteArrayInputStream(array);
            System.out.println("Available bytes at the beginning: "
                               + input.available());
            input.skip(2);    // skip first 2 bytes.
            System.out.print("The bytes read from the input stream: ");

            for(int i= 0; i < array.length; i++) {
                // Reads the bytes
                int data = input.read();
                if (data!=-1) System.out.print(data + ", ");
            }
            input.close();    // close the ByteArrayInputStream
        }
        catch(Exception e) { e.printStackTrace(); }
    }
}

// Available bytes at the beginning: 4
// The bytes read from the input stream: 30, 40,
```



- It is of the **java.io** package can be used to write an array of output data (in bytes).

- **Create a ByteArrayOutputStream :**

```
import java.io.ByteArrayOutputStream
// Creates a ByteArrayOutputStream with default size
ByteArrayOutputStream out = new ByteArrayOutputStream();
// Creating a ByteArrayOutputStream with specified size
ByteArrayOutputStream out = new ByteArrayOutputStream(int size);
// Default Size = 32 Bytes;
```

- **Methods of ByteArrayOutputStream :**

1. `write(int byte)` : writes the specified **byte** to the output stream
2. `write(byte[] array)` : writes the bytes from the specified array to the output stream
3. `write(byte[] arr, int start, int length)` : writes the number of bytes equal to length to the output stream from an array starting from the position start
4. `writeTo(ByteArrayOutputStream out1)` : writes the entire data of the current output stream to the specified output stream



Access Data from ByteArrayOutputStream

- 5. toByteArray() : returns the array present inside the output stream
- 6. toString() : returns the entire data of the output stream
in string form

Other Methods

- 7. size() : returns the size of the array in the output stream
- 8. flush() : clears the output stream

```
import java.io.ByteArrayOutputStream;

class ByteOutputSTREAM {
    public static void main(String[] args) {
        String data = "At least we agree that buying a glass house" +
                      "was a lousy idea.";

        try {
            // Creates an output stream
            ByteArrayOutputStream out = new ByteArrayOutputStream();
            byte[] array = data.getBytes(); // converts a string to Byte Array.

            // Writes data to the output stream
            out.write(array);

            // Retrieves data from the output stream in string format
        }
    }
}
```



```
String streamData = out.toString();
System.out.println("Data using toString(): " + streamData);

byte[] byteData = out.toByteArray();
System.out.print("Data using toByteArray(): ");
for(int i=0; i<byteData.length; i++) {
    System.out.print((char)byteData[i]);
}
out.close();
}
catch(Exception e) { e.printStackTrace(); }
}
}
```



```
// Data using toString(): At least we agree that buying a
//                          glass house was a lousy idea.
// Data using toByteArray(): At least we agree that buying a
//                          glass house was a lousy idea.
```

- It is of the **java.io** package can be used to read objects. And extends the **InputStream** abstract class.
- **Create a ObjectInputStream :**

```
import java.io.ObjectInputStream // import package first
// Creates a file input stream linked with the specified file
FileInputStream fileStream = new FileInputStream(String file);
// Creates an object input stream using the file input stream
ObjectInputStream objStream = new ObjectInputStream(fileStream);
```

- **Methods of ObjectInputStream :**
 1. **read()** : reads a **byte** of data from the input stream
 2. **readBoolean()** : reads data in **boolean** form
 3. **readChar()** : reads data in character form
 4. **readInt()** : reads data in integer form
 5. **readObject()** : reads the object from the input stream



6. `available()` : returns the available number of bytes in the input stream
7. `mark()` : marks the position in input stream up to which data has been read
8. `reset()` : returns the control to the point in the input stream where the mark was set
9. `skipBytes()` : skips and discards the specified bytes from the input stream
10. `close()` : closes the object input stream

```
import java.io.FileInputStream;    import java.io.FileOutputStream;
import java.io.ObjectInputStream;  import java.io.ObjectOutputStream;
import java.io.Serializable;
```

```
class Dog implements Serializable {
    String name, breed;
    public Dog(String N, String B) {    name = N;    breed = B;    }
}
```

```
class ObjectInputStream {
    public static void main(String[] args) {
        // Creates an object of Dog class
    }
}
```



```
Dog dog = new Dog("Baloo", "Australian Cattle");
try {
    // Creates an ObjectOutputStream, ObjectOutputStream & FileOutputStream
    ObjectOutputStream input = new ObjectOutputStream(fileStream);
    ObjectOutputStream output = new ObjectOutputStream(file);
    FileOutputStream file = new FileOutputStream("file.txt");

    output.writeObject(dog);    // Writes objects to output stream
    FileInputStream fileStream = new FileInputStream("file.txt");

    Dog newDog = (Dog) input.readObject();    // Reads the objects

    System.out.println("Dog Name: " + newDog.name);
    System.out.println("Dog Breed: " + newDog.breed);

    output.close();    input.close();
}
catch (Exception e) { e.printStackTrace(); }
}

// Dog Name: Baloo
// Dog Breed: Australian Cattle
```

- It is of the **java.io** package can be used to write objects. And it extends the **OutputStream** abstract class.
- **Create a ObjectOutputStream :**

```
import java.io.ObjectOutputStream    // import package first
// Creates a FileOutputStream linked with the specified file
FileOutputStream fileStream = new FileOutputStream(String file);
// Creates the ObjectOutputStream
ObjectOutputStream objStream = new ObjectOutputStream(fileStream);
```

- **Methods of ObjectOutputStream :**

1. **write()** : writes a **byte** of data to the output stream
2. **writeBoolean()** : writes data in **boolean** form
3. **writeChar()** : writes data in character form
4. **writeInt()** : writes data in integer form
5. **writeObject()** : writes object to the output stream
6. **flush()** : clears all the data from the output stream
7. **drain()** : puts all the buffered data in the output stream
8. **close()** : closes the output stream



```
import java.io.FileInputStream;    import java.io.FileOutputStream;
import java.io.ObjectInputStream;  import java.io.ObjectOutputStream;
class ObjectOutputStream {
    public static void main(String[] args) {
        int data1 = 50;
        String data2 = "This is an Example of ObjectOutputStream";
        try {
            FileOutputStream file = new FileOutputStream("file.txt");
            // Creates an ObjectOutputStream
            ObjectOutputStream output = new ObjectOutputStream(file);
            // writes objects to output stream
            output.writeInt(data1);    output.writeObject(data2);
            // Reads data using the ObjectInputStream
            FileInputStream fileStream = new FileInputStream("file.txt");
            ObjectInputStream objStream = new ObjectInputStream(fileStream);
            System.out.println("Integer data :" + objStream.readInt());
            System.out.println("String data: " + objStream.readObject());
            output.close();    objStream.close();
        } catch (Exception e) {    e.printStackTrace();    }
    }
}

// Integer data :50
// String data: This is an Example of ObjectOutputStream
```

- Both **BufferedInputStream** & **BufferedOutputStream** class of the **java.io** package is used with other **input/output** streams to
- Create a **BufferedInputStream** & **BufferedOutputStream** :

```
import java.io.BufferedInputStream    // import package
// Creates a FileInputStream
FileInputStream file = new FileInputStream(String path);
// Creates a BufferedInputStream
BufferedInputStream buffer = new BufferedInputStream(file);

import java.io.BufferedOutputStream    // import package
// Creates a FileOutputStream
FileOutputStream file = new FileOutputStream(String path);
// Creates a BufferedOutputStream
BufferedOutputStream buffer = new BufferedOutputStream(file);
```

read/write the data (in bytes) more efficiently.

- Methods of `BufferedInputStream` & `BufferedOutputStream` :

1. `read()` : reads a single `byte` from the input stream
2. `read(byte[] arr)` : reads bytes from the stream and stores in the specified array
3. `read(byte[] arr, int start, int length)` :
reads the number of bytes equal to the length from the stream & stores in the specified array starting from the position start
4. `available()` : To get the number of available bytes in the i/p stream
5. `skip()` : To discard and skip the specified number of bytes
6. `mark()` : mark the position in i/p stream up to which data has been read
7. `reset()` : returns the control to the point in the i/p stream where the mark was set
8. `write()` - writes a single `byte` to the internal buffer of the output stream
9. `write(byte[] array)` - writes the bytes from the specified array



to the output stream

10. `write(byte[] arr, int start, int length)` - writes the number of bytes equal to length to the output stream from an array starting from the position start
11. `flush()` To clear the internal buffer
12. `close()` : To close the buffered input/output stream

Consider the Given `Input.txt` File :

```
/* Input.txt
```

```
Java is a class-based, object-oriented programming language  
that is designed to have as few implementation dependencies  
as possible. ...
```

```
Java applications are typically compiled to bytecode that can  
run on any Java virtual machine (JVM) regardless of the  
underlying computer architecture.
```

```
*/
```

```
import java.io.FileInputStream; import java.io.BufferedInputStream;  
import java.io.FileOutputStream; import java.io.BufferedOutputStream; 
```

```
public class BufferedInputStream {
    public static void main(String args[]) {
        try {
            // Creates and Opens both Input.txt and Output.txt
            FileInputStream ifile = new FileInputStream("Input.txt");
            FileOutputStream ofile = new FileOutputStream("output.txt");

            // Creates a BufferedInputStream & BufferedOutputStream
            BufferedInputStream ibuffer = new BufferedInputStream(ifile);
            BufferedOutputStream obuffer = new BufferedOutputStream(ofile);

            // Returns the available number of bytes
            System.out.println("Available bytes at the beginning: "
                               + ibuffer.available());

            // Reads 3 bytes from the file
            ibuffer.read(); ibuffer.read(); ibuffer.read();
            // Returns the Remaining available number of bytes
            System.out.println("Available bytes after 3 read() : "
                               + ibuffer.available());

            ibuffer.skip(5); // Skip next 5 Bytes (Chars)
            System.out.println("Available bytes After skip(): "
                               + ibuffer.available());
        }
    }
}
```




```
// Read from File Input.txt and Write into Output.txt
int i = ibuffer.read();
while (i != -1) {
    obuffer.write(i);
    // Reads next byte from the input stream
    i = ibuffer.read();
}
//close both ibuffer and obuffer;
ibuffer.close();  obuffer.close();
}

catch (Exception e) {  e.printStackTrace();  }
}

// Available bytes at the beginning: 296
// Available bytes at the end: 293
// Available bytes After skip(): 288

/*  Output.txt    (Generated through the program)

a class-based, object-oriented programming language
that is designed to have as few implementation dependencies
```

as possible. ...

Java applications are typically compiled to bytecode that can run on any Java virtual machine (JVM) regardless of the underlying computer architecture.

**/*

- It is of the **java.io** package can be used to write output data in commonly readable form (text) instead of bytes. And it extends the abstract class **OutputStream**.
- It converts the primitive data (**integer, character**) into the text format instead of bytes. It then writes that formatted data to the output stream.
- It does not throw any input/output exception and has a feature of auto flushing.

- Create a **BufferedInputStream**

```
import java.io.PrintStream
```

Using other output **streams** (Option-1)

```
// Creates a FileOutputStream
FileOutputStream file = new FileOutputStream(String file);
// Creates a PrintStream
PrintStream output = new PrintStream(file, autoFlush);
```

Using **filename** (Option-2)

```
// Creates a PrintStream
PrintStream output = new PrintStream(String file, boolean autoFlush);
// Creates a PrintStream using some character encoding
PrintStream output = new PrintStream(String file, boolean autoFlush,
                                     Charset cs);
```



- Methods of **PrintStream**

1. `print()` : prints the specified data to the output stream
2. `println()` : prints the data to the output stream along with a **new** line character at the end
3. `close()` : closes the print stream
4. `checkError()` : checks **if** there is an error in the stream and returns a **boolean** result
5. `append()` : appends the specified data to the stream

```
import java.io.PrintStream;
class PrintSTREAM {
    public static void main(String[] args) {
        String data = "My favorite Subject in school is nothing\n";
        int age=30;
        System.out.print(data);    // Prints on Console
        try {
            PrintStream output = new PrintStream("output.txt");
            output.print(data); // Write into a File
            output.printf("I am %d years old.", age);
            output.close();
        } catch (Exception e) { e.printStackTrace(); }
    }
    // Analyse Program and Guess the Output ?
}
```

- They are of the **java.io** package can be used to convert Bytes data into Character data & viceversa.
- Both works with other input/output streams and are known as a bridge between **byte** streams and **character** streams.
- **InputStreamReader** reads bytes from the input stream as characters.
- **OutputStreamWriter** converts its characters into bytes.
- **Create an InputStreamReader & OutputStreamWriter**

```
java.io.InputStreamReader;  
// Creates an InputStream  
FileInputStream file = new FileInputStream(String path);  
// Creates an InputStreamReader or with Character encoding  
InputStreamReader input = new InputStreamReader(file);  
InputStreamReader input = new InputStreamReader(file, Charset cs);  
  
java.io.OutputStreamWriter;  
// Creates an OutputStream  
FileOutputStream file = new FileOutputStream(String path);  
// Creates an OutputStreamWriter or with Character encoding  
OutputStreamWriter output = new OutputStreamWriter(file);  
OutputStreamWriter output = new OutputStreamWriter(file, Charset cs);
```



● Methods of InputStreamReader & OutputStreamWriter :

InputStreamReader Methods : -

1. `read()` : reads a single character from the reader
2. `read(char[] array)` : reads the characters from the reader and stores in the specified array
3. `read(char[] array, int start, int length)` : reads the number of characters equal to length from the reader and stores in the specified array starting from the start
5. `ready()` : checks if the stream is ready to be read
6. `mark()` : mark the position in stream up to which data has been read
7. `reset()` : returns the control to the point in the stream where the mark was set

OutputStreamWriter Methods : -

1. `write()` : writes a single character to the writer
2. `write(char[] array)` : writes the characters from the specified array to the writer
3. `write(String data)` : writes the specified string to the writer
4. `flush()` : forces to write all the data present in the writer to the corresponding destination
5. `append()` : inserts the specified character to the current writer



Common Methods

1. `close()` : To close the Input/Output stream
2. `getEncoding()` : used to get the type of encoding used to store data in the input/output stream

```
/* Consider Input.txt
```

```
Java is a class-based, object-oriented programming language  
that is designed to have as few implementation dependencies  
as possible. ...
```

```
Java applications are typically compiled to bytecode that can  
run on any Java virtual machine (JVM) regardless of the  
underlying computer architecture.
```

```
*/
```

```
import java.io.*;  
import java.nio.charset.Charset;  
class InputOutputStreamReaderWriter {  
    public static void main(String[] args) {  
        char[] array = new char[500] ;
```

```
try {
```



```
// Creates a FileInputStream
FileInputStream ifile = new FileInputStream("Input.txt");
FileOutputStream ofile = new FileOutputStream("Output.txt");

// Creates an InputStreamReader with default encoding
InputStreamReader input = new InputStreamReader(ifile);
InputStreamReader input2 = new InputStreamReader(ifile,
    Charset.forName("UTF16"));
System.out.println("Character encoding of input: "
    + input2.getEncoding());
System.out.println("Character encoding of input: "
    + input.getEncoding());
input.read(array);
// Creates an OutputStreamWriter
OutputStreamWriter output = new OutputStreamWriter(ofile);
output.write(array);

input.close();    output.close();
}
catch(Exception e) {    e.printStackTrace();    }
}
}

// Character encoding of input: UTF-16
// Character encoding of input: UTF8
```



```
/* Output.txt (File Generated)
```

```
Java is a class-based, object-oriented programming language  
that is designed to have as few implementation dependencies  
as possible. ...
```

```
Java applications are typically compiled to bytecode that can  
run on any Java virtual machine (JVM) regardless of the  
underlying computer architecture.
```

```
*/
```

- They are of the **java.io** package can be used to read/write data (in characters) from files. They extends **InputStreamReader** and **OutputStreamReader** Classess respectively.
- **Create an FileReader & FileWriter :**

```
import java.io.FileReader // import package first;
// Using the name of the file
FileReader input = new FileReader(String name);
FileReader input = new FileReader(String file, Charset cs);

// Using an object of the file
FileReader input = new FileReader(File fileObj);

import Java.io.FileWriter // import package first;
// Using the name of the file
FileWriter output = new FileWriter(String name);
FileWriter input = new FileWriter(String file, Charset cs);

// Using an object of the file
FileWriter input = new FileWriter(File fileObj);
```

● Methods of FileReader & FileWriter :

Methods of FileReader

1. `read()` : reads a single character from the reader
2. `read(char[] array)` : reads the characters from the reader and stores in the specified array
3. `read(char[] array, int start, int length)` : reads the number of characters equal to length from the reader and stores in the specified array starting from the position start
4. `ready()` : checks if the file reader is ready to be read
5. `mark()` : mark the position in file reader up to which data has been read
6. `reset()` : returns the control to the point in the reader where the mark was set

Methods of FileWriter

1. `write()` : writes a single character to the writer
2. `write(char[] array)` : writes the characters from the specified array to the writer
3. `write(String data)` : writes the specified string to the writer
4. `flush()` : forces to write all the data present in the writer to the corresponding destination
5. `append()` : inserts the specified character to the current writer

Common Methods



1. `getEncoding()` : used to get the type of encoding that is used to store/write data in the file
2. `close()` : To close the `FileReader/FileWriter`.

/ Consider Input.txt as below*

Java is a class-based, object-oriented programming language that is designed to have as few implementation dependencies as possible. ...

**/*

```
import java.io.*;
import java.nio.charset.Charset;

class FileReaderWriter {
    public static void main(String[] args) {
        char[] array = new char[500]; // Creates an array of character
        try {
            // Creates a reader using the FileReader
            FileReader input = new FileReader("Input.txt");
            FileWriter output = new FileWriter("Output.txt");

            input.read(array); // Reads characters from Input.txt
```



```

output.write(array); // Write Characters into Output.txt

FileReader input2 = new FileReader("Input.txt", Charset.forName("UTF16"))

// Returns the character encoding of the file reader
System.out.println("Character encoding of input1: " + input.getEncoding());
System.out.println("Character encoding of input2: " + input2.getEncoding());
// Closes the reader and writer
input.close(); input2.close();    output.close();
}

catch(Exception e) { e.printStackTrace();    }
}                                // Character encoding of input1: UTF-8
}                                // Character encoding of input2: UTF-16

/* Output.txt file is generated as

Java is a class-based, object-oriented programming language
that is designed to have as few implementation dependencies
as possible. ...          */

```

- They are of the **java.io** package can be used with other **readers/writers** to read/write data (in characters) more efficiently.
- **Create an BufferedReader & BufferedWriter :**

```
import java.io.BufferedReader // import package first
// Creates a FileReader
FileReader file = new FileReader(String file);
// Creates a BufferedReader
BufferedReader buffer = new BufferedReader(file);
// Creates a BufferdReader with specified size internal buffer
BufferedReader buffer = new BufferedReader(file, int size);

import java.io.BufferedWriter // import package first
// Creates a FileWriter
FileWriter file = new FileWriter(String name);
// Creates a BufferedWriter
BufferedWriter buffer = new BufferedWriter(file);
// Creates a BufferedWriter with specified size internal buffer
BufferedWriter buffer = new BufferedWriter(file, int size);
```

● Methods of BufferedReader & BufferedWriter :

Methods of BufferedReader

1. `read()` : reads a single character from the internal buffer of the reader
2. `read(char[] array)` : reads the characters from the reader and stores in the specified array
3. `read(char[] array, int start, int length)` : reads the number of characters equal to length from the reader and stores in the specified array starting from the position start
4. `skip()` : To discard and skip the specified number of characters
5. `ready()` : checks if the file reader is ready to be read
6. `mark()` : mark the position in reader up to which data has been read
7. `reset()` : returns the control to the point in the reader where the mark was set

Methods of BufferedWriter

1. `write()` : writes a single character to the internal buffer of the writer
2. `write(char[] array)` : writes the characters from the specified array to the writer
3. `write(String data)` : writes the specified string to the writer
4. `flush()` : To clear the internal buffer
5. `newline()` : inserts a new line to the writer
6. `append()` : inserts the specified character to the current write



Common Methods :

1. close() Method : To close the buffered reader/buffered writer

```
/* Consdier Input.txt
```

```
Java is a class-based, object-oriented programming language  
that is designed to have as few implementation dependencies  
as possible. ...
```

```
*/
```

```
import java.io.*;  
public class BufferedReaderWriter {  
    public static void main(String args[]) {  
  
        char[] array = new char[500];    // Creates an character array  
        try {  
            FileReader ifile = new FileReader("Input.txt");  
            FileWriter ofile = new FileWriter("Output.txt");  
  
            // Creates a BufferedReader  
            BufferedReader input = new BufferedReader(ifile);  
            BufferedWriter output = new BufferedWriter(ofile);
```




```
// Skips the 10 characters
input.skip(10);          input.read(array);

// Reads the characters
output.write(array);

// closes the reader and writer
input.close(); output.close();
}

catch (Exception e) { e.printStackTrace(); }
}

/* Output.txt file is generated.

class-based, object-oriented programming language
that is designed to have as few implementation dependencies
as possible. ...

*/
```

- They are of the **java.io** package can be used to read/write data (in characters) from strings. And extends the abstract class Reader.

- **Create an StringReader & StringWriter :**

```
import java.io.StringReader // import package first
// Creates a StringReader
```

```
StringReader input = new StringReader(String data);
```

```
import java.io.StringWriter // import package first
// Creates a StringWriter
```

```
StringWriter output = new StringWriter();
```

```
// Creates a StringWriter with specified string buffer capacity
```

```
StringWriter output = new StringWriter(int size);
```

- **Methods of StringReader & StringWriter :**

```
// Methods of StringReader :
```

1. read() : reads a single character from the string reader
2. read(char[] array) : reads the characters from the reader and stores in the specified array
3. read(char[] array, int start, int length) : reads the number of characters equal to length from the reader and stores in the specified array starting from the position start
4. skip() : To discard and skip the specified number of characters
5. ready() : checks if the string reader is ready to be read
6. mark() : marks the position in reader up to which data has been read

7. `reset()` : returns the control to the point in the reader where the mark was set

//Methods of StringWriter :

1. `write()` : writes a single character to the string writer
2. `write(char[] array)` : writes the characters from the specified array to the writer
3. `write(String data)` : writes the specified string to the writer
4. `getBuffer()` : returns the data present in the string buffer
5. `toString()` : returns the data present in the string buffer as a string
6. `flush()` : forces to write all the data present in the writer to the string buffer
7. `append()` : inserts the specified character to the current writer

```
import java.io.*;
```

```
public class StringReaderWriter {  
    public static void main(String[] args) {  
        String data = "Java is a class-based, object-oriented programming \n"  
            + "language that is designed to have as few implementation \n"  
            + "dependencies as possible. ... ";  
        // Create a character array  
        char[] array = new char[500];  
        try {
```



```
// Create a StringReader
StringReader input = new StringReader(data);
StringWriter output = new StringWriter();

input.skip(5);    // Use the skip() method
input.read(array); // Use the read method
System.out.println("Data after skipping 5 characters:");
System.out.println(array);

output.write(data);
// Returns the string buffer
StringBuffer stringBuffer = output.getBuffer();
System.out.println("StringBuffer: " + stringBuffer);

// Returns the string buffer in string form
String string = output.toString();
System.out.println("String: " + string);

input.close(); output.close(); // Close both Reader/Writer
}
catch(Exception e) { e.printStackTrace(); }
}
/*
Data after skipping 5 characters:
```

```
is a class-based, object-oriented programming  
language that is designed to have as few implementation  
dependencies as possible. ...  
StringBuffer: Java is a class-based, object-oriented programming  
language that is designed to have as few implementation  
dependencies as possible. ...  
String: Java is a class-based, object-oriented programming  
language that is designed to have as few implementation  
dependencies as possible. ...  
*/
```

- It is of the **java.io** package can be used to write output data in a commonly readable form (text). And extends the abstract class **Writer**.
- It converts the primitive data (**int**, **float**, **char**, etc.) into the text format. It then writes that formatted data to the writer.
- **Create an PrintWriter :**

```
java.io.PrintWriter // import package first;
// Using other writers
    // Creates a FileWriter
    FileWriter file = new FileWriter("output.txt");
    // Creates a PrintWriter
    PrintWriter output = new PrintWriter(file, autoFlush);
// Using other output streams
    // Creates a FileOutputStream
    FileOutputStream file = new FileOutputStream("output.txt");
    // Creates a PrintWriter
    PrintWriter output = new PrintWriter(file, autoFlush);
// Using Filename
    // Creates a PrintWriter
    PrintWriter output = new PrintWriter(String file, boolean autoFlush);
    // Creates a PrintWriter using some character encoding
    PrintWriter output = new PrintWriter(String file,
        boolean autoFlush, Charset cs);
```



• Methods of PrintWriter :

1. `print()` : prints the specified data to the writer
2. `println()` : prints the data to the writer along with a **new** line character at the end
3. `printf()` : used to print the formatted string.
4. `close()` : closes the print writer
5. `checkError()` : checks **if** there is an error in the writer and returns a **boolean** result
6. `append()` : appends the specified data to the writer

```
import java.io.PrintWriter;

class PRINTWRITER {

    public static void main(String[] args) {

        int age = 50;

        String data = "Java is a class-based, OOP Language \n"
            + "that is designed to have as few implementation dependencies \n"
            + "as possible.... ";
    }
}
```



```
try {
    PrintWriter output = new PrintWriter("Output.txt");

    output.print(data);
    output.printf("\nI am %d years old.", age);
    output.close();
}

catch(Exception e) { e.printStackTrace(); }
}
}

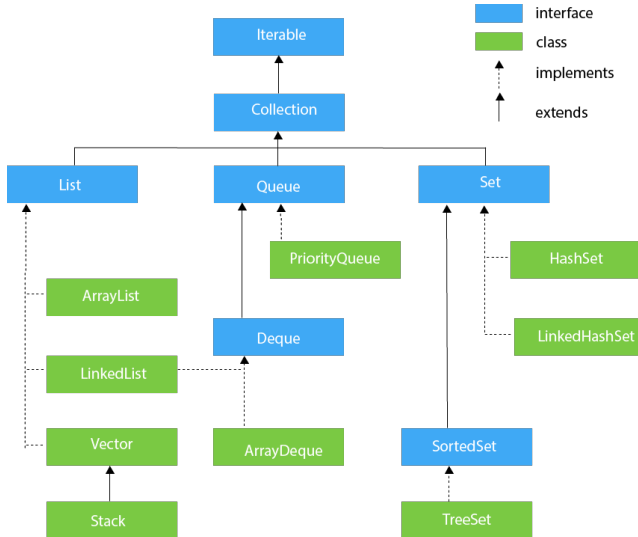
/*      Output.txt is Generated

Java is a class-based, OOP Language
that is designed to have as few implementation dependencies
as possible....
I am 50 years old.

*/
```

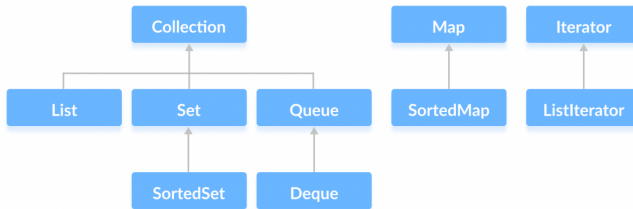

- It is a framework that provides an architecture to store and manipulate the group of objects.
- Java Collections can achieve all the operations that you perform on a data such as searching, sorting, insertion, manipulation, and deletion.
- Java Collection means a single unit of objects. Java Collection framework provides many interfaces (Set, List, Queue, Deque) and classes (ArrayList, Vector, LinkedList, PriorityQueue, HashSet, LinkedHashSet, TreeSet).
- **Java Collection** : represents a single unit of objects, i.e., a group.
- **Java Framework** : It provides readymade architecture and represents a set of classes and interfaces (Optional)
- **Collection framework** : represents a unified architecture for storing and manipulating a group of objects. It has: Interfaces and its implementations, i.e., Classes and Algorithms
- **Java collections framework** : It provides a set of **interfaces** and **classes** to implement various data structures and algorithms.

The **java.util** package contains all the classes and interfaces for the Collection framework.



The **java.util** package contains all the classes and interfaces for the Collection framework.

Java Collections Framework



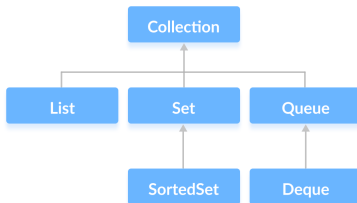
JCF

- It is the root interface of the collections framework hierarchy.
- Java does not provide direct implementations of the Collection interface but provides implementations of its subinterfaces like List, Set, and Queue



- Java **Collection** interface is the root interface of the collections framework.
- The Java Collection framework includes other interfaces as well: **Map** and **Iterator**. And may also have subinterfaces.
- **Subinterfaces** of the Collection Interface:
 - **List** Interface : is an ordered collection that allows us to add and remove elements like an array.
 - **Set** Interface : allows us to store elements in different sets similar to the set in mathematics. It cannot have duplicate elements.
 - **Queue** Interface : is used when we want to store and access elements in First In, First Out manner
- Java **Map** Interface : allows elements to be stored in **key/value** pairs. **Keys** are unique names that can be used to access a particular element in a map. And, each **Key** has a single **Value** associated with it.
- Java **Iterator** Interface : provides methods that can be used to access elements of collections

The **Collection** interface is the root interface of the Java collections framework.



Methods of Collection : includes various methods that can be used to perform different operations on objects.

1. `add()` - inserts the specified element to the collection
2. `size()` - returns the size of the collection
3. `remove()` - removes the specified element from the collection
4. `iterator()` - returns an iterator to access elements of the collection
5. `addAll()` - adds all the elements of a specified collection to the collection
6. `removeAll()` - removes all the elements of the specified collection from the collection
7. `clear()` - removes all the elements of the collection

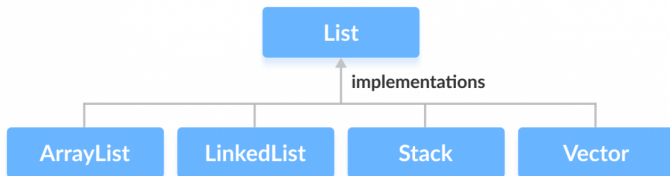


- 1 `public boolean add(E e)` : is used to insert an element in collection.
- 2 `public boolean addAll(Collection<? extends E> c)` : is used to insert the specified collection elements in the invoking collection.
- 3 `public boolean remove(Object element)` : used to delete an element.
- 4 `public boolean removeAll(Collection<?> c)` : used to delete all elements of the specified collection from the invoking collection.
- 5 `default boolean removeIf(Predicate<? super E> filter)` : is used to delete all the elements of the collection that satisfy the specified predicate.
- 6 `public boolean retainAll(Collection<?> c)` : It is used to delete all the elements of invoking collection except the specified collection.
- 7 `public int size()` : returns the total no. of elements in the collection.
- 8 `public void clear()` : removes all elements from the collection.
- 9 `public boolean contains(Object element)` : used to search an element.
- 10 `public boolean containsAll(Collection<?> c)` :
It is used to search the specified collection in the collection.

- 11 `public` `Iterator` `iterator()` : It returns an iterator.
- 12 `public` `Object[]` `toArray()` : It converts collection into array.
- 13 `public` `<T>` `T[]` `toArray(T[] a)` : It converts collection into array. Here the runtime type of the returned array is that of the specified array.
- 14 `public` `boolean` `isEmpty()` : It checks `if` collection is empty.
- 15 `default` `Stream<E>` `parallelStream()` :
It returns a possibly parallel `Stream` with the collection as its source.
- 16 `default` `Stream<E>` `stream()` : It returns a sequential `Stream` with the collection as its source.
- 17 `default` `Splitter<E>` `splitter()` :
It generates a `Splitter` over the specified elements in the collection.
- 18 `public` `boolean` `equals(Object element)` : It matches two collections.
- 19 `public` `int` `hashCode()` : It returns the hash code number of the collection.

List Interface

It is an ordered collection that allows us to **store** and **access** elements sequentially. And extends the **Collection** interface. **Classes that Implement List** :



How to use List? : We must import java.util.List package in order to use **List**.

```
// ArrayList implementation of List  
List<String> list1 = new ArrayList<>();
```

```
// LinkedList implementation of List  
List<String> list2 = new LinkedList<>();
```


Methods of List :

1. `add()` - adds an element to a list
2. `addAll()` - adds all elements of one list to another
3. `get()` - helps to randomly access elements from lists
4. `iterator()` - returns iterator object that can be used to sequentially access elements of lists
5. `set()` - changes elements of lists
6. `remove()` - removes an element from the list
7. `removeAll()` - removes all the elements from the list
8. `clear()` - removes all the elements from the list
(efficient than `removeAll()`)
9. `size()` - returns the length of lists
10. `toArray()` - converts a list into an array
11. `contains()` - returns `true` if a list contains specified element



Example-1 : Implementation of ArrayList I



The **ArrayList** class of the Java collections framework provides the functionality of **resizable-arrays**.

```
import java.util.List;  import java.util.ArrayList;

class ArrayList {
    public static void main(String[] args) {
        // Creating list using the ArrayList class
        List<Integer> MyList = new ArrayList<>();
        List<Integer> MyList1 = new ArrayList<>();
        MyList1.add(10);    MyList1.add(20); // adding Elements into MyList-1
        // Add elements to the list
        MyList.add(1);    MyList.add(5);    MyList.add(3);    MyList.add(7);
        MyList.addAll(2,MyList1);    // Adding Elements from MyList-1
        System.out.println("List: " + MyList);
        System.out.println("IndexOf : " + MyList.indexOf(10));
        // Access element from the list
        System.out.println("Accessed Element: " + MyList.get(2));
        // Remove element from the list
        System.out.println("Removed Element: " + MyList.remove(1));
        System.out.println("Accessed Element: " + MyList.set(3, 30));
        System.out.println("Size of MyList : " + MyList.size());
        System.out.println("Is 10 Exists in the List : " + MyList.contains(10));
        MyList.sort(null); // Sort Ascending Order;
        System.out.println("Sort List: " + MyList);
    }
}
```



Example-1 : Implementation of ArrayList II



```
System.out.println("Is List Empty : " + MyList.isEmpty());
MyList.clear();
System.out.println("Is List Empty : " + MyList.isEmpty());
}
}

/*
    List: [1, 5, 10, 20, 3, 7]
    IndexOf : 2
    Accessed Element: 10
    Removed Element: 5
    Accessed Element: 3
    Size of MyList : 5
    Is 10 Exists in the List : true
    Sort List: [1, 7, 10, 20, 30]
    Is List Empty : false
    Is List Empty : true

*/
```

- The **Vector** class is an implementation of the **List** interface that allows us to create resizable-arrays similar to the **ArrayList** class.
- **Java Vector vs. ArrayList :**
 - The **Vector** class synchronizes each individual operation. This means whenever we want to perform some operation on vectors, the Vector class automatically applies a lock to that operation.
 - In **ArrayList**, methods are not synchronized. Instead, it uses the **Collections.synchronizedList()** method that synchronizes the list as a whole.
- **Creating a Vector :**

```
import java.util.Vector;  
Vector<Type> vector = new Vector<>();
```

Example :

```
Vector<Integer> vector= new Vector<>();  
Vector<String> vector= new Vector<>();
```

- **Methods of Vector :**

1. `add(element)` - adds an element to vectors
2. `add(index, element)` - adds an element to the specified position
3. `addAll(vector)` - adds all elements of a vector to another vector
4. `get(index)` - returns an element specified by the index
5. `iterator()` - returns an iterator object to sequentially access vector elements
6. `remove(index)` - removes an element from specified position
7. `removeAll()` - removes all the elements
8. `clear()` - removes all elements. It is more efficient than `removeAll()`
9. `set()` - changes an element of the vector
10. `size()` - returns the size of the vector
11. `toArray()` - converts the vector into an array

- 12. `toString()` - converts the vector into a String
- 13. `contains()` - searches the vector for specified element and returns a boolean result

```
import java.util.Vector;

class JavaVectorClass {
    public static void main(String[] args) {
        Vector<String> animals= new Vector<>();
        animals.add("Dog");    animals.add("Horse");
        animals.add("Pig");    animals.add("Cat");
        System.out.println("Before Sort Vector: " + animals);
        animals.sort(null);    // Sort Animals
        System.out.println("After Sort Vector: " + animals);
        String element = animals.remove(1);    // Using remove()
        System.out.println("Removed Element: " + element);
        animals.add("Donkey");
        System.out.println("After Element Removed: " + animals);
        System.out.println("Index of Pig: " + animals.indexOf("Pig"));
        System.out.println("Check Horse : " + animals.contains("Horse"));
        System.out.println("Check Parrot : " + animals.contains("Parrot"));
        animals.add("Cow");
        System.out.println("Check Size : " + animals.size());
        animals.set(3, "Dog");
    }
}
```



```
System.out.println("After Set Vector: " + animals);
animals.clear();    // Using clear()
System.out.println("Vector after clear(): " + animals);
System.out.println("Check Empty or Not : " + animals.isEmpty());
}
}
/*
    Befor Sort Vector: [Dog, Horse, Pig, Cat]
    After Sort Vector: [Cat, Dog, Horse, Pig]
    Removed Element: Dog
    After Element Removed: [Cat, Horse, Pig, Donkey]
    Index of Pig: 2
    Check Horse : true
    Check Parrot : false
    Check Size : 5
    After Set Vector: [Cat, Horse, Pig, Dog, Cow]
    Vector after clear(): []
    Check Empty or Not : true
*/
```

- The **Stack** class extends the **Vector** class.
- In **Stack**, elements are stored and accessed in **Last In First Out**. i.e., elements are added/removed only at top of the stack.
- **Creating a Stack :**

```
import java.util.Stack  
Stack<Type> stacks = new Stack<>();
```

Examples:

```
Stack<Integer> stacks = new Stack<>();  
Stack<String> stacks = new Stack<>();
```

- **Methods of Stack :**

Since Stack **extends** the Vector **class**, it inherits all the methods Vector

Other Methods

1. push() - To add an element to the top of the stack.
2. pop() - To remove an element from the top of the stack
3. peek() - returns an object from the top of the stack
4. search() - To search an element in the stack.
5. empty() - To check whether a stack is empty or not




```
import java.util.Stack;

class JavaSTACK {
    public static void main(String[] args) {

        Stack<String> animals= new Stack<>();    // Stack Created

        animals.push("Horse");  animals.push("Cat");   animals.push("Dog");
        System.out.println("Stack: " + animals);

        animals.add("Zibra");
        System.out.println("Stack: " + animals);
        System.out.println("Peek Element : " + animals.peek());
        System.out.println("Element Poped : " + animals.pop());
        System.out.println("Search Dog : " + animals.search("Dog"));
        System.out.println("Search Dog : " + animals.contains("Dog"));

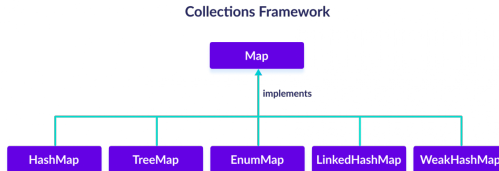
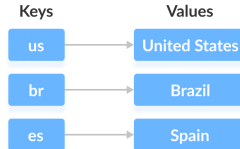
        System.out.println("Is the stack empty? " + animals.empty());
        animals.clear();    // Clear and Check for empty.
        System.out.println("Is the stack empty? " + animals.isEmpty());
    }
}

/*
```

```
Stack: [Horse, Cat, Dog]
Stack: [Horse, Cat, Dog, Zibra]
Peek Element : Zibra
Element Poped : Zibra
Search Dog : 1
Search Dog : true
Is the stack empty? false
Is the stack empty? true
```

**/*

- It is of Java collections framework provides the functionality of the **map** data structure.
- Elements of Map are stored in **key/value** pairs. **Keys** are unique values associated with individual **Values**.
- It cannot contain duplicate keys. And, each key is associated with a single value.



- **Methods of Map :**

1. `put(K, V)` - Inserts the **both** (K,V) into the map as Element.
If the key is present, its value is replaced.
2. `putAll()` - Inserts all the entries from the specified map.
3. `putIfAbsent(K, V)` - Inserts the association **if** the '**K**'
is not already associated with '**V**'.
4. `get(K)` - Returns the value associated with '**K**'.
If the key is not found, it returns **null**.
5. `getOrDefault(K, defaultValue)` - Returns the value associated with
the specified key '**K**'. If the key is not found,
it returns the **defaultValue**.
6. `containsKey(K)` - Checks **if** '**K**' is present in the map or not.
7. `containsValue(V)` - Checks **if** '**V**' is present in the map or not.
8. `replace(K, V)` - Replace the value '**K**' with the **new** '**V**'.
9. `replace(K, oldValue, newValue)` - Replaces the value of the '**K**'
with the **new** '**V**' only **if** the '**K**' is associated with '**V**'.



10. `remove(K)` - Removes the entry from the map represented by '`K`'.
11. `remove(K, V)` - Removes the entry from the map that has '`K`' with '`V`'.
12. `keySet()` - Returns a set of all the Key's present in a map.
13. `values()` - Returns a set of all the Values present in a map.
14. `entrySet()` - Returns a set of all the '`K`'/'`V`' mapping in a map.

```
import java.util.Map;  import java.util.HashMap;

class HashMAP {
    public static void main(String[] args) {
        // Creating a map using the HashMap
        Map<String, Integer> numbers = new HashMap<>();
        numbers.put("One", 1);  numbers.put("Two", 2);
        numbers.putIfAbsent("Three", 3);
        System.out.println("Map: " + numbers);
        System.out.println("Keys: " + numbers.keySet());
        System.out.println("Values: " + numbers.values());
        System.out.println("Entries: " + numbers.entrySet());
        System.out.println("Get the Value at K : " + numbers.get("Two"));
    }
}
```



```
System.out.println("Contains K :" + numbers.containsKey("Two"));
System.out.println("Contains K :" + numbers.containsValue(4));
numbers.replace("Two", 20);
System.out.println("After Replace Map Entries are : " + numbers);
numbers.replace("Two", 20, 2);
System.out.println("After Replace Map Entries are : " + numbers);
System.out.println("Removed Value: " + numbers.remove("Two"));
}
}

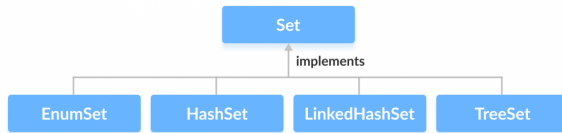
/*
Map: {One=1, Two=2, Three=3}
Keys: [One, Two, Three]
Values: [1, 2, 3]
Entries: [One=1, Two=2, Three=3]
Get the Value at K :2
Contains K :true
Contains K :false
After Replace Map Entries are : {One=1, Two=20, Three=3}
After Replace Map Entries are : {One=1, Two=2, Three=3}
Removed Value: 2
*/
```

1. A HashMap contains values based on the key.
 2. It contains only unique elements.
 3. It may have one null key and multiple null values.
 4. It maintains **no order**.
-
1. A LinkedHashMap contains values based on the key.
 2. It contains only unique elements.
 3. It may have one null key and multiple null values.
 4. It is same as HashMap instead maintains insertion order.
-
1. A TreeMap contains values based on the key. It implements the NavigableMap interface and extends AbstractMap class.
 2. It contains only unique elements.
 3. It cannot have null key but can have multiple null values.
 4. It is same as HashMap instead maintains ascending order(Sorted using the natural order of its key)
-
1. A Hashtable is an array of list. Each list is known as a bucket. The position of bucket is identified by calling the hashCode() method.
 2. A Hashtable contains values based on the key.
 3. It contains only unique elements.
 4. It may have not have any null key or value.
 5. It is synchronized and is a legacy class.

- Java LinkedHashMap
- Java WeakHashMap
- Java EnumMap
- Java SortedMap Interface
- Java NavigableMap Interface
- Java TreeMap
- Java ConcurrentMap Interface
- Java ConcurrentHashMap

The Above Data Structure have some additional Methods other than MAP, Kindly go through it.

- It is of Java Collections framework provides the features of the mathematical set in Java. And extends the **Collection** interface. Unlike the **List** interface, **Sets** cannot contain duplicate elements.
- The Following Classes uses the functionalities of Set Interface



- **Methods of Set Interface :**

1. `add()` - adds the specified element to the set
2. `addAll()` - adds all the elements of the specified collection to the set
3. `iterator()` - returns an iterator that can be used to access elements of the set sequentially
4. `remove()` - removes the specified element from the set
5. `removeAll()` - removes all the elements from the set.

- 6. `retainAll()` - retains all the elements in the set that are also present in another specified set
- 7. `clear()` - removes all the elements from the set
- 8. `size()` - returns the **length** (number of elements) of the set
- 9. `toArray()` - returns an array containing all the elements of the set
- 10. `contains()` - returns **true** if the set contains the specified element
- 11. `containsAll()` - returns **true** if the set contains all the elements of the specified collection
- 12. `hashCode()` - returns a hash code **value**
(address of the element in the set)

- **Set Operations :**

1. Union - to get the union of two sets x and y,
we can use `x.addAll(y)`
2. Intersection - to get the intersection of two sets x and y,
we can use `x.retainAll(y)`
3. Subset - to check if x is a subset of y,
we can use `y.containsAll(x)`

```
import java.util.Set; import java.util.HashSet;
```

```
class HashSet {  
    public static void main(String[] args) {  
        // Creating a set using the HashSet class  
        Set<Integer> set1 = new HashSet<>();  
  
        // Add elements to the set1  
        set1.add(2);    set1.add(3);  
        System.out.println("Set1: " + set1);  
  
        // Creating another set using the HashSet class  
        Set<Integer> set2 = new HashSet<>();
```



```
// Add elements
set2.add(1);    set2.add(2);
System.out.println("Set2: " + set2);

// Set2 Intersection Set1
set2.retainAll(set1);
System.out.println("Intersection of : " + set2);

// Set2 Union Set1
set2.addAll(set1);
System.out.println("Union is: " + set2);

System.out.println("Subset of :"+ set1.containsAll(set2));
}
}

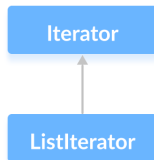
/*

Set1: [2, 3]
Set2: [1, 2]
Intersection of : [2]
Union is: [2, 3]
Subset of :true

*/
```

- The Java collections framework provides various algorithms that can be used to manipulate elements stored in data structures.
 - Algorithms in Java are static methods that can be used to perform various operations on collections.
 - Algorithms are generic in nature (i.e., Can apply on any Collections).
1. `sort()` : is used to sort elements.
 2. `shuffle()` : is used to destroy, the order present in Data Structure.
It does just the opposite of the sorting.
 3. `reverse()` : reverses the order of elements
 4. `fill()` : replace every element with the specified value
 5. `copy()` : creates a copy of elements from the source to destination
 6. `swap()` : swaps the position of two elements in a collection
 7. `addAll()` : adds all the elements of a collection to other collection
 8. `binarySearch()` : searches **for** the specified element and its position.
 9. `frequency()` : returns the count an element present in collection.
 10. `disjoint()` : checks **if** two collections contain some common element
 11. `min()` : are used to find the minimum element.
 12. `max()` : are used to find the maximum element.

- Iterator interface provides the facility of iterating the elements in a forward direction only.
- It allows us to access elements of a collection. It has a subinterface **ListIterator**.



- **Methods of Iterator:**

1. `hasNext()` : returns `true` if the iterator has more elements otherwise it returns `false`.
2. `next()` : returns the element and moves the cursor pointer to the next element.
3. `remove()` : removes the last elements returned by the iterator.
4. `forEachRemaining()` : performs the specified action `for` each remaining element of the collection



- It provides the functionality to access(bidirectional) elements of a list.

- **Methods of ListIterator:**

1. `hasNext()` : returns **true** if there exists an element in the list
2. `next()` : returns the next element of the list
3. `nextIndex()` : returns the index of the element that the `next()` method will **return**
4. `previous()` : returns the previous element of the list
5. `previousIndex()` : returns the index of the element that the `previous()` method will **return**
6. `remove()` : removes the element returned by either `next()` or `previous()`
7. `set()` : replaces the element returned by either `next()` or `previous()` with the specified element

```
import java.util.ArrayList; import java.util.ListIterator;
```

```
class LISTIterator {  
    public static void main(String[] args) {  
        // Creating an ArrayList  
        ArrayList<Integer> numbers = new ArrayList<>();  
        numbers.add(1); numbers.add(3); numbers.add(2);  
        System.out.println("ArrayList: " + numbers);  
  
        // Creating an instance of ListIterator  
        ListIterator<Integer> iterate = numbers.listIterator();  
    }  
}
```



```
// Using the next() method
System.out.println("Next Element: " + iterate.next());

// Using the nextIndex()
System.out.println("Position of Next Element: " + iterate.nextIndex());

// Using the hasNext() method
System.out.println("Is there any next element? " + iterate.hasNext());

System.out.println("Previous Element: " + iterate.previous());

// Using the previousIndex()
System.out.println("Position of the Previous element: " + iterate.previousIndex());
}
}

/*
    ArrayList: [1, 3, 2]
    Next Element: 1
    Position of Next Element: 1
    Is there any next element? true
    Previous Element: 1
    Position of the Previous element: -1
*/
```


- **Serialization** is a process of writing the state of an object into a byte-stream.
- It is mainly used in **Hibernate, RMI, JPA, EJB** and For serializing the object, we call the `writeObject()` method of `ObjectOutputStream`, and for deserialization we call the `readObject()` method of `ObjectInputStream` class.
- It is a mechanism to convert an object into stream of bytes so that it can be written into a file, transported through a network or stored into database.
- **De-serialization** is just a vice versa.

Or

- **Serialization** is converting an object to stream of bytes and **De-serialization** is rebuilding the object from stream of bytes.
- A class must implement **`java.io.Serializable`** interface to be eligible for serialization.
- Serializing the object done, by calling **`writeObject()`** method of `ObjectOutputStream`, and
- Deserialization we call the **`readObject()`** method of `ObjectInputStream` class.



```
// Student.java;
```

```
public class Student implements java.io.Serializable{
    private int stuRollNum, stuAge;
    private String stuName;
    private transient String stuAddress;
    private transient int stuHeight;

    public Student(int roll, int age, String name, String address, int height){
        this.stuRollNum = roll;      this.stuAge = age;
        this.stuName = name;         this.stuAddress = address;
        this.stuHeight = height;
    }
    public int getStuRollNum() { return stuRollNum; }
    public int getStuAge() { return stuAge; }
    public String getStuName() { return stuName; }
    public String getStuAddress() { return stuAddress; }
    public int getStuHeight() { return stuHeight; }
}
```

```
// StudentSerialization.java
```

```
import java.io.FileOutputStream; import java.io.ObjectOutputStream;
import java.io.IOException;
```



```
public class StudentSerialization {
    public static void main(String args[]) {
        Student obj = new Student(50396, 31, "Pradeep K V", "Chennai", 6);
        try{
            FileOutputStream fos = new FileOutputStream("Student.ser");
            ObjectOutputStream oos = new ObjectOutputStream(fos);
            oos.writeObject(obj);
            oos.close();    fos.close();
            System.out.println("Serialization Done!!");
        }catch(IOException ioe){ System.out.println(ioe); }
    }
}
```

```
/*  Serialization Done!!  */
// StudentDeSerialization.java
```

```
import java.io.FileInputStream;    import java.io.ObjectInputStream;
import java.io.IOException;
public class StudentDeSerialization {
    public static void main(String args[]) {
        Student o=null;

        try{
            FileInputStream fis = new FileInputStream("Student.ser");
            ObjectInputStream ois = new ObjectInputStream(fis);
```



```
o = (Student)ois.readObject();   ois.close();   fis.close();
}
catch(IOException ioe) { ioe.printStackTrace(); return; }
catch(ClassNotFoundException cnfe) {
    System.out.println("Student Class is not found.");
    cnfe.printStackTrace();   return;
}
System.out.println("Student Name:"+o.getStuName());
System.out.println("Student Age:"+o.getStuAge());
System.out.println("Student Roll No:"+o.getStuRollNum());
System.out.println("Student Address:"+o.getStuAddress());
System.out.println("Student Height:"+o.getStuHeight());
}
}

/*
    Student Name:Pradeep K V
    Student Age:31
    Student Roll No:50396
    Student Address:null
    Student Height:0
*/
```

- Java transient keyword is used in serialization. If you define any data member as transient, it will not be serialized.
- Example : In a class as Student, it has three data members **id**, **name** and **age**. If you serialize the object, all the values will be serialized but don't want to serialize one value, e.g. Age then we can declare the age data member as **transient**.

```
import java.io.*;

class Emp implements Serializable {
    private static final long serialVersionUID = 50396L;
    transient int a;
    static int b;
    String name;
    int age;

    // Default constructor
    public Emp(String name, int age, int a, int b) {
        this.name = name;    this.age = age;    this.a = a;    this.b = b;
    }
}

public class Serial_DeSerial_Test {
```

```
public static void printdata(Emp object1) {
    System.out.println("name = " + object1.name);
    System.out.println("age = " + object1.age);
    System.out.println("a = " + object1.a);
    System.out.println("b = " + object1.b);
}

public static void main(String[] args) {
    Emp object = new Emp("Pradeep K V", 30, 2, 1000);
    String filename = "KVP.txt";
    // Serialization
    try { // Saving of object in a file
        FileOutputStream file = new FileOutputStream(filename);
        ObjectOutputStream out = new ObjectOutputStream(file);
        // Method for serialization of object
        out.writeObject(object);
        out.close(); file.close();
        System.out.println("Object has been serialized Data \n"
            + "before Deserialization.");

        printdata(object);
        object.b = 2000; // value of static variable changed
    }
    catch (IOException ex) { System.out.println("IOException is caught");

    object = null;
    // Deserialization
```

```

try {           // Reading the object from a file
    FileInputStream file = new FileInputStream(filename);
    ObjectInputStream in = new ObjectInputStream(file);
    // Method for deserialization of object
    object = (Emp)in.readObject();
    in.close(); file.close();
    System.out.println("Object has been deserialized Data \n"
        + " after Deserialization.");
    printdata(object);
}
catch (IOException ex) { System.out.println("IOException is caught");
catch (ClassNotFoundException ex) {
    System.out.println("ClassNotFoundException is caught");
}
}
}

/*           Object has been serialized Data before Deserialization.
name = Pradeep K V
age = 30,  a = 2,  b = 1000
Object has been deserialized Data after Deserialization.
name = Pradeep K V
age = 30,  a = 0,  b = 2000          */

```

Thanks



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

*Thank
you!*



Java Programming

JavaFX and JDBC

Dr. Pradeep K V

Associate Professor (Sr.)

School of Computer Science and Engineering

VIT - Chennai



Java Programming

JDBC - Java Database Connectivity

Dr. Pradeep K V

Associate Professor (Sr.)

School of Computer Science and Engineering

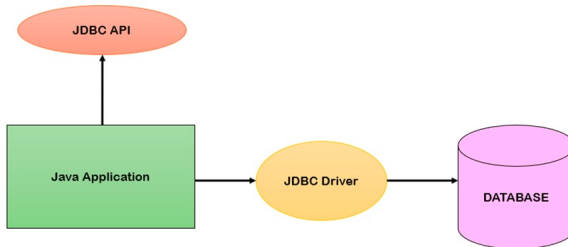
VIT - Chennai



- **JDBC** (Java Database Connectivity)
- JDBC Introduction, JDBC Classes and Interfaces
- JDBC Drivers and steps to Connect Java App & Database
- JDBC to connect MySQL (Oracle JDK - Practice)
- DriverManager Class, Connection Interface, Statement Interface
- ResultSet Interface, PreparedStatement Interface, ResultSetMetaData Interface DatabaseMetaData Interface.
- Examples....

- Introduction to **JavaFX** and its Features
- JavaFX Overview and its Architecture
- JavaFX Stage, Scene , Scene Graphs, Nodes, Controls, Layout
- Java UI Controls : Labels, Buttons : Radio Buttons, CheckBox Buttons,
- TextField, PasswordField, Menu Bar, ToolTip, Progress Bar, Progress Indicator, Status Bar, Text, Color, Font.

- JDBC stands for Java Database Connectivity.
- JDBC is a Java API to connect and execute the query with the database.
- It is a part of JavaSE (Java Standard Edition).
- JDBC API uses JDBC drivers to connect with the database.
- JDBC enables Java applications to connect to Relational databases like MySQL, PostgreSQL, MSSQL Server, Oracle , H2 Database etc.
- JDBC API makes is possible to query and update relational databases, as well as call stored procedures, and obtain meta data about the database.



Why to Learn JDBC?

JDBC is a standard Java API for database-independent connectivity between the Java programming language and a wide range of databases.

The JDBC library includes APIs for each of the tasks mentioned below that are commonly associated with database usage.

- Making a connection to a database.
- Creating SQL or MySQL statements.
- Executing SQL or MySQL queries in the database.
- Viewing & Modifying the resulting records.

Applications of JDBC

- 1 Java Applications
- 2 Java Applets(JavaFX) and Java Servlets
- 3 Java Server Pages (JSPs)
- 4 Enterprise JavaBeans (EJBs).content...



JDBC is Database Independent

- The Java JDBC API standardizes how to connect to a database, how to execute queries against it, how to navigate the result of such a query, and how to execute updates in the database, how to call stored procedures, and how to obtain meta data from the database.
- "**Standardizes** " means, the code looks same across different database products. Thus, changing to another database will be a lot easier, if your project needs that in the future

JDBC is Not SQL Independent

- JDBC does not standardize the SQL sent to the database.
- The SQL is written by the programmer, the user of the JDBC API.
- The SQL dialect used by the various different databases will vary slightly, so to be 100% database independent , you SQL must also be 100% database independent (i.e. use commands understood by all databases)



JDBC is Not For Non-Relational Databases

- The Java JDBC API is intended for interaction with relational databases, meaning databases that you interact with via standard SQL.
- The JDBC API is not intended for non-relational databases like MongoDB, Cassandra, Dynamo etc. (NoSQL)
- JDBC API to access tabular data stored in any relational database. (Structured)

Popular Relational Databases

- H2Database
- MariaDB
- PostgreSQL
- Derby
- Microsoft SQL Server
- Oracle

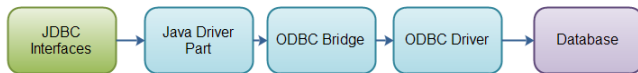


- The current version of JDBC (**Stable**) is 4.3.
- The **java.sql** package contains classes and interfaces for JDBC API.
- A list of popular interfaces of JDBC API are given below:
 - **Driver** interface
 - **Connection** interface
 - **Statement** interface
 - **PreparedStatement** interface
 - **CallableStatement** interface
 - **ResultSet** interface
 - **ResultSetMetaData** interface
 - **DatabaseMetaData** interface
 - **RowSet** interface
- A list of popular classes of JDBC API are given below:
 - **DriverManager** class
 - **Blob** class
 - **Clob** class
 - **Types** class

JDBC Driver is a software component that enables java application to interact with the database. (4 types of JDBC drivers).

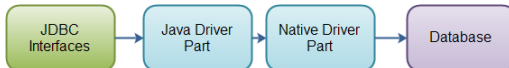
- **Type-1 : JDBC-ODBC** bridge driver :

- It translates the JDBC interface calls to ODBC calls
- An ODBC bridge then calls the ODBC driver of the given database. (were) mostly intended to be used in the beginning, when there were no type 4 drivers (all Java drivers)



- **Type-2 : Native-API** driver :

- It is like a Type 1 driver, except the ODBC part is replaced with a native code part instead.
- The native code part is targeted at a specific database product.



- **Type-3 : Network Protocol (fully java)** driver :

- It is an all Java driver, that sends the JDBC interface calls to an intermediate server, which in-turn connects to the database on behalf of the JDBC driver



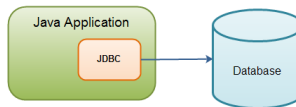
- **Type-4 : Thin (fully java)** driver :

- It is an all Java driver which connects directly to the database. It is implemented for a specific database product. Today, most JDBC drivers are type 4 drivers



There are 5 steps to connect any java application with the database using JDBC. These steps are as follows:

- **Register** the Driver class
- **Create** connection
- **Create** statement
- **Execute** queries
- **Close** connection



- ④ Register(Load) the driver class : The `forName()` method of Class is used to register and to dynamically load the driver class.

Syntax :

```
Class.forName("driverClassName");
```

Example :

```
Class.forName("oracle.jdbc.driver.OracleDriver");
```

- ② Create the connection object : The `getConnection()` method of **DriverManager** class is used to establish connection with the database.

Syntax :

```
String url, usr, passwd;
```

```
1) public static Connection getConnection(url)
```

`throws` SQLException

```
2) public static Connection getConnection(url,usr,password)
```

`throws` SQLException

Example :

```
String url = "jdbc:h2: /test"
```

```
String user = "deepu" ;
```

```
String passwd = "*****" ;
```

```
1. Connection con =Driver Manager. getConnection("url");
```

```
2. Connection con =Driver Manager. getConnection  
("url",user,passwd);
```



- 3 Create the Statement object : The **createStatement()** method of Connection interface is used to create statement. The object of statement is responsible to execute queries with the database.
 - 1) **public Statement createStatement()** :
 - 2) **public Statement createStatement(int resultSetType, int resultSetConcurrency)** :
Creates a Statement object that will generate ResultSet objects with the given type and concurrency.
 - 3) **public void setAutoCommit(boolean status)** :
is used to set the commit status. By default it is true.
 - 4) **public void commit()** :
saves the changes made since the previous commit/rollback permanent.
 - 5) **public void rollback()** :
Drops all changes made since the previous commit/rollback.
 - 6) **public void close()** :
closes the connection and Releases a JDBC resources immediately.
- 4 Execute the query : The **executeQuery()** method of **Statement** interface is used to execute queries to the database. It returns the object of **ResultSet** that can be used to get all the records of a table.

- 1) `public ResultSet executeQuery(String sql):`
is used to execute SELECT query.
It returns the object of ResultSet.
- 2) `public int executeUpdate(String sql):`
is used to execute specified query,
it may be create, drop, insert, update, delete etc.
- 3) `public boolean execute(String sql):`
is used to execute queries that may `return` multiple results.
- 4) `public int[] executeBatch():`
is used to execute batch of commands.

- Close the connection object : By closing connection **object** statement and **ResultSet** will be closed automatically. The **close()** method of Connection interface is used to close the connection.

```
Statement stmt = con.createStatement(
    ResultSet.TYPE_SCROLL_INSENSITIVE,
    ResultSet.CONCUR_UPDATABLE);
```

Table: List of JDBC driver names and database URL

RDBMS	JDBC driver name	URL format
MySQL	com.mysql.cj.jdbc.Driver	jdbc:mysql://hostname/ databaseName
ORACLE	oracle.jdbc.driver.OracleDriver	jdbc:oracle:thin:@hostname: portNumber:databaseName
DB2	com.ibm.db2.jdbc.net.DB2Driver	jdbc:db2:hostname:portNumber/ databaseName
Sybase	com.sybase.jdbc.SybDriver	jdbc:sybase:Tds:hostname: portNumber/databaseName

Example-1 : JDBC to connect MySQL- I



```
import java.sql.*;
class EmpDBTest{
    public static void main(String args[]){
        try{
            final String JDBC_DRIVER = "com.mysql.cj.jdbc.Driver";
            final String DB_URL = "jdbc:mysql://localhost:3306/kvp";
            final String USER = "pradeep";
            final String PASS = "password";

            // Driver Registering (Loading)
            Class.forName(JDBC_DRIVER);

            // Connection Establishment (Create Connection)
            Connection con=DriverManager.getConnection(DB_URL,USER,PASS);

            // Statement Creation
            Statement stmt=con.createStatement();

            // Execute a Statement
            ResultSet rs=stmt.executeQuery("select * from emp");
            while(rs.next())
                System.out.println(rs.getInt(1)+"    "+rs.getString(2)
                                   +"    "+rs.getString(3));
```




```
        con.close();    // Close Connection
    }
    catch(Exception e){ System.out.println(e);}
}
}
```

50396	Deepu	42
50394	Nachi	41
50389	Tulasi	38
50391	R K Singh	48
50393	Ramesh R	48

- It acts as an interface between user and drivers.
- It keeps track of the drivers that are available and handles establishing a connection between a database and the appropriate driver.
- It maintains a list of Driver classes that have registered themselves by calling the method **DriverManager.registerDriver()**.

Useful methods of **DriverManager** class

1. `public static void registerDriver(Driver driver) :`
is used to register the given driver with DriverManager.
2. `public static void deregisterDriver(Driver driver) :`
is used to deregister/drop the given driver with DriverManager.
3. `public static Connection getConnection(String url) :`
is used to establish the connection with the specified url.
4. `public static Connection getConnection(url, userName, passWord);`
is used to establish the connection with the specified url, username and password.

- A Connection is the session between java application and database.
- This interface is a factory of **Statement**, **PreparedStatement**, and **DatabaseMetaData** i.e. object of Connection can be used to get the object of **Statement** and **DatabaseMetaData** .
- It provide many methods for transaction management like **commit()**, **rollback()** etc.

Commonly used methods

- 1) `public Statement createStatement()` :
- 2) `public Statement createStatement(int resultSetType, int resultSetConcurrency)`
Creates a Statement object that will generate ResultSet objects with the given type and concurrency.
- 3) `public void setAutoCommit(boolean status):`
is used to set the commit status. By default it is true.
- 4) `public void commit():`
saves the changes made since the previous commit/rollback permanent.
- 5) `public void rollback():`
Drops all changes made since the previous commit/rollback.
- 6) `public void close():`
closes the connection and Releases a JDBC resources immediately.



- It provides methods to execute queries with the database.
- It is a factory of **ResultSet** i.e. it provides factory method to get the object of ResultSet.

Commonly used methods

- 1) `public ResultSet executeQuery(String sql):`
is used to execute SELECT query.
It returns the object of ResultSet.
- 2) `public int executeUpdate(String sql):`
is used to execute specified query,
it may be create, drop, insert, update, delete etc.
- 3) `public boolean execute(String sql):`
is used to execute queries that may `return` multiple results.
- 4) `public int[] executeBatch():`
is used to execute batch of commands.

- The object of **ResultSet** maintains a cursor pointing to a row of a table. Initially, cursor points to before the first row.
- By default, **ResultSet** object can be moved forward only and it is not updatable.
- But, can make this object to move **forward/backward** direction by passing either **TYPE_SCROLL_INSENSITIVE** or **TYPE_SCROLL_SENSITIVE** in **createStatement(int,int)** method and can make this object as updatable by:

```
Statement stmt = con.createStatement(  
    ResultSet.TYPE_SCROLL_INSENSITIVE,  
    ResultSet.CONCUR_UPDATABLE);
```

Commonly used methods

- 1) `public boolean next():`
is used to move the cursor to the one row next from the current position.
- 2) `public boolean previous():`
is used to move the cursor to the one row previous from the current position.
- 3) `public boolean first():`
is used to move the cursor to the first row in resultset object.
- 4) `public boolean last():`
is used to move the cursor to the last row in resultset object.
- 5) `public boolean absolute(int row):`
is used to move the cursor to the specified row number in the ResultSet object.
- 6) `public boolean relative(int row):`



is used to move the cursor to the relative row number in the ResultSet object, it may be positive or negative.

- 7) `public int getInt(int columnIndex):`
is used to `return` the data of specified column index of the current row as `int`.
- 8) `public int getInt(String columnName):`
is used to `return` the data of specified column name of the current row as `int`.
- 9) `public String getString(int columnIndex):`
is used to `return` the data of specified column index of the current row as `String`.
- 10) `public String getString(String columnName):`
is used to `return` the data of specified column name of the current row as `String`.

- It is a subinterface of Statement and used to execute parameterized query.
- String sql="insert into emp values(?,?,?);"
- Why use PreparedStatement?
 - **Improves performance:** The performance of the application will be faster if used **PreparedStatement** interface because query is compiled only once.

Commonly used methods

1. `public void setInt(int paramIndex, int value)`
2. `public void setString(int paramIndex, String value)`
sets the integer/String value to the given parameter index.
3. `public void setFloat(int paramIndex, float value)`
4. `public void setDouble(int paramIndex, double value)`
sets the float/double value to the given parameter index.
5. `public int executeUpdate()` : executes the query.
It is used for create, drop, insert, update, delete etc.
6. `public ResultSet executeQuery()`
executes the select query. It returns an instance of ResultSet.



- The metadata means data about data i.e. can get further information from the data.
- It is useful, when to get metadata of a table like total number of **column**, **column name**, **column type** etc.

Commonly used methods

1. `public int getColumnCount() throws SQLException :`
it returns the total number of columns in the ResultSet object.
2. `public String getColumnName(int index) throws SQLException :`
it returns the column name of the specified column index.
3. `public String getColumnName(int index) throws SQLException:`
it returns the column type name for the specified index.
4. `public String getTableName(int index) throws SQLException :`
it returns the table name for the specified column index.

- It provides methods to get meta data of a database such as **database product name, database product version, driver name, name of total number of tables, name of total number of views** etc.

Commonly used methods

1. `public String getDriverName() throws SQLException:`
2. `public String getDriverVersion() throws SQLException:`
it returns the name and version number of the JDBC driver.
3. `public String getUsername() throws SQLException:`
it returns the username of the database.
4. `public String getDatabaseProductName() throws SQLException:`
5. `public String getDatabaseProductVersion() throws SQLException:`
it returns the product name and version of the database.
6. `public ResultSet getTables(catalog, schemaPattern, tableNamePattern, types[]) throws SQLException:`
it returns the description of the tables of the specified catalog.
(TABLE, VIEW, ALIAS, SYSTEM TABLE, SYNONYM etc...)



Example-2 : Insert Record- I



```
import java.sql.*;
class insertEmpRec{
    public static void main(String args[]){

        final String JDBC_DRIVER = "com.mysql.cj.jdbc.Driver";
        final String DB_URL = "jdbc:mysql://localhost:3306/kvp";
        final String USER = "pradeep";
        final String PASS = "password";

        try{
            Class.forName(JDBC_DRIVER);
            Connection con=DriverManager.getConnection(DB_URL,USER,PASS);
            PreparedStatement stmt=con.prepareStatement(
                "insert into emp values(?,?,?)");
            //1 specifies the first parameter
            stmt.setInt(1,50390);
            stmt.setString(2,"Alok");
            stmt.setInt(3,56);

            int i=stmt.executeUpdate();
        }
    }
}
```



Example-2 : Insert Record- II



```
        System.out.println(i+" records inserted");  
        con.close();  
    }catch(Exception e){ System.out.println(e);}  
}  
}
```

```
mysql> select * from emp;  
+-----+-----+-----+  
| id      | name      | age  |  
+-----+-----+-----+  
| 50396   | Deepu     | 42   |  
| 50394   | Nachi     | 41   |  
| 50389   | Tulasi    | 38   |  
| 50391   | R K Singh | 48   |  
| 50393   | Ramesh R  | 48   |  
| 50001   | Pradeep   | 43   |  
+-----+-----+-----+  
6 rows in set (0.01 sec)
```

Before

```
mysql> select * from emp;  
+-----+-----+-----+  
| id      | name      | age  |  
+-----+-----+-----+  
| 50396   | Deepu     | 42   |  
| 50394   | Nachi     | 41   |  
| 50389   | Tulasi    | 38   |  
| 50391   | R K Singh | 48   |  
| 50393   | Ramesh R  | 48   |  
| 50001   | Pradeep   | 43   |  
| 50390   | Alok      | 56   |  
+-----+-----+-----+  
7 rows in set (0.00 sec)
```

After



Example-3 : Delete Record- I



```
import java.sql.*;
class deleteEmpRec{
    public static void main(String args[]){

        final String JDBC_DRIVER = "com.mysql.cj.jdbc.Driver";
        final String DB_URL = "jdbc:mysql://localhost:3306/kvp";
        final String USER = "pradeep";
        final String PASS = "password";

        try{
            Class.forName(JDBC_DRIVER);
            Connection con=DriverManager.getConnection(DB_URL,USER,PASS);
            PreparedStatement stmt=con.prepareStatement(
                "delete from emp where id=?");

            //1 specifies the first parameter in the query
            stmt.setInt(1,50393);

            int i=stmt.executeUpdate();
            System.out.println(i+" records inserted");
        }
    }
}
```



Example-3 : Delete Record- II



```
        con.close();  
    }catch(Exception e){ System.out.println(e);}  
}  
}
```

```
mysql> select * from emp;  
+-----+-----+-----+  
| id    | name      | age  |  
+-----+-----+-----+  
| 50396 | Deepu     | 42   |  
| 50394 | Nachi     | 41   |  
| 50389 | Tulasi    | 38   |  
| 50391 | R K Singh | 48   |  
| 50393 | Ramesh R  | 48   |  
| 50001 | Pradeep   | 43   |  
| 50390 | Alok      | 56   |  
+-----+-----+-----+  
7 rows in set (0.00 sec)
```

Before

```
mysql> select * from emp;  
+-----+-----+-----+  
| id    | name      | age  |  
+-----+-----+-----+  
| 50396 | Deepu     | 42   |  
| 50394 | Nachi     | 41   |  
| 50389 | Tulasi    | 38   |  
| 50391 | R K Singh | 48   |  
| 50001 | Pradeep   | 43   |  
| 50390 | Alok      | 56   |  
+-----+-----+-----+  
6 rows in set (0.00 sec)
```

After



```
import java.sql.*;
class updateEmpRec{
    public static void main(String args[]){

        final String JDBC_DRIVER = "com.mysql.cj.jdbc.Driver";
        final String DB_URL = "jdbc:mysql://localhost:3306/kvp";
        final String USER = "pradeep";
        final String PASS = "password";

        try{
            Class.forName(JDBC_DRIVER);
            Connection con=DriverManager.getConnection(DB_URL,USER,PASS);
            PreparedStatement stmt=con.prepareStatement(
                "update emp set name=? where id=?");

            //1 specifies the first parameter in the query i.e. name
            stmt.setString(1,"Alok Chauhan");
            //1 specifies the first parameter in the query
            stmt.setInt(2,50390);
```

Example-4 : Update Record- II



```
int i=stmt.executeUpdate();  
System.out.println(i+" records Updated");  
con.close();  
}catch(Exception e){ System.out.println(e);}  
}  
}
```

```
mysql> select * from emp;  
+-----+-----+-----+  
| id      | name      | age  |  
+-----+-----+-----+  
| 50396   | Deepu     | 42   |  
| 50394   | Nachi     | 41   |  
| 50389   | Tulasi    | 38   |  
| 50391   | R K Singh | 48   |  
| 50001   | Pradeep   | 43   |  
| 50390   | Alok      | 56   |  
+-----+-----+-----+  
6 rows in set (0.00 sec)
```

Before

```
mysql> select * from emp;  
+-----+-----+-----+  
| id      | name      | age  |  
+-----+-----+-----+  
| 50396   | Deepu     | 42   |  
| 50394   | Nachi     | 41   |  
| 50389   | Tulasi    | 38   |  
| 50391   | R K Singh | 48   |  
| 50001   | Pradeep   | 43   |  
| 50390   | Alok Chauhan | 56   |  
+-----+-----+-----+  
6 rows in set (0.00 sec)
```

After



Example-5 : Describe Table- I



```
import java.sql.*;
class describeEmpRec{
    public static void main(String args[]){

        final String JDBC_DRIVER = "com.mysql.cj.jdbc.Driver";
        final String DB_URL = "jdbc:mysql://localhost:3306/kvp";
        final String USER = "pradeep";
        final String PASS = "password";

        try{
            Class.forName(JDBC_DRIVER);
            Connection con=DriverManager.getConnection(DB_URL,USER,PASS);

            PreparedStatement ps=con.prepareStatement("select * from emp");
            ResultSet rs=ps.executeQuery();
            ResultSetMetaData rsmd=rs.getMetaData();

            System.out.println("Total columns: "+rsmd.getColumnCount());
            System.out.println("First Column Name: "+rsmd.getColumnName(1));
            System.out.println("First Column Type: "+rsmd.getColumnTypeName(1));
            System.out.println("Second Column Name: "+rsmd.getColumnName(2));
            System.out.println("Second Column Type: "+rsmd.getColumnTypeName(2));
            System.out.println("Third Column Name: "+rsmd.getColumnName(3));
            System.out.println("Third Column Type: "+rsmd.getColumnTypeName(3));
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Example-5 : Describe Table- II



```
int i=stmt.executeUpdate();  
System.out.println(i+" records Updated");  
con.close();  
}catch(Exception e){ System.out.println(e);}  
}  
}
```

```
/*  
    Total columns: 3  
    First Column Name: id  
    First Column Type: INT  
    Second Column Name: name  
    Second Column Type: VARCHAR  
    Third Column Name: age  
    Third Column Type: INT  
*/
```

```
mysql> desc emp;
```

Field	Type	Null	Key	Default	Extra
id	int	YES		NULL	
name	varchar(40)	YES		NULL	
age	int	YES		NULL	



Example-6 : Describe Database- I



```
import java.sql.*;
class DataBaseDesc{
    public static void main(String args[]){

        final String JDBC_DRIVER = "com.mysql.cj.jdbc.Driver";
        final String DB_URL = "jdbc:mysql://localhost:3306/kvp";
        final String USER = "pradeep";
        final String PASS = "password";

        try{
            Class.forName(JDBC_DRIVER);
            Connection con=DriverManager.getConnection(DB_URL,USER,PASS);

            DatabaseMetaData dbmd=con.getMetaData();

            System.out.println("Driver Name: "+dbmd.getDriverName());

            System.out.println("Driver Version: "+dbmd.getDriverVersion());

            System.out.println("UserName: "+dbmd.getUserName());

            System.out.println("Database Product Name: "
                               +dbmd.getDatabaseProductName());
```



```
        System.out.println("Database Product Version: "
                            +dbmd.getDatabaseProductVersion());

        con.close();
    }catch(Exception e){ System.out.println(e);}
}
```

```
Driver Name: MySQL Connector/J
Driver Version: mysql-connector-java-8.0.23
(Revision: 18bbd5e68195d0da083cbd5bd0d05d76320df7cd)
UserName: pradeep@localhost
Database Product Name: MySQL
Database Product Version: 8.0.23
```

Thanks



*Thank
you!*

Java Programming

JavaFX - Java Special Effects

Dr. Pradeep K V
Associate Professor (Sr.)
School of Computer Science and Engineering
VIT - Chennai



- JavaFX is a Java library used to develop Desktop applications as well as Rich Internet Applications (RIA).
- The applications built in JavaFX, can run on multiple platforms including Web, Mobile and Desktops.
- JavaFX is intended to replace swings in Java applications as a GUI framework.
- JavaFX also provides its own components and doesn't depend upon the operating system.
- It is lightweight and hardware accelerated.
- It supports various operating systems including Windows, Linux and Mac OS.

History

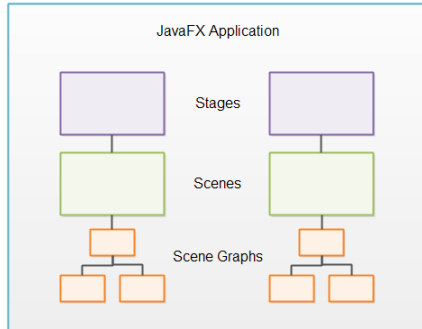
Developed by Chris Oliver and called as F3 (**Form Follows Function**). Sun Micro-systems acquired F3 project as JavaFX in June, 2005. JavaFX 19 is the latest JavaFX release launched in September 2022.



Feature	Description
Java Library	It is a Java library which consists of many classes and interfaces that are written in Java.
FXML	FXML is the XML based Declarative language. The coding can be done in FXML to provide the more enhanced GUI to the user.
Scene Builder	Scene Builder generates FXML mark-up which can be ported to an IDE.
Web view	Web pages can be embedded with JavaFX Apps. Web View uses WebKitHTML technology to embed web pages.
Built in UI controls	JavaFX contains Built-in components which are not dependent on operating system. The UI component are just enough to develop a full featured application.
CSS like styling	JavaFX code can be embedded with the CSS to improve the style of the application. We can enhance the view of our application with the simple knowledge of CSS.

Feature	Description
Swing interoperability	The JavaFX applications can be embedded with swing code using the Swing Node class. We can update the existing swing application with the powerful features of JavaFX.
Canvas API	Canvas API provides the methods for drawing directly in an area of a JavaFX scene.
Rich Set of APIs	JavaFX provides a rich set of API's to develop GUI applications.
Integrated Graphics Library	An integrated set of classes are provided to deal with 2D and 3D graphics.
Graphics Pipeline	JavaFX graphics are based on Graphics rendered pipeline(prism). It offers smooth graphics which are hardware accelerated.
High Performance Media Engine	The media pipeline supports the playback of web multimedia on a low latency. It is based on a Gstreamer Multimedia framework.
Self-contained application deployment model	Self Contained application packages have all of the application resources and a private copy of Java and JavaFX Runtime.

- JavaFX comes with a large set of built-in GUI components, like **buttons**, **Text-fields**, **tables**, **trees**, **menus**, **charts** and much more.
- JavaFX components can be styled using **CSS**, and can use FXML to compose a GUI instead of doing it in Java code.
- JavaFX contains a set of ready-to-use chart components, so you don't have to code that from scratch every time you need a basic chart.
- JavaFX also comes with support for **2D** and **3D** graphics as well as audio and video support.
- JavaFX even contains a **WebView** based on the popular WebKit browser, so you can embed web pages or web applications inside JavaFX.



Overview

- A JavaFX application contains one or more **Stages** which corresponds to windows.
- Each **Stages** has a scene attached to it.
- Each **Scene** can have an object graph of **controls** , **layouts** etc. attached to it, called the **Scene Graphs**.

Stages :

- It is the outer frame for a JavaFX application corresponds to a window.
- An application can have multiple windows open. Each with own stage.
- Each **stage** is represented by a Stage object inside a JavaFX application.
- A JavaFX application has a primary Stage object which is created for us by the JavaFX runtime.
- A JavaFX application can create additional Stage objects if it needs additional windows open. For instance, for **dialogs**, **wizards** etc.

Scene :

- It is required, to display anything on a stage in a JavaFX application.
- A stage can only show one scene at a time, but it is possible to exchange the scene at runtime.
- A scene is represented by a Scene object inside a JavaFX application.
- A JavaFX application must create all Scene objects it needs

Scene Graph :

- All visual components (controls , layouts etc.) must be attached to a scene to be displayed, and that scene must be attached to a stage for the whole scene to be visible.
- The total object graph of all the controls, layouts etc. attached to a scene is called the scene graph.

Node :

- All components attached to the scene graph are called nodes . All nodes are sub classes of a **JavaFX** class called *javafx.scene.Node* .
- There are two types of nodes: **Branch** nodes and **Leaf** nodes.
- A **Branch** node is a node that can contain other nodes (child nodes). Branch nodes are also referred to as parent nodes because they can contain child nodes.
- A **Leaf** node which cannot contain other nodes.

Controls :

- JavaFX controls are JavaFX components which provide some kind of control functionality inside a JavaFX application. For instance, a **button**, **radio button**, **table**, **tree** etc.
- For a control to be visible it must be attached to the scene graph of some Scene object.
- Controls are usually nested inside some JavaFX layout component that manages the layout of controls relative to each other.

- The GUI of every desktop application mainly considers **UI elements**, **layouts** and **behaviour**.
- The **UI** elements are the one which are actually shown to the user for interaction or information exchange.
- **Layout** defines the organization of the UI elements on the screen.
- **Behaviour** is the reaction of the UI element when some event is occurred on it.
- However, the package **javafx.scene.control** provides all the necessary classes for the UI components like **Button**, **Label**, etc.
- Every class represents a specific UI control and defines methods for their styling.

Java UI Control Components

SN	Control	Description
1	Label	is used to define a simple text on the screen. and is placed with the node, it describes.
2	Button	controls the function of the application. Button class is used to create a labelled button.



3	RadioButton	is used to provide various options to the user. The user can only choose one option among all. It is either selected or deselected.
4	CheckBox	is used to get the kind of information from the user which contains various choices. User marked the checkbox either on (true) or off(false).
5	TextField	is basically used to get the input from the user in the form of text. <code>javafx.scene.control.TextField</code> represents TextField
6	PasswordField	is used to get the user's password. Anything typed in it, is not visible to anyone.
7	HyperLink	used to refer any of the webpage through application. represented by the class <code>javafx.scene.control.HyperLink</code>
8	Slider	is used to move a slider over the range of values to select one of them.
9	ProgressBar	is used to show the work progress to the user. And is by the class <code>javafx.scene.control.ProgressBar</code> .

10	ProgressIndicator	Instead of showing the analogue progress to the user, it shows the digital progress so that the user may know the amount of work done in percentage.
11	ScrollBar	is used to scroll down the application pages.
12	Menu	JavaFX provides a Menu class to implement menus. Menu is the main component of any application.
13	ToolTip	is used to provide hint to the user about any component. It is mainly used to provide hints about the text fields or password fields being used in the application.

- **javafx.scene.control.Label** class represents label control. and is the component that is used to place any text information on the screen.
- It is mainly used to describe the purpose of the other components to the user.

Constructors:

Label(): creates an empty Label
Label(String text): creates Label with the supplied text
Label(String text, Node graphics): creates Label with the supplied text and graphics

Example : Label to Print Text("Dr. Pradeep .K .V")

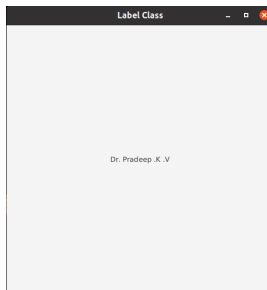
```
package KVP;

import javafx.application.Application;
import javafx.scene.Scene;    import javafx.scene.control.Label;
import javafx.stage.Stage;    import javafx.scene.layout.StackPane;

public class LabelTest extends Application {
    @Override
    public void start(Stage primaryStage) throws Exception {
        // TODO Auto-generated method stub
    }
}
```



```
Label my_label=new Label("Dr. Pradeep .K .V");
StackPane root = new StackPane();
Scene scene=new Scene(root,500,500);
root.getChildren().add(my_label);
primaryStage.setScene(scene);
primaryStage.setTitle("Label Class");
primaryStage.show();
}
public static void main(String[] args) { launch(args); }
}
```



```
package KVP;

import java.io.FileInputStream;           import java.io.FileNotFoundException;
import javafx.application.Application;   import javafx.scene.Scene;
import javafx.scene.control.Label;        import javafx.scene.image.Image;
import javafx.scene.image.ImageView;      import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class LabelImage extends Application {
    @Override
    public void start(Stage primaryStage) throws FileNotFoundException {
        StackPane root = new StackPane();

        FileInputStream input= new FileInputStream(
                                "/home/pradeep/Pictures/KVP3.jpg");
        Image image = new Image(input);
        ImageView imageview=new ImageView(image);

        Label my_label=new Label("Dr. Pradeep K V", imageview);
        Scene scene=new Scene(root,500,500);
        root.getChildren().add(my_label);

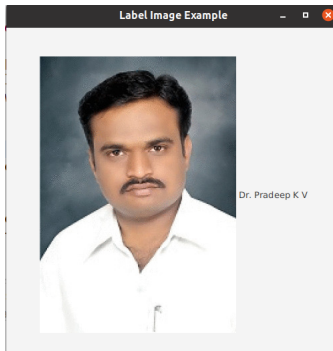
        primaryStage.setScene(scene);
        primaryStage.setTitle("Label Image Example");
    }
}
```



Example-2 : JavaFX Label with Image II



```
        primaryStage.show();  
    }  
    public static void main(String[] args) { launch(args); }  
}
```



- It is represented by **javafx.scene.control.Button** class.
- It is a component that can control the behaviour of the Application.
- An event is generated whenever the button gets clicked.

- **Create** : It can be created by instantiating **Button** class.

```
Button btn = new Button("My Button");
```

- **Adding** a Button to the scene : To visualize the button on the screen

```
Button btn=new Button("My Button");  
Scene scene=new Scene(root,300,300);  
root.getChildren().add(btn);
```

- **Setting** the **Text** of the Button (2 Ways)
 - Passing the text into the class constructor
 - By calling `setText("text")` method

- **Wrapping** Button Text : Text into Multiple Line (if Long)

```
btn.setTextWrapText(true);
```



- **Setting** the **image** on the button : Button class contains a constructor which can accept graphics along with the text displayed on the button.
Button class also provides an instance method named **setGraphic()**, to pass the image view object.
- Button **Action** : Button class provides **setOnAction()** method, used to set the action for the button click event. An object of the anonymous class implementing the **handle()** method, is passed in this method as a parameter.
- Button **Effects** : can apply the effects, using **javafx.scene.effect** package

```
FileInputStream input=new FileInputStream("
                                /home/pradeep/Pictures/KVP3.jpg");
Image image = new Image(input);
ImageView img=new ImageView(image);
```

```
// Button btn=new Button("MyButtonImage",img); // Constructor
Button btn=new Button(); // Constructor
btn.setGraphic(img); // Placing Image on Button
btn.setTextWrapText(true); // WrapText True
```



```
btn.setOnAction(new EventHandler<ActionEvent>() {  
    @Override  
    public void handle(ActionEvent arg0) {  
        // TODO Auto-generated method stub  
        System.out.println("Button clicked");  
    }  
} );
```

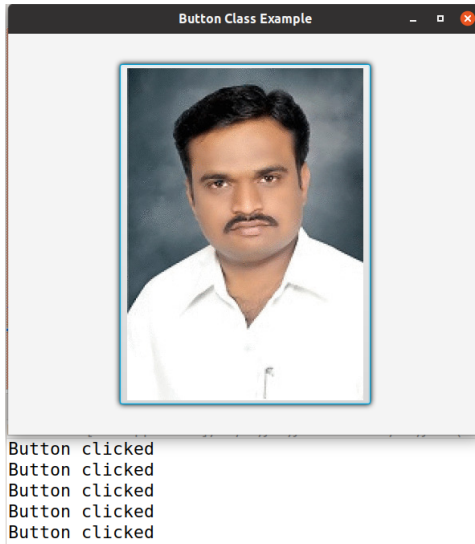
Example : JavaFX Buttons, Image and Action Listener

```
package KVP;  
  
import java.io.FileInputStream;  
import javafx.application.Application; import javafx.event.ActionEvent;  
import javafx.event.EventHandler;      import javafx.scene.Scene;  
import javafx.scene.control.Button;    import javafx.scene.effect.DropShadow;  
import javafx.scene.image.Image;       import javafx.scene.image.ImageView;  
import javafx.scene.layout.StackPane;  import javafx.stage.Stage;
```




```
public class ButtonTest extends Application {
    public void start(Stage primaryStage) throws Exception {
        FileInputStream input=new FileInputStream(
            "/home/pradeep/Pictures/KVP3.jpg");
        Image image = new Image(input);  ImageView img=new ImageView(image);
        DropShadow shadow = new DropShadow();
        StackPane root = new StackPane();
        Button btn=new Button();          btn.setEffect(shadow);
        btn.setGraphic(img);              btn.setWrapText(true);

        btn.setOnAction(new EventHandler<ActionEvent>() {
            public void handle(ActionEvent arg0) {
                System.out.println("Button clicked");
            }
        });
        Scene scene=new Scene(root,500,500);
        root.getChildren().add(btn);
        primaryStage.setScene(scene);
        primaryStage.setTitle("Button Class Example");
        primaryStage.show();
    }
    public static void main(String[] args) { launch(args); }
}
```



- It is used to provide various options to the user. The user can only choose one option among all (**Selected/Deselected**)
- It can be used in a scenario of multiple choice questions in the quiz where only one option needs to be chosen by the student.

```
package KVP;

import javafx.application.Application;
import javafx.scene.Scene;   import javafx.scene.control.Label;
import javafx.scene.control.RadioButton;
import javafx.scene.control.ToggleGroup;
import javafx.scene.layout.VBox;      import javafx.stage.Stage;

public class RadioButtonTest extends Application {
    public static void main(String[] args) { launch(args); }
    public void start(Stage primaryStage) throws Exception {

        ToggleGroup group = new ToggleGroup();
        RadioButton button1 = new RadioButton("Mango");
        RadioButton button2 = new RadioButton("Apple");
```



```
RadioButton button3 = new RadioButton("Banana");
RadioButton button4 = new RadioButton("Orange");

Label l = new Label("Select any One Fruit ");

button1.setToggleGroup(group);
button2.setToggleGroup(group);
button3.setToggleGroup(group);
button4.setToggleGroup(group);
VBox root=new VBox();
root.setSpacing(10);
root.getChildren().addAll(l,button1,button2,button3,button4);

Scene scene=new Scene(root,400,300);
primaryStage.setScene(scene);
primaryStage.setTitle("Radio Button Example");
primaryStage.show();
}
}
```



- It is used to provide more than one choices to the user.
- It can be used in a scenario where the user is prompted to select more than one option or the user wants to select multiple options.
- Instantiate **javafx.scene.control.CheckBox** class to implement CheckBox.

- **Create :**

```
CheckBox checkbox = new CheckBox();           // Constructor
CheckBox checkbox = new CheckBox("Label Name"); // Parametrized
```

- **Example**

```
package KVP;

import javafx.application.Application; import javafx.scene.Scene;
import javafx.scene.control.CheckBox;  import javafx.scene.control.Label;
import javafx.scene.layout.VBox;      import javafx.stage.Stage;
public class MyCheckBox extends Application {
    public static void main(String[] args) { launch(args); }

    public void start(Stage primaryStage) throws Exception {
        Label l = new Label("Tick you Favorite Player ");

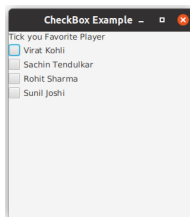
        CheckBox c1 = new CheckBox("Virat Kohli");
```



```
CheckBox c2 = new CheckBox("Sachin Tendulkar");
CheckBox c3 = new CheckBox("Rohit Sharma");
CheckBox c4 = new CheckBox("Sunil Joshi");

VBox root = new VBox();
root.getChildren().addAll(c1,c2,c3,c4);
root.setSpacing(5);

Scene scene=new Scene(root,200,200);
primaryStage.setScene(scene);
primaryStage.setTitle("CheckBox Example");
primaryStage.show();
}
}
```



- It is basically used to get the input from the user in the form of text.
- **javafx.scene.control.TextField** represents TextField.
- It provides various methods to deal with textfields in JavaFX.
- It can be created by instantiating **TextField** class.
- **Create :**

```
TextField tf1 = new TextField();  
TextField tf2 = new TextField(Integer Value);
```

- **Example-1 : Name and E-Mail :**

```
package KVP;  
import javafx.application.Application;  
import javafx.geometry.Insets;      import javafx.scene.Scene;  
import javafx.scene.control.Label;  import javafx.scene.control.TextField;  
import javafx.scene.layout.HBox;    import javafx.scene.paint.Color;  
import javafx.stage.Stage;  
  
public class TextFieldTest extends Application {  
    public void start(Stage stage) {  
  
        TextField textField1 = new TextField("Enter Name");  
        TextField textField2 = new TextField("Enter E-mail");
```



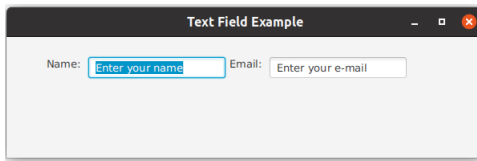

```

Label label1 = new Label("Name: ");
Label label2 = new Label("Email: ");

HBox box = new HBox(5);           //Adding labels for nodes

box.setPadding(new Insets(25, 5 , 5, 50));
box.getChildren().addAll(label1, textField1, label2, textField2);

//Setting the stage
Scene scene = new Scene(box, 595, 150, Color.BEIGE);
stage.setTitle("Text Field Example");
stage.setScene(scene);           stage.show();
}
public static void main(String args[]){  launch(args);  }
}
    
```



- Example-2 : User Name and Password :**



```
package KVP;

import javafx.application.Application;
import javafx.scene.Scene;           import javafx.scene.control.Button;
import javafx.scene.control.Label;    import javafx.scene.control.TextField;
import javafx.scene.layout.GridPane; import javafx.stage.Stage;

public class Login extends Application {

    public static void main(String[] args) { launch(args); }

    public void start(Stage primaryStage) throws Exception {
        Label user_id=new Label("User ID");
        Label password = new Label("Password");

        TextField tf1=new TextField();
        TextField tf2=new TextField();

        Button b = new Button("Submit");
        b.setOnAction(e->System.out.println("You entered: User_ID: "
                                             +tf1.getText()+" "+"Password: "
                                             +tf2.getText()));

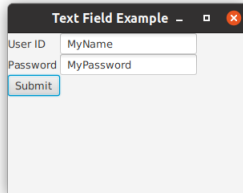
        GridPane root = new GridPane();
        root.addRow(0, user_id, tf1);
```



```
root.addRow(1, password, tf2);
root.addRow(2, b);

Scene scene=new Scene(root,300,200);
primaryStage.setScene(scene);
primaryStage.setTitle("Text Field Example");
primaryStage.show();
}
}
```

You entered: User_ID: MyNamePassword: MyPassword



- Typing a password in a text field is not secure for the user. And The Application must use a specific component to get the password from the user.
- It can be created by instantiating **javafx.scene.control.PasswordField** class.
- It contains a method named as **setPromptText()** for showing a prompt text to the user in password field.
- The data written in the it is retrieved by **getText()** method.
- **Example-1 : User Name and Password :**

```
package KVP;

import javafx.application.Application;   import javafx.scene.Scene;
import javafx.scene.control.Button;       import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.scene.control.PasswordField;
import javafx.scene.layout.GridPane;     import javafx.stage.Stage;

public class LoginPass extends Application {
    public static void main(String[] args) { launch(args); }
    public void start(Stage primaryStage) throws Exception {
```



```
Label user_id=new Label("User ID");
Label password = new Label("Password");

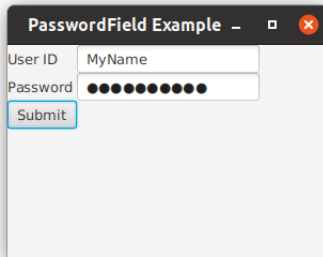
TextField tf=new TextField();
PasswordField pf=new PasswordField();
pf.setPromptText("Enter Password");

Button b = new Button("Submit");
b.setOnAction(e->System.out.println("User_ID: "+tf.getText()+" "+
                                     "Password: "+pf.getText()));

GridPane root = new GridPane();
root.addRow(0, user_id, tf);
root.addRow(1, password, pf);
root.addRow(5, b);

Scene scene=new Scene(root,300,200);
primaryStage.setScene(scene);
primaryStage.setTitle("PasswordField Example");
primaryStage.show();
}
}
```

User_ID: MyName Password: MyPassword



The screenshot shows a JavaFX window titled "PasswordField Example". Inside the window, there are two text input fields. The first field is labeled "User ID" and contains the text "MyName". The second field is labeled "Password" and contains masked characters (dots). Below these fields is a button labeled "Submit".

- It is the main component of a any application.
- In JavaFX, **javafx.scene.control.Menu** class provides all the methods to deal with menus. This class needs to be instantiated to create a Menu.

- **Example :**

```
package KVP;

import javafx.application.Application; import javafx.scene.Scene;
import javafx.scene.layout.BorderPane; import javafx.scene.control.*;
import javafx.stage.Stage;

public class MenuItems extends Application {
    public static void main(String[] args) { launch(args); }
    public void start(Stage primaryStage) throws Exception {
        BorderPane root = new BorderPane();
        Scene scene = new Scene(root,300,300);
        MenuBar menubar = new MenuBar();

        Menu FileMenu = new Menu("File");
        MenuItem filemenu1=new MenuItem("new");
        MenuItem filemenu2=new MenuItem("Save");
        MenuItem filemenu3=new MenuItem("Exit");

        Menu EditMenu=new Menu("Edit");
```

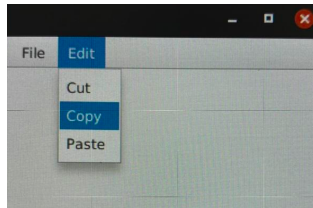
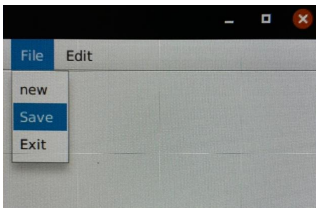


```
MenuItem EditMenu1=new MenuItem("Cut");
MenuItem EditMenu2=new MenuItem("Copy");
MenuItem EditMenu3=new MenuItem("Paste");

EditMenu.getItems().addAll(EditMenu1,EditMenu2,EditMenu3);
root.setTop(menubar);
FileMenu.getItems().addAll(filemenu1,filemenu2,filemenu3);
menubar.getMenus().addAll(FileMenu,EditMenu);

primaryStage.setScene(scene);
primaryStage.show();

}
```



- It is used to provide hint to the user about any component. It is mainly used to provide hints about the **text fields** or **password fields** being used in the application.
- It can be created by instantiating **javafx.scene.control.ToolTip** class.

- **Example :**

```
package KVP;

import javafx.application.Application;      import javafx.scene.Scene;
import javafx.scene.control.PasswordField;
import javafx.scene.control.ToolTip;
import javafx.scene.layout.StackPane;      import javafx.stage.Stage;

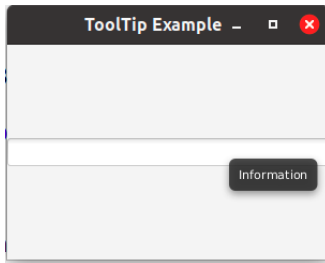
public class toolTip extends Application {

    public void start(Stage primaryStage) throws Exception {
        PasswordField pf = new PasswordField();
        Tooltip tool=new Tooltip();
        StackPane root = new StackPane();
        tool.setText("Information");
        pf.setTooltip(tool);
        root.getChildren().add(pf);
    }
}
```



```
Scene scene = new Scene(root,300,200);
primaryStage.setScene(scene);
primaryStage.setTitle("ToolTip Example");
primaryStage.show();
}

public static void main(String[] args) { launch(args); }
```



- **Progress Bar** is used to show the work progress to the user.
- It is represented by `javafx.scene.control.ProgressBar`.
- `setProgress()` is used to set the Progress.
- **Example :**

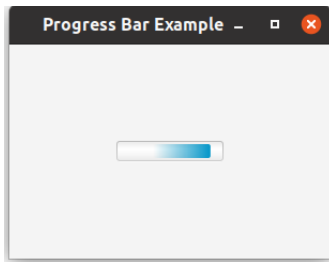
```
package KVP;

import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.ProgressBar;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class progressBar extends Application {
    public void start(Stage primaryStage) throws Exception {
        StackPane root = new StackPane();
        ProgressBar pb = new ProgressBar();
        // progress.setProgress(0.5f);
        root.getChildren().add(pb);
    }
}
```

```
Scene scene = new Scene(root,300,200);
primaryStage.setScene(scene);
primaryStage.setTitle("Progress Bar Example");
primaryStage.show();
}

public static void main(String[] args) { launch(args); }
```



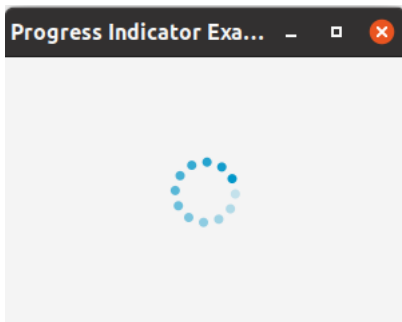
- It is similar to Progress Bar to some extent.
- Instead of showing the analogue progress to the user, it shows the digital progress so that the user may know the amount of work done in percentage.
- It is represented by **javafx.scene.control.ProgressIndicator** class. This class needs to be instantiated in order to create Progress Indicator.
- **Example :**

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.ProgressIndicator;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class progressIndicator extends Application{
    public void start(Stage primaryStage) throws Exception {
        ProgressIndicator PI=new ProgressIndicator();
        //    PI.setProgress(0.75f);
        StackPane root = new StackPane(); root.getChildren().add(PI);
        Scene scene = new Scene(root,300,200);
        primaryStage.setScene(scene);
        primaryStage.setTitle("Progress Indicator Example");
        primaryStage.show();
    }
}
```



```
}  
public static void main(String[] args) { launch(args); }  
}
```



- Is used to provide the text based information on the interface of application.
- JavaFX library provides a class called **javafx.scene.text.Text**. It offers different methods for altering different text properties.
- Properties :**

Property	Description and Setter Methods
boundstype	setBoundsType (TextBoundsType value) : It determines the way in which the bounds of the text is being calculated.
font	setFont (Font value)Font of the text.
fontsmoothingType	setFontSmoothingType (FontSmoothingType value) Defines the requested smoothing type for the font.
linespacing	setLineSpacing (double spacing) Vertical space in pixels between the lines. It is double type property.

strikethrough	setStrikeThrough (boolean value) This is a boolean type property. and can put a line through the text by setting this property to true.
textalignment	setTextAlignment (TextAlignment value) Horizontal Text alignment
textorigin	setTextOrigin (VPos value) Origin of text coordinate system in local coordinate system.
text	setText (String value) It defines the text string which is to be displayed.
underline	setUnderLine (boolean value) It is a boolean type property. Text can be underlined by setting this to true .
wrappingwidth	setWrappingWidth (double value) Width limit for the text from where the text is to be wrapped. It is a double type property.
x	setX (double value)X coordinate of the text
y	setY (double value)Y coordinate of the text

- **Creating a text node** : The class **javafx.scene.text.Text** needs to be instantiated in order to create the text node. **setText(string)** to set the string as a text for the text class object.
- **Font** and **position** of the Text : JavaFX enables us to apply various fonts to the text nodes. The setter method **setFont()** is used to set the property font of the Text. The class Font belongs the package **javafx.scene.text**
- Applying **Stroke** and **Color** to Text : Stroke means the padding at the boundary of the text. **javafx.scene.text.Text** class provides a method named **setStroke()** which accepts the Paint class object as an argument.
- To set the **width** of the stroke by passing a width value of double type into **setStrokeWidth()** method.
- To set the color of the Text, **javafx.scene.text.Text** class provides another method named **setFill()**.
- Applying **Decorations** to the text : The decorations to the text by setting the properties strikethrough and underline of **javafx.scene.text.Text** class.

```
// Create a Text Object
```

```
Text <text_Object> = new Text();
```

```
text.setText(<string-text>);
```

```
// setFont()
```

```
<text_object>.setFont(Font.font(<String font_family>,  
                                <FontWeight>, <FontPosture>, <FontSize>)
```

1. Family: it represents the family of the font.
It is of string type and should be an appropriate font family present in the system.
2. Weight: **this** Font **class property** is **for** the weight of the font.
There are 9 values which can be used as the font weight.
The values are FontWeight.**BLACK**, BOLD, EXTRA_BOLD, EXTRA_LIGHT, LIGHT, MEDIUM, NORMAL, SEMI_BOLD, THIN.
3. Posture: **this** Font **class property** represents the posture of the font.
It can be either FontPosture.**ITALIC** or FontPosture.**REGULAR**.
4. Size: **this** is a **double** type property.
It is used to set the size of the font.



//Decoration of the Text

//pass true to put a line across the text

`<TextObject>.setStrikeThrough(Boolean value)`

//pass true to underline the text

`<TextObject>.setUnderLine(Boolean value)`

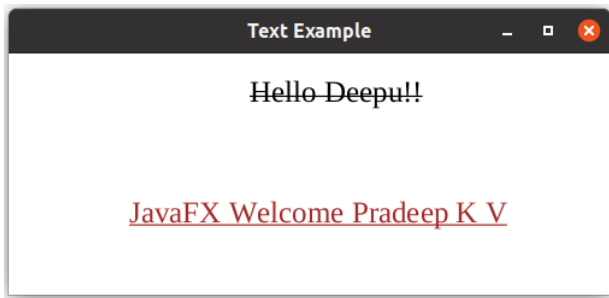
```
package KVP;
import javafx.application.Application;      import javafx.scene.Group;
import javafx.scene.Scene;      import javafx.scene.text.Font;
import javafx.scene.text.Text;  import javafx.stage.Stage;
public class FontText extends Application {
    public void start (Stage primaryStage) throws Exception {
        Text text = new Text(); {
            text.setX(200);    text.setY(40);
            text.setFont(Font.font("Liberation Serif", 25));
            text.setText("Hello Deepu!!");
            text.setStrikethrough(true);

            Text text1 =new Text();
            text1.setX(100);    text1.setY(140);
            text1.setText ( "JavaFX Welcome Pradeep K V");
            text1.setFont(Font.font ("Libera tion Serif", 25));
            text1.setFill(Color.BROWN);
            text1.setUnderline(true);

            Group root = new Group();
            Scene scene = new Scene(root,500,200);
            root.getChildren().addAll(text, text1);
            primaryStage.setScene(scene);
            primaryStage.setTitle("Text Example");
```



```
        primaryStage.show();  
    }  
    public static void main (String [] args) { launch(args); }  
}
```



- **JavaFX 2D Shapes** : Shape properties, Line, Rectangle, Ellipse, Arc, Circle, Polygons, Cubic Curve, Quad Curve, Color and Gradient Color.
- **JavaFX Effects** : ColorAdjust, ColorInput, ImageInput, Blend, Bloom, Glow, BoxBlur, GaussianBlur, MotionBlur, Reflection, SepiaTone, Shadow, DropShadow, InnerShadow, Lighting, Light.Distant, Light.Spot, Light.Point.
- **JavaFX Transformation** : Transformation, Translation, Rotation, Scaling, Shearing, Multiple Transformations.
- **JavaFX Animations** : Rotate/Scale/Translate Transition, Fade Transition, Fill Transition, Stroke Transition, Sequential Transition, Parallel Transition, Pause Transition, Path Transition.
- **JavaFX 3D Shapes** : 3D Object Properties, Box, Cylinder, Sphere
- **JavaFX Layouts** : Layouts, BorderPane, HBox, VBox, StackPane, GridPane, FlowPane.
- **JavaFX Charts** : Pie Chart, Line Chart, Area Chart, Bar Chart, Bubble Chart, Scatter Chart, Stacked Area Chart, Stacked Bar Chart.
- **Media with JavaFX** : Playing Audio and Video.
- **JavaFX Event Handling** : Convenience methods, Event Filters, Event Handlers

- <https://www.tutorialspoint.com/javafx/index.html>
- <https://docs.oracle.com/javase/tutorial/>
- <http://tutorials.jenkov.com/javafx/index.html>
- <https://www.javatpoint.com/features-of-java>
- <http://www.java2novice.com/java-fundamentals/>
- http://archive.oreilly.com/o_reillyschool/courses/java2/
- <https://docs.oracle.com/javase/7/do>
- <https://www.javatpcs/api/point.com/>
- <https://beginnersbook.com/java-tutorial-for-beginners-with-examples/>
- <https://www.javaworld.com/>
- <https://www.geeksforgeeks.org/>

Thanks



*Thank
you!*

- **GUI applications, Web applications** can all be created using JavaFX.
- An Event occurs, When a user interacts with the application (nodes).
- For **Example**, **clicking a button**, **moving the mouse**, **entering a character** via keyboard, **selecting an item from a list**, and **scrolling the page** are all activities that cause an event to occur.

Event Types :

- **Foreground** : are mainly occurred due to the direct interaction of the user with the application. Such as **clicking the button**, **pressing a key**, **selecting an item from the list**, **scrolling the page**, etc.
- **Background** : doesn't require the user's interaction with the application. These events are mainly occurred to the **OS interrupts**, **failure**, **Operation completion**, etc.

JavaFX allows you to handle a wide range of events. The base class for an event is the class **Event** from the package **javafx.event**.

- **Mouse** Event : is an i/p event that occurs when a mouse is clicked and is represented by the class named **MouseEvent**. It includes actions like **mouse clicked / pressed / released / moved / target (Enter/Exit)** etc.
- **Key** Event : is an i/p event that indicates the key stroke occurred on a node. It is represented by the class named **KeyEvent**. It includes actions like **key pressed / released / typed**.
- **Drag** Event : is an i/p event which occurs when the mouse is dragged and is represented by the class named **DragEvent**. It includes actions like drag entered, drag dropped, drag entered target, drag exited target, drag over, etc.
- **Window** Event : is an event related to window **showing/hiding** actions and is represented by the class named **WindowEvent**.

Event Handling

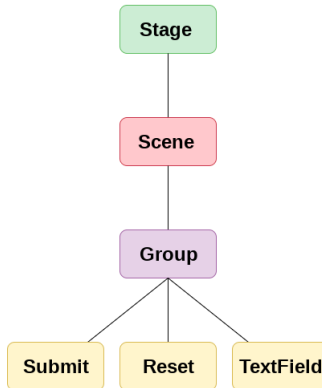
The mechanism that controls the event and decides what should happen if an event occurs is known as event handling. The code that is known as an event handler is conducted when an event happens in this mechanism.

JavaFX provides handlers and filters to handle events. Every event has :

- **Target** : The node on which an event occurred. It can be a **window**, **scene**, and a **node**.
- **Source** : The source from which the event is generated. In the above scenario, mouse is the source of the event.
- **Type** : Type of the occurred event; in case of mouse event - **mouse pressed/released** are the type of events.

Whenever an event is generated, JavaFX undergoes the following phases.

- 1 **Route Construction** : An **Event Dispatch Chain** is created in order to determine the default route of the event, whenever it is generated. This chain contains the path from the **stage** to the **Node** on which the event is generated.



- ② **Event Capturing Phase** : Once the **Event Dispatch Chain** is created, the event is dispatched from the source node of the event.
All the nodes are traversed by the event from top to bottom. If the event filter is registered with any of these nodes, then it will be executed.
If any of the nodes are not registered with the event filter then the event is transferred to the target node. The target node processes the event in that case.
- ③ **Event Bubbling** : Once the event is processed by the target node or by any of the registered filter, the event traverses all the nodes again from the bottom to the stage node. If any of these nodes are registered with the event filter, then it will get executed otherwise the process gets completed.
- ④ **Event Handlers and Filters** : Event Handlers and filters contains application logic to process an event. A node can be registered to more than one Event Filters. The interface **javafx.event.EventHandler** must be implemented by all the event handlers and filters.

- JavaFX provides convenience methods for handling events within JavaFX application. They make it simple to create and register event handlers to respond to **KeyEvent**, **MouseEvent**, **Action Event**, **Drag & Drop Events**, and many other types of events.
- The **Node** class contains a number of Event Handler properties that can be assigned to User-Defined Event Handlers via the setter methods defined in the class.
- Setting the Node class's EventHandler properties to user-defined event handlers automatically registers the handlers to receive the corresponding event types.

The **EventHandler** properties, as well as their setter methods (convenience methods), are described in the table below.

EventHandler Property	Description
onContextMenuRequested	setOnContextMenuRequested (EventHandler value) This is a ContextMenuEvent EventHandler. And is assigned to a function which is to be called when the context menu is requested on the node.

onDragDetected	setOnDragDetected (EventHandler value) This is a MouseEvent EventHandler. And it indicates a function which is to be called when the drag gesture is detected.
onDragDone	setOnDragDone (EventHandler value) This is a DragEvent Event Handler
onDragDropped	setOnDragDropped (EventHandler value) This is of the type EventHandler of DragEvent. This is assigned to a function which is to be called when the mouse is released during a drag and drop operation.
onDragEntered	setOnDragEntered (EventHandler value) This is of the type EventHandler of DragEvent. This is assigned to a function which is to be called when the drag gesture enters the node.
onDragExited	setOnDragExited (EventHandler value) This is of the type EventHandler of DragEvent. This is assigned to a function which is to be called when the drag gesture exits the node.

onDragOver	setOnDragOver (EventHandler value) This is of the type EventHandler of DragEvent. This is assigned to a function which is to be called when the drag gesture progresses within the node.
onInputMethodTextChanged	setOnInputMethodTextChanged (EventHandler value) This is of the type EventHandler of InputMethodEvent. This is assigned to a function which is to be called when the Node has focus and the input method text has changed.
onKeyPressed	setOnKeyPressed (EventHandler value) This is of the type EventHandler of KeyEvent. This is assigned to a function which is to be called when the Node has focus and key is pressed.
onKeyReleased	setOnKeyReleased (EventHandler value) This is of the type EventHandler of KeyEvent. This is assigned to a function which is to be called when the Node has focus and key is released.

onKeyTyped	setOnKeyTyped (EventHandler value) This is of the type EventHandler of KeyEvent. This is assigned to a function which is to be called when the Node has focus and key is typed.
onMouseClicked	setOnMouseClicked (EventHandler value) This is of the type EventHandler of MouseEvent. This is assigned to a function which is to be called when the mouse button is clicked on the node.
onMouseDragEntered	setOnMouseDragEntered (EventHandler value) This is of the type EventHandler of MouseDragEvent. This is assigned to a function which is to be called when a full press drag release gesture enters the node.
onMouseDragExited	setOnMouseDragExited (EventHandler value) This is of the type EventHandler of MouseDragEvent. This is assigned to a function which is to be called when a full press drag release gesture exits the node.
onMouseDragged	setOnMouseDragged (EventHandler value) This is of the type EventHandler of MouseDragEvent. This is assigned to a function which is to be called when the mouse button is pressed and dragged on the node.

onMouseDown	setOnMouseDown (EventHandler value) This is of the type EventHandler of MouseEvent. This is assigned to a function which is to be called when a full press drag release gesture progresses within the node.
onMouseDownOver	setOnMouseDownOver (EventHandler value) This is of the type EventHandler of MouseEvent. This is assigned to a function which is to be called when a full press drag release gesture progresses within the node.
onMouseDownReleased	setOnMouseDownReleased (EventHandler value) This is of the type EventHandler of MouseEvent. This is assigned to a function which is to be called when a full press drag release gesture ends within the node.
onMouseEntered	setOnMouseEntered (EventHandler value) This is of the type EventHandler of MouseEvent. This is assigned to a function which is to be called when the mouse enters the node.
onMouseExited	setOnMouseExited (EventHandler value) This is of the type EventHandler of MouseEvent. This is assigned to a function which is to be called when the mouse exits the node.
onMouseMoved	setOnMouseMoved (EventHandler value) This is of the type EventHandler of MouseEvent. This is assigned to a function which is to be called when the mouse moves within the node and no button has been pushed.

onMousePressed	setOnMousePressed (EventHandler value) This is of the type EventHandler of MouseEvent. This is assigned to a function which is to be called when the mouse button is pressed on the node.
onMouseReleased	setOnMouseReleased (EventHandler value) This is of the type EventHandler of MouseEvent. This is assigned to a function which is to be called when the mouse button is released.
onRotate	setOnRotate (EventHandler value) This is of the type EventHandler of RotateEvent. This is assigned to a function which is to be called when the rotation action is performed.
onRotationFinished	setOnRotationFinished (EventHandler value) This is of the type EventHandler of RotateEvent. This is assigned to a function which is to be called when the rotation gesture ends.
onRotationStarted	setOnRotationStarted (EventHandler value) This is of the type EventHandler of RotateEvent. This is assigned to a function which is to be called when the rotation gesture is first detected.

onScrollFinished	setOnScrollFinished (EventHandler value) This is of the type EventHandler of ScrollEvent. This is assigned to a function which is to be called when the scroll gesture ends.
onScroll	setOnScroll (EventHandler value) This is of the type EventHandler of ScrollEvent. This is assigned to a function which is to be called when the scroll action is performed.
onScrollStarted	setOnScrollStarted (EventHandler value) This is of the type EventHandler of ScrollEvent. This is assigned to a function which is to be called when the scrolling gesture is detected.
onSwipeDown	setOnSwipeDown (EventHandler value) This is of the type EventHandler of SwipeEvent. This is assigned to a function which is to be called when the downwards swipe happens.
onSwipeUP	setOnSwipeUP (EventHandler value) This is of the type EventHandler of SwipeEvent. This is assigned to a function which is to be called when the upwards swipe happens.

onSwipeLeft	setOnSwipeLeft (EventHandler value) This is of the type EventHandler of SwipeEvent. This is assigned to a function which is to be called when the leftwards swipe happens.
onSwipeRight	setOnSwipeRight (EventHandler value) This is of the type EventHandler of SwipeEvent. This is assigned to a function which is to be called when the Rightwards swipe happens.
onTouchMoved	setOnTouchMoved (EventHandler value) This is of the type EventHandler of TouchEvent. This is assigned to a function which is to be called when the touch point is moved.
onTouchReleased	setOnTouchReleased (EventHandler value) This is of the type EventHandler of TouchEvent. This is assigned to a function which is to be called when the touch point is released.
onTouchStationary	setOnTouchStationary (EventHandler value) This is of the type EventHandler of TouchEvent. This is assigned to a function which is to be called when the touch point is pressed and stays still

onZoomFinished	setOnZoomFinished (EventHandler value) This is of the type EventHandler of ZoomEvent. This is assigned to a function which is to be called when the zooming gesture ends.
onZoom	setOnZoom (EventHandler value) This is of the type EventHandler of ZoomEvent. This is assigned to a function which is to be called when the zooming gesture is performed.
onZoomStarted	setOnZoomStarted (EventHandler value) This is of the type EventHandler of ZoomEvent. This is assigned to a function which is to be called when the zooming gesture is detected.

```
package application;
import javafx.animation.RotateTransition;
import javafx.application.Application;   import javafx.scene.Group;
import javafx.event.ActionEvent;         import javafx.event.EventHandler;
import javafx.scene.shape.Rectangle;     import javafx.scene.Scene;
import javafx.scene.control.Button;       import javafx.stage.Stage;
import javafx.scene.paint.Color;          import javafx.util.Duration;

public class setOnAction extends Application {
    public static void main(String[] args) { Application.launch(args); }

    @Override
    public void start(Stage primaryStage) {
        Rectangle rect = new Rectangle(100,100,120,120); // Creating Rectangle

        // Setting Stroke and colour for the rectangle
        rect.setFill(Color.BLUE);      rect.setStroke(Color.BLACK);

        // Instantiating RotateTransition class
        RotateTransition rotate = new RotateTransition();

        //Setting properties for the Rotate Transition class
        rotate.setAutoReverse(false);  rotate.setByAngle(360);
        rotate.setDuration(Duration.millis(500));
```

Example : setOnAction() II



```
rotate.setNode(rect);                rotate.setCycleCount(500);

Button btn = new Button(); //Creating the play button

//Setting properties for the play button
btn.setText("Play");                btn.setTranslateX(100);
btn.setTranslateY(250);

//defining the convenience method to register the event handler
//to handle the Action event.
btn.setOnAction(new EventHandler<ActionEvent>() {
    public void handle(ActionEvent event) { rotate.play(); }
});

Button btn1 = new Button("Pause"); //Creating the pause button

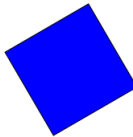
//Setting properties for the pause button
btn1.setTranslateX(160);            btn1.setTranslateY(250);

//Handling event for the pause button click event
btn1.setOnAction(new EventHandler<ActionEvent>() {
    @Override
    public void handle(ActionEvent arg0) { rotate.pause(); }
});
```




```
Group root = new Group();           //Configuring group and scene
Scene scene = new Scene(root, 400, 350);
root.getChildren().addAll(btn,rect,btn1);
primaryStage.setScene(scene);
primaryStage.setTitle("Handling Events");
primaryStage.show();
}
```

Handling Events



Play

Pause

Example : setOnKeyEvent() I



```
package application;

import javafx.application.Application; import javafx.stage.Stage;
import javafx.event.EventHandler;      import javafx.scene.Group;
import javafx.scene.control.TextField; import javafx.scene.Scene;
import javafx.scene.input.KeyEvent;    import javafx.scene.paint.Color;

public class setOnKeyEvent extends Application{
    @Override
    public void start(Stage primaryStage) throws Exception {
        //Creating TextFields and setting position for them
        TextField tf1 = new TextField(); TextField tf2 = new TextField();
        tf1.setTranslateX(100);      tf1.setTranslateY(100);
        tf2.setTranslateX(300);      tf2.setTranslateY(100);

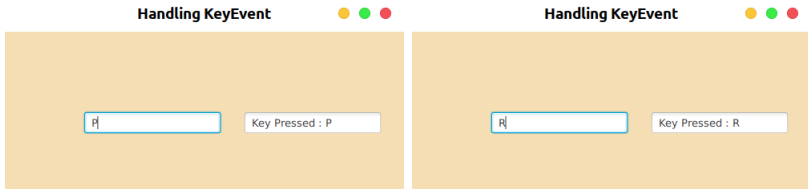
        //Handling KeyEvent for textfield 1
        tf1.setOnKeyPressed(new EventHandler<KeyEvent>() {
            @Override
            public void handle(KeyEvent key) {
                // TODO Auto-generated method stub
                tf2.setText("Key Pressed : "+" "+key.getText());
            }
        });
    }
}
```



Example : setOnKeyEvent() II



```
//setting group and scene
Group root = new Group();      root.getChildren().addAll(tf2,tf1);
Scene scene = new Scene(root,500,200,Color.WHEAT);
primaryStage.setScene(scene);
primaryStage.setTitle("Handling KeyEvent");
primaryStage.show();
}
public static void main(String[] args) { launch(args); }
```



- They are used to handle the events generated by **Keyboard Actions**, **Mouse Actions**, and many more source nodes.
- EH are used to handle the events in the Event bubbling phase. There can be more than one Event handlers for a single node and vice-versa.
- **Adding an Event Handler :**
 - EH must be registered for a node in order to process the events in the event bubbling phase.
 - EH is the implementation of the **EventHandler** interface.
 - The **handle()** method of the interface contains the logic which is executed when the event is triggered.
 - To register the EH, **addEventHandler()** is used in which two arguments are passed. One is **event type** and the other is **EventHandler** object.

Syntax :

```
node.addEventHandler(<EventType>,new EventHandler<Event-Type>() {  
    public void handle(<EventType> e) {  
        //handling code  
    }  
});
```

```
node.removeEventHandler(<EventType>,handler);
```



```
package application;
import javafx.animation.TranslateTransition; import javafx.scene.Group;
import javafx.application.Application;      import javafx.scene.Scene;
import javafx.event.EventHandler;            import javafx.stage.Stage;
import javafx.scene.control.Button;          import javafx.util.Duration;
import javafx.scene.input.MouseEvent;        import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;

public class JavaFXEventHandler extends Application{
    @Override
    public void start(Stage primaryStage) throws Exception {
        // TODO Auto-generated method stub
        //Creating Circle and setting the color and stroke in the circle
        Circle c = new Circle(100,100,50);
        c.setFill(Color.RED);          c.setStroke(Color.BLACK);

        //creating play button and setting coordinates for the button
        Button btn = new Button("Play");
        btn.setTranslateX(125);        btn.setTranslateY(200);

        // creating pause button and setting coordinate for the pause button
        Button btn1 = new Button("Pause");
        btn1.setTranslateX(175);       btn1.setTranslateY(200);
    }
}
```



```
//Instantiating TranslateTransition class to create the animation
TranslateTransition trans = new TranslateTransition();

//setting attributes for the TranslateTransition
trans.setAutoReverse(true);    trans.setByX(200);
trans.setCycleCount(100);      trans.setDuration(Duration.millis(500));
trans.setNode(c);

//Creating EventHandler
EventHandler<MouseEvent> handler = new EventHandler<MouseEvent>() {
    @Override
    public void handle(MouseEvent event) {
        //animation will be played when the play button is clicked
        if(event.getSource()==btn){    trans.play();    }
        //animation will be paused when the pause button is clicked
        if(event.getSource()==btn1){    trans.pause();    }
        event.consume();
    }
};

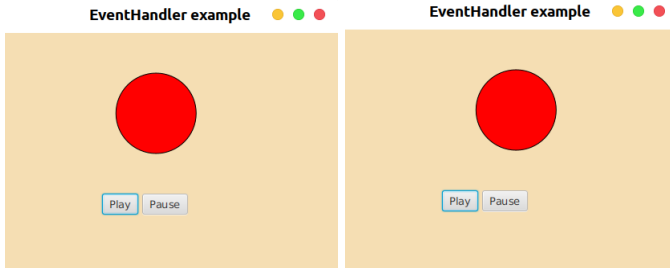
//Adding Handler for the play and pause button
btn.setOnMouseClicked(handler);    btn1.setOnMouseClicked(handler);

//Creating Group and scene
Group root = new Group();    root.getChildren().addAll(c,btn,btn1);
```

Example-1 : Event Handlers III



```
Scene scene = new Scene(root,420,300,Color.WHEAT);  
primaryStage.setScene(scene);  primaryStage.setTitle("EventHandler example")  
primaryStage.show();  
}  
public static void main(String[] args) {  launch(args);  }  
}
```



Thanks



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

*Thank
you!*

